Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(МАГИСТЕРСКАЯ РАБОТА)
по направлению подготовки
09.04.01 - «Информатика и вычислительная техника»

GRADUATION THESIS
(MASTER'S GRADUATION THESIS)
Field of Study
09.04.01 – «Software Engineering»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Software Engineering»

Тема / Topic	**Анализ объектных моделей в современных языках программирования/ Analysis of object models in modern programming languages**

Работу выполнил / Thesis is executed by	**Шалагин Егор Сергеевич / Shalagin Egor**	подпись / signature
Руководитель выпускной квалификационной работы / Supervisor of Graduation Thesis	**Зуев Евгений Александрович / Zouev Eugene**	подпись / signature
Консультанты / Consultants		подпись / signature

Иннополис, Innopolis, 2025

Contents

List of Tables

List of Figures

Abstract

This work conducts a systematic comparison of the object models in ten modern programming languages: C#, C++, C, Golang, Java, Python, Ruby, JavaScript, Scala, Smalltalk, and Zonnon. The research is motivated by the operational challenges faced by software engineers when choosing technologies given the significant differences in how object models are implemented in different language ecosystems. The main objective of the research is to identify the similarities, differences, strengths, and weaknesses of various object models, and based on this analysis, to develop proposals for an enhanced model. As a result of the study, the theoretical foundations of models are systematized, a detailed examination of their implementation in the selected languages is conducted, identifying application domains and common pitfalls, and an original model integrating best practices is proposed. The results obtained serve as a practical guide for developers when choosing a language and for software architects when designing object-oriented systems.

Chapter 1

Introduction

Object-oriented programming continues to be one of the fundamental paradigms in the software industry. However, despite decades of its existence, a unified standard for its implementation has never been established. Modern programming languages offer a variety of object models: from the strict and static ones in Java and C# to the prototype-based in JavaScript and the minimalist in Golang. This diversity, on one hand, provides developers with freedom of choice, but on the other, creates inconveniences when applying these approaches in various situations. Most of the research conducted focuses on individual languages or syntax, without offering a systematic understanding of the architectural principles underlying their object systems.

The goal of this master's thesis is to conduct a comprehensive comparative analysis of the object models of a number of modern programming languages and, based on it, to develop an original, enhanced model. To achieve this goal, the work addresses a series of tasks: it unveils the theoretical foundations of object models, analyzes the criticism directed at OOP, and conducts a detailed analysis of the implementation of models in languages such as C#, C++, C,

Golang, Java, Python, Ruby, JavaScript, Scala, Smalltalk, and Zonnon, identifying their advantages, disadvantages, and typical application areas. The practical section illustrates characteristic usage errors with specific examples. The final part presents the result of a comparative analysis with the identified common trends and fundamental differences, which serves as the basis for the proposal of the author's unified object model designed to mitigate the shortcomings of existing approaches identified during the study.

The object of the study is the object models themselves, and the subject is their architectural principles, similarities, differences, and practical aspects of implementation. The methodological foundation of the work consists of theoretical methods of analysis and systematization, as well as practical methods of comparative analysis based on the study of documentation and code writing. The theoretical significance of the work lies in the systematization of knowledge about object models, while the practical significance lies in the proposal of a new, more effective model for future developments. The structure of the work includes....

Chapter 2

Literature Review

2.1 The Concept and Components of the Object Model

The object model represents a fundamental framework that defines how a programming language represents and supports objects, classes, inheritance, encapsulation, polymorphism, and other related abstractions. According to commonly accepted terminology, the term object model has two interrelated meanings: the properties of objects within a particular programming language, technology, notation, or methodology that employs these objects, and a set of interfaces or classes through which a program can explore and manipulate specific aspects of its environment.Examples of object models include the Java Object Model, the Component Object Model (COM), and the Object Modeling Technique (OMT). Such object models are typically defined using concepts such as class, generic function, message, inheritance, polymorphism, and encapsulation.

Cardelli and Wegner in their work "On Understanding Types, Data Abstraction, and Polymorphism," defined object-oriented languages through three key requirements [1]. A language is considered object-oriented if it satisfies the following conditions:

- Support objects that represent data abstractions with an interface composed of named operations and an encapsulated internal state
- Objects in the language are associated with a specific object type
- Types may inherit attributes from their supertypes

These requirements were formulated in the form of a formula:

$$object - oriented = dataabstractions + objecttypes + typeinheritance$$

Booch, in his seminal work Object-Oriented Analysis and Design with Applications, describes the object model as a conceptual representation of the organized complexity of software systems [2].

> Booch (p. 40-41) concludes that the conceptual framework for anything object-oriented is the object model—a conceptual representation of the organized complexity of software. It consists of four major elements, i.e. abstraction, encapsulation, modularity, and hierarchy. In addition, the object model contains three minor element, i.e. typing, concurrency, and persistence.

This distinction between major and minor elements is fundamental: without the major elements, the model ceases to be object-oriented, whereas the minor elements are useful but not essential for supporting object orientation.

2.1.1 Major Components of the Object Model

The four major components of the object model form the fundamental basis on which object-oriented programming is built. Without these four elements, a programming language cannot achieve true object orientation, since they collectively determine how systems are broken down into components, organized, and managed at the conceptual level.

Abstraction

Abstraction is the process of extracting essential characteristics of an object or concept while suppressing unnecessary implementation details. As Liskov

[3] emphasizes, data abstraction is a valuable method for organizing programs to make them easier to modify and maintain.

According to Liskov's foundational work [3], a data abstraction is characterized by the separation of what an abstraction is from how it is implemented, such that implementations of the same abstraction can be substituted freely. The implementation of a data object is concerned with how that object is represented in memory, and this information is hidden from programs that use the abstraction by restricting those programs so that they cannot manipulate the representation directly, but instead must call the operations defined by the abstraction.

Encapsulation

Encapsulation is the bundling of data (attributes) and the methods that operate on that data into a single cohesive unit, typically implemented as a class. More importantly, encapsulation involves the enforcement of access controls that restrict direct manipulation of an object's internal state. This concept originated from Parnas's pioneering work on information hiding [4], which established the principle that modules should be designed such that they hide information from other modules—information that is likely to change. Parnas [4] articulated this principle by stating:

> Every module in the second decomposition is characterized by its knowledge of a design decision which it hides from all others. Its interface or definition was chosen to reveal as little as possible about its inner workings.

Encapsulation hides the internal representation of an object from external access, enabling interaction with the object exclusively through its public inter-

face—a set of methods specifically provided for this purpose. Besides providing data protection, encapsulation provides a stable interface that allows developers to refactor and optimize the internal implementation without affecting dependent code.

Modularity

The principle of modularity is designing software systems as sets of discrete, independent units or components that can be designed, implemented, and maintained separately. Parnas' [4] fundamental idea of modular decomposition: modules should be designed and implemented independently, be simple enough to be fully understandable, and allow changing the implementation of a module without affecting the behavior of other modules.

In object-oriented programming, modularity comes about by packaging classes and objects into coherent units with well-defined boundaries and minimal interdependencies. According to Liskov [3], the data abstractions offer a mechanism to structure the programs into modules in which each module is responsible for implementing an abstraction.

Hierarchy

Hierarchy is the organization of abstractions into ordered structures in which more general or abstract concepts are connected to more specific or concrete concepts through "is-a" and "part of" relationships. In object-oriented programming, hierarchy is implemented using two main mechanisms: inheritance hierarchies and composition hierarchies.

In inheritance hierarchies, classes are organized into tree-like or lattice struc-

tures in which subclasses inherit attributes and behavior from their superclasses. The principle underlying such structures is Liskov's substitution principle, formulated by Liskov [3] and formalized in subsequent works [5]. This principle states that objects of a base type must be replaceable with objects of derived types without changing the correctness of the program.

In composition hierarchies, objects combine simpler objects through a "part-of" relationship, creating structures where complex objects are aggregates from simpler components. This organizational principle allows systems of any complexity to be built from well-understood, reusable building blocks. As noted in the seminal work on design patterns [6], composition provides a more flexible alternative to inheritance, allowing changes to be made to the relationships between objects at runtime that would otherwise be fixed at compile time.

Hierarchy provides several critical benefits: it enables the management of complexity through layered abstraction; it promotes code reuse through inheritance of common behavior; it establishes natural relationships between concepts that mirror real-world domain structures; and it facilitates polymorphism and dynamic dispatch, allowing for flexible, extensible program architectures.

2.1.2 Minor Components of the Object Model

The minor components are features that improve the object model's expressiveness and usability, but aren't strictly needed for a language to be considered as object-oriented. Presence of this components in modern programming languages shows the evolution of object-oriented principles for solving practical problems in modern software development.

Typing

Typing is defined a system of rules and mechanisms that determine how data types are associated with variables, expressions, and values, as well as how operations on values of different types are permitted. The type system assigns specific types to each element in the program, defining both the kind of data that can be stored and the operations that are allowed on that data.

Concurrency

Concurrency is defined as the ability of a system to manipulate several computations that are logically parallel, either through true simultaneous execution on multiprocessor systems or through alternating execution on single-processor systems. In the context of the object model, concurrency resolves the issue on how objects maintain consistency and correctness when multiple threads or processes simultaneously access and modify them.

Different object-oriented languages solve the problem of synchronizing values during parallel execution using different mechanisms. Some use mutual exclusion locks to ensure that only one thread can execute a particular method on an object at a time. Others support active objects that manage their own internal threads; still others provide transactional semantics, in which parallel operations are executed sequentially, ensuring consistency.

Persistence

Persistence refers to the ability to store and retrieve objects or their state between program runs. Without persistence, all objects created during program execution are lost when the program terminates, which limits the applicability of

object-oriented programming to stateless or short-lived computations.

The object model includes persistence through various approaches. The orthogonal persistence approach automatically saves and restores objects using special mechanisms without the need for explicit serialization code. With explicit programming, objects must implement serialization interfaces [7], or developers must write code to convert objects to storage representations and back.

2.1.3 The Selected Definition of the Object Model

Based on the conducted literature review, the following definition of the object model is adopted for the purposes of this work:

The object model is a conceptual framework that integrates a set of interrelated components — four major ones (abstraction, encapsulation, modularity, and hierarchy) and three minor ones (typing, concurrency, and persistence). It defines the means by which a programming language represents, organizes, and supports objects, classes, their interactions, inheritance relationships, mechanisms of polymorphism, and other related abstractions for the effective management of software system complexity.

2.2 Critical Analysis of Object-Oriented Programming Approach

The object-oriented programming paradigm, despite its significant influence on software development practices, demonstrates fundamental limitations and unresolved problems that hinder its applicability and effectiveness in various problem-solving areas. This section presents a systematic and critical examination of the main shortcomings of OOP. The critical perspective developed here provides a theoretical basis for evaluating the actual implementation of object models in modern programming languages.

2.2.1 Theoretical Inconsistencies in Inheritance and Subtyping

A fundamental issue in typed object-oriented languages concerns the conflation of inheritance with subtyping relations. Cook, Hill, and Canning demonstrate that inheritance, when used as a mechanism for incremental modification of recursive object definitions, does not necessarily produce subtypes [8]. The traditional assumption that inheritance hierarchies determine conformance relations proves inadequate when dealing with recursive types and contravariant method types.

The distinction between inheritance and subtyping stems from type-theoretic considerations. Cardelli and Wegner in their analysis of type systems establishes that inheritance as an implementation mechanism and subtyping represent orthogonal concerns [1]. Their concepts demonstrates that subtyping relations require adherence to the Liskov substitution principle, whereas inheritance mechanisms impose additional constraints that compromise expressiveness [9]. The consequence of this theoretical inconsistency is that inheritance-based type hierarchies

cannot enforce type safety without either compromising expressiveness or introducing type insecurities. Eiffel exemplifies this problem: by identifying classes with types and inheritance with subtyping, the language exhibits type insecurities as documented by Cardelli and Wegner [1].

The following C# example 2.1 demonstrates how inheritance does not automatically guarantee proper subtyping behavior 2.2, particularly with covariance and contravariance in method signatures:

2.2.2 The Fragile Base Class Problem

One of the most significant architectural deficiencies in inheritance-based OOP systems is the fragile base class problem, formally studied by Mikhajlov and Sekerinski [10]. This problem occurs in open object-oriented systems employing code inheritance as an implementation reuse mechanism: seemingly safe modifications to a base class can cause derived classes to malfunction, despite no explicit violation of method contracts or public interfaces. Mikhajlov and Sekerinski distinguish between two manifestations: the syntactic fragile base class problem, which necessitates recompilation of extension and client classes when base classes change, and the semantic fragile base class problem, wherein self-recursion vulnerabilities create situations where base class revisions break extension class functionality without changing the external interface [10].

The fundamental cause lies in the interaction between dynamic dispatch, self-reference (encoded as self or this), and encapsulation boundaries. When a subclass overrides methods that the base class uses internally through dynamic dispatch, changes to the base class implementation can alter method invocation sequences, causing subclass assumptions about execution order to be violated.

```
public abstract class Animal
{
    public virtual void Reproduce(Animal mate)
    {
        Console.WriteLine("Animals reproducing");
    }

    public abstract void Feed(Food food);
}

public class Dog : Animal
{
    // VIOLATION: Contravariant parameter - accepts broader type
    // This breaks Liskov Substitution Principle when used polymorphically
    public override void Reproduce(Animal mate)
    {
        if (!(mate is Dog))
            throw new ArgumentException("Dogs can only reproduce with dogs");
        Console.WriteLine("Dogs reproducing");
    }

    // VIOLATION: Covariant return would be OK, but Food parameter is problematic
    public override void Feed(Food food)
    {
        Console.WriteLine($"Dog eating: {food.Name}");
    }
}

public class Cat : Animal
{
    public override void Reproduce(Animal mate)
    {
        if (!(mate is Cat))
            throw new ArgumentException("Cats can only reproduce with cats");
        Console.WriteLine("Cats reproducing");
    }

    public override void Feed(Food food)
    {
        Console.WriteLine($"Cat eating: {food.Name}");
    }
}

public class Food { public string Name { get; set; } }
```

Fig. 2.1. Inheritance without proper subtyping

```
Animal dog = new Dog();
Animal cat = new Cat();

// This code violates the contract established by the base class
// The compiler allows it, but runtime fails - TYPE INSECURITY
try
{
    dog.Reproduce(cat); // Runtime exception: "Dogs can only reproduce with dogs"
}
catch (ArgumentException ex)
{
    Console.WriteLine($"Type safety violation: {ex.Message}");
}
```

Fig. 2.2. Inheritance without proper subtyping: manifestation

This coupling violation fundamentally undermines the promise of modular reuse through inheritance. In open systems where subclasses are developed independently of base class implementations, predicting the consequences of base class modifications becomes impractical [10].

2.2.3 Excessive Coupling and Tight Interdependencies

The inheritance mechanism introduces structural coupling between base and derived classes that compromises modularity and reusability. Booch's analysis of class quality metrics identifies coupling as a critical measure of design quality, noting that inheritance creates strong coupling between superclasses and subclasses [2]. This coupling creates dependencies where modifications in parent classes propagate through inheritance chains, a phenomenon that Hitz and Montazeri characterize as particularly problematic: excessive coupling between object classes is detrimental to modular design and prevents reuse of individual components in alternative applications [11].

Beyond inheritance-induced coupling, OOP systems exhibit architectural issues with circular dependencies among classes. Circular dependencies create

```
public class BankAccount
{
    protected decimal _balance;

    public virtual void Deposit(decimal amount)
    {
        _balance += amount;
        Console.WriteLine($"Deposited {amount}. Balance: {_balance}");
    }

    public virtual void Withdraw(decimal amount)
    {
        if (amount <= _balance)
        {
            _balance -= amount;
            Console.WriteLine($"Withdrawn {amount}. Balance: {_balance}");
        }
    }

    public virtual decimal GetBalance()
    {
        return _balance;
    }
}
```

Fig. 2.3. Fragile Base Class: first implementation base class

tight mutual coupling that reduces the possibility of separate reuse and creates domino effects where changes in one module spread unwanted side effects to dependent modules. This architectural pattern violates the foundational principle of modular decomposition, particularly problematic in large-scale systems where dependency management becomes increasingly complex.

The Gang of Four Design Patterns documentation explicitly acknowledges this limitation, stating that inheritance exposes a subclass to details of its parent's implementation, and therefore "inheritance breaks encapsulation" [6]. The authors warn that implementation of a subclass can become so bound to the implementation of its parent that any change in the parent's implementation forces the subclass to change. They further note that polymorphism combined with implementation inheritance can create systems where it is difficult to predict which

```
public class PremiumAccount : BankAccount
{
    private decimal _monthlyFee = 10m;
    private int _freeTransactions = 10;
    private int _transactionCount = 0;

    public override void Deposit(decimal amount)
    {
        _transactionCount++;
        if (_transactionCount > _freeTransactions)
        {
            base.Deposit(amount - _monthlyFee); // Account for fee
        }
        else
        {
            base.Deposit(amount);
        }
    }

    public override void Withdraw(decimal amount)
    {
        _transactionCount++;
        base.Withdraw(amount); // Assumes Withdraw will update _balance
    }

    public void ApplyMonthlyFee()
    {
        _balance -= _monthlyFee;
    }
}
```

Fig. 2.4. Fragile Base Class: first implementation deriving class

```
public class BankAccount_V2
{
    protected decimal _balance;
    protected decimal _totalFees = 0;

    // CHANGE: Added automatic fee application
    public virtual void Deposit(decimal amount)
    {
        _balance += amount;
        ApplyTransactionFee(); // NEW: automatic fee
        Console.WriteLine($"Deposited {amount}. Balance: {_balance}");
    }

    public virtual void Withdraw(decimal amount)
    {
        if (amount <= _balance)
        {
            _balance -= amount;
            ApplyTransactionFee(); // NEW: automatic fee
            Console.WriteLine($"Withdrawn {amount}. Balance: {_balance}");
        }
    }

    protected virtual void ApplyTransactionFee()
    {
        _balance -= 1m; // Transaction fee
        _totalFees += 1m;
    }

    public virtual decimal GetBalance()
    {
        return _balance;
    }
}
```

Fig. 2.5. Fragile Base Class: misspelled implementation base class

```
public class PremiumAccount_V2 : BankAccount_V2
{
    private decimal _monthlyFee = 10m;
    private int _freeTransactions = 10;
    private int _transactionCount = 0;

    public override void Deposit(decimal amount)
    {
        _transactionCount++;
        if (_transactionCount > _freeTransactions)
        {
            // PROBLEM: Base.Deposit now also applies transaction fee!
            // Double deduction: _monthlyFee + automatic ApplyTransactionFee()
            base.Deposit(amount - _monthlyFee);
        }
        else
        {
            base.Deposit(amount); // Unexpected fee added by base class
        }
    }

    public override void Withdraw(decimal amount)
    {
        _transactionCount++;
        base.Withdraw(amount); // Unexpected fee added by base class
    }
}
```

Fig. 2.6. Fragile Base Class: misspelled implementation deriving class

```
var account = new PremiumAccount_V2();
account.Deposit(100); // Expected: balance = 100; Actual: balance = 89 (100 - 10 - 1)
Console.WriteLine($"Balance: {account.GetBalance()}"); // Shows unexpected deduction

// Subclass assumptions about execution order are violated
// The fragile base class problem manifests as silent data corruption
```

Fig. 2.7. Fragile Base Class: demonstration

method will be invoked in response to a message, complicating both debugging and maintenance. architectural coupling

```
public class Account {
    protected double balance;  // Exposed implementation detail

    public Account(double initialBalance) {
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }
        balance += amount;
        logTransaction("DEPOSIT", amount);
    }

    protected void logTransaction(String type, double amount) {
        System.out.printf("[%s] $%.2f | Balance: $%.2f%n",
                          type, amount, balance);
    }
}
```

Fig. 2.8. Base class Account exposing internal state and workflow through protected members

The examples demonstrate two critical manifestations of excessive coupling in object-oriented systems. Figures 2.8 and 2.9 illustrate inheritance-induced coupling where SavingsAccount becomes dependent on implementation details of its parent Account: the protected field balance, the internal workflow sequence (deposit → log), and the assumption that logTransaction() executes after state modification. This breaks encapsulation because the subclass gains access to superclass internals that should remain hidden. Critically, any modification to Account's implementation—renaming balance, reordering workflow steps, or changing logTransaction()'s signature—necessitates corresponding changes in SavingsAccount, creating ripple effects through the inheritance hierarchy. Figures 2.10 2.11 2.12 exhibit architectural coupling through circular dependencies

```
public class SavingsAccount extends Account {
    private static final double INTEREST_RATE = 0.05;

    public SavingsAccount(double initialBalance) {
        super(initialBalance);
    }

    @Override
    public void deposit(double amount) {
        super.deposit(amount);
        applyInterest();  // Breaks if parent changes workflow order
    }

    private void applyInterest() {
        // Direct manipulation of protected state violates encapsulation
        double interest = balance * INTEREST_RATE;
        balance += interest;
        logTransaction("INTEREST", interest);
    }
}
```

Fig. 2.9. Derived class SavingsAccount tightly coupled to parent's implementation details

```
public class Order {
    private Customer customer;
    private List<OrderItem> items;

    public Order(Customer customer) {
        this.customer = Objects.requireNonNull(customer);
        this.items = new ArrayList<>();
        // Creates circular dependency: Order $\rightarrow$ Customer
        customer.registerOrder(this);  // Customer $\rightarrow$ Order
    }

    public void addItem(OrderItem item) {
        items.add(item);
    }

    public double calculateTotal() {
        return items.stream()
                    .mapToDouble(OrderItem::getPrice)
                    .sum();
    }

    public Customer getCustomer() {
        return customer;
    }
}
```

Fig. 2.10. Class Order establishing bidirectional dependency with Customer

```
public class Customer {
    private String id;
    private List<Order> orders;

    public Customer(String id) {
        this.id = id;
        this.orders = new ArrayList<>();
    }

    public void registerOrder(Order order) {
        orders.add(order);
    }

    public double getLifetimeValue() {
        return orders.stream()
                     .mapToDouble(Order::calculateTotal)
                     .sum();
    }
}
```

Fig. 2.11. Class Customer completing circular dependency cycle and depending on Order internals

```
public class OrderItem {
    private String productId;
    private double price;
    private int quantity;

    public OrderItem(String productId, double price, int quantity) {
        this.productId = productId;
        this.price = price;
        this.quantity = quantity;
    }

    public double getPrice() {
        return price * quantity;  // Implementation detail exposed to callers
    }
}
```

Fig. 2.12. Supporting class OrderItem whose internal calculation logic is exposed to dependent classes

between Order and Customer, where each class maintains a direct reference to the other. This mutual dependency prevents independent reuse of either component, violates modular decomposition principles, and creates domino effects where changes to Order's calculation logic (calculateTotal()) propagate unpredictably to Customer.getLifetimeValue().

2.2.4 Multiple Inheritance and Ambiguity Resolution

Languages supporting multiple inheritance encounter the well-documented diamond problem, wherein a derived class inheriting from multiple parents creates ambiguity about which implementation of inherited methods should be invoked when those parents themselves share a common ancestor. This problem reflects fundamental challenges in the inheritance model: the attempt to linearize and resolve multiple method resolution paths introduces complexity that defeats the transparency and simplicity promised by object-oriented design.

Languages addressing this through single inheritance restrictions or interface-based approaches acknowledge that inheritance as a reuse mechanism creates structural problems that require linguistic constraints. The very fact that languages must prohibit or heavily restrict inheritance patterns suggests limitations in the fundamental mechanism itself.

In the C++ implementation (Figs. 2.13, 2.14, 2.15, 2.16), class Bat contains two distinct subobjects of Animal—one through the Mammal path and another through the Bird path—each maintaining independent state (health). This duplication violates the object identity principle: a bat logically possesses a single health value, yet the inheritance model creates two physically separate instances. Access to inherited members becomes ambiguous at compile time, requiring ex-

```
class Animal {
protected:
    int health;  // Shared state from common ancestor
public:
    Animal() : health(100) {}
    void breathe() {
        std::cout << "Breathing with health=" << health << std::endl;
    }
};
```

Fig. 2.13. Base class Animal defining shared state and behavior in diamond inheritance hierarchy

```
class Mammal : public Animal {
    // NO override of breathe(); inherits Animal::breathe() directly
    // NO modification of health initialization
};
```

Fig. 2.14. Intermediate class Mammal inheriting from Animal without overriding base methods

```
class Bird : public Animal {
    // NO override of breathe(); inherits Animal::breathe() directly
    // NO modification of health initialization
};
```

Fig. 2.15. Intermediate class Bird inheriting from Animal without overriding base methods

```
class Bat : public Mammal, public Bird {
};

int main() {
    Bat bat;
    bat.breathe();        // COMPILATION ERROR: ambiguous call
    bat.health = 50;      // COMPILATION ERROR: ambiguous member access
}
```

Fig. 2.16. Derived class Bat exhibiting diamond inheritance ambiguity without virtual inheritance

```
class Animal:
    def __init__(self):
        self.health = 100  # State defined in common ancestor

    def breathe(self):
        return f"Breathing (health={self.health})"
```

Fig. 2.17. Base class Animal defining shared state in Python diamond hierarchy

```
class Mammal(Animal):
    def __init__(self):
        super().__init__()
        # NO override of breathe(); delegates to Animal.breathe()
```

Fig. 2.18. Intermediate class Mammal inheriting from Animal without method override

```
class Bird(Animal):
    def __init__(self):
        super().__init__()
        # NO override of breathe(); delegates to Animal.breathe()
```

Fig. 2.19. Intermediate class Bird inheriting from Animal without method override

```
class Bat(Mammal, Bird):
    def __init__(self):
        super().__init__()

bat = Bat()
print(bat.breathe())
print(Bat.__mro__)          # Shows linearization path
# (<class '__main__.Bat'>, <class '__main__.Mammal'>,
#  <class '__main__.Bird'>, <class '__main__.Animal'>, <class 'object'>)
```

Fig. 2.20. Derived class Bat demonstrating method resolution order limitations in Python

plicit path qualification ($\mathrm{bat.Mammal::breathe()}$) that exposes implementation details and defeats encapsulation. The Python implementation (Figs. 2.17 , 2.18, 2.19, 2.20) mitigates lookup ambiguity through C3 linearization, but the fundamental conceptual problem persists: multiple inheritance paths to a single logical ancestor create tension between the inheritance graph's topology and the linear method resolution order.

2.2.5 The Reusability Paradox

Despite promises of enhanced reusability through inheritance and polymorphism, empirical software development experience reveals significant obstacles to achieving code reuse in OOP systems. Graham observes that object-oriented programming offers "a sustainable way to write spaghetti code," suggesting that the anticipated reusability gains often fail to materialize [12]. The problem lies partly in the tightly coupled nature of inheritance-based designs: inheriting from an existing class binds the derived class not just to the inherited interface but to the entire implementation dependency graph of the parent class.

Effective reuse typically requires either shallow inheritance hierarchies or careful extraction of orthogonal concerns into separately reusable components. This requirement contradicts the hierarchical organization that inheritance encourages, creating a tension between the inheritance mechanism and practical reusability goals.

The inheritance-based approach in Fig. 2.21 exemplifies the reusability paradox: despite leveraging inheritance to extend $\mathrm{HashSet}$'s functionality, the derived class becomes tightly coupled to implementation details invisible in the public interface. Specifically, $\mathrm{HashSet.addAll()}$ internally invokes $\mathrm{add()}$ for each

```java
// Problematic reuse through inheritance
public class InstrumentedHashSet<E> extends HashSet<E> {
    private int addCount = 0;

    @Override
    public boolean add(E e) {
        addCount++;
        return super.add(e);
    }

    @Override
    public boolean addAll(Collection<? extends E> c) {
        addCount += c.size();
        return super.addAll(c);
    }

    public int getAddCount() {
        return addCount;
    }
}
```

Fig. 2.21. Instrumented set implementation using inheritance

element—a behavioral detail not specified in the Set interface contract. Consequently, when clients invoke addAll({'apple', 'banana', 'cherry'}), the counter increments six times (three via the overridden addAll() and three via subsequent add() calls), violating expectations. The composition-based solution in Fig. 2.22 achieves reliable reuse by decoupling instrumentation logic from the underlying set implementation through explicit delegation.

2.2.6 State Management Complexity and Object Identity

Object-oriented systems organize code around mutable state encapsulated within objects. This approach fundamentally differs from functional programming's emphasis on immutability and pure functions. The combination of mutable state, object identity, and encapsulation creates complexity in reasoning about program behavior. When objects maintain mutable internal state, analyzing program correctness requires understanding not only the method calls but

```java
// Robust reuse through composition
public class InstrumentedSet<E> implements Set<E> {
    private final Set<E> delegate;
    private int addCount = 0;

    public InstrumentedSet(Set<E> delegate) {
        this.delegate = Objects.requireNonNull(delegate);
    }

    @Override
    public boolean add(E e) {
        addCount++;
        return delegate.add(e);
    }

    @Override
    public boolean addAll(Collection<? extends E> c) {
        addCount += c.size();
        return delegate.addAll(c);
    }

    public int getAddCount() {
        return addCount;
    }

    // Delegation to all other Set methods omitted for brevity
    @Override public int size() { return delegate.size(); }
    @Override public boolean isEmpty() { return delegate.isEmpty(); }
    // ... remaining Set interface methods delegated to 'delegate'
}
```

Fig. 2.22. Instrumented set implementation using composition

also the complete history of state modifications, increasing the difficulty of formal verification and testing.

Reynolds' distinction between values (immutable, state-based equality) and entities (distinct identity, mutable state) illustrates the conceptual tension in OOP [13]. In value-oriented systems, equality testing is unambiguous and immutability simplifies reasoning. In object-oriented systems, the notion of object identity introduces questions about whether two references point to the same object or equivalent objects, complications that functional approaches avoid entirely through immutable values.

Furthermore, the emphasis on object identity creates problems in concurrent systems where mutable shared state becomes a liability. Multiple threads attempting to access and modify object state require extensive synchronization, complicating concurrent programming compared to functional approaches based on immutable data and pure transformations [13].

Figure 2.23 defines $\mathrm{BankAccount}$ as an entity with mutable state ($\mathrm{balance}$) and dual equality semantics: reference identity ($==$) and state-based equivalence ($.\mathrm{equals}()$). Figure 2.24 demonstrates the resulting reasoning complexity. Lines 5–14 show how shared identity causes non-local side effects: mutation through reference a implicitly alters the state observed through b, requiring the programmer to track aliasing relationships. Lines 16–27 reveal following problem: initially equivalent entities c and d diverge after mutation, that is violating referential transparency.

```
public class BankAccount {
    private final String accountId;
    private int balance;

    public BankAccount(String accountId, int initialBalance) {
        this.accountId = accountId;
        this.balance = initialBalance;
    }

    public void deposit(int amount) {
        this.balance += amount;
    }

    public void withdraw(int amount) {
        this.balance -= amount;
    }

    public int getBalance() {
        return this.balance;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof BankAccount)) return false;
        BankAccount other = (BankAccount) obj;
        return this.accountId.equals(other.accountId) &&
               this.balance == other.balance;
    }
}
```

Fig. 2.23. Mutable entity class with identity and state-based equality semantics

```
public class AccountManager {
    public static void demonstrateIdentityProblem() {
        // Two references to the SAME entity (identity equivalence)
        BankAccount a = new BankAccount("A123", 100);
        BankAccount b = a;

        assert (a == b);                // true: same identity
        assert a.equals(b);             // true: same state

        a.deposit(50);

        // Mutation affects BOTH references due to shared identity
        assert a.getBalance() == 150;
        assert b.getBalance() == 150;   // Unexpected side effect!

        // Two DISTINCT entities with identical initial state (value equivalence)
        BankAccount c = new BankAccount("B456", 100);
        BankAccount d = new BankAccount("B456", 100);

        assert (c != d);                // false: different identities
        assert c.equals(d);             // true: equivalent state

        c.deposit(50);

        // Mutation breaks state equivalence despite identical initial state
        assert !c.equals(d);            // State divergence after mutation
        assert c.getBalance() == 150;
        assert d.getBalance() == 100;   // Independent state evolution
    }
}
```

Fig. 2.24. Demonstration of identity-value tension and state mutation effects

2.2.7 Absence of Clear Separation Between Specification and Implementation

OOP languages typically conflate class specifications with implementation details through a single construct. Unlike languages supporting explicit separation between interfaces and implementations, OOP classes bundle these concerns, making it difficult to understand what behavior clients should depend upon versus what represents implementation-specific detail. This conflation particularly affects evolution and maintenance: modifications to implementation details that preserve external contracts can nonetheless break derived classes through the fragile base class problem.

2.2.8 Empirical Evidence of Design Difficulties

The widespread adoption of design patterns and anti-pattern literature documents systematic difficulties with OOP system design. The Gang of Four patterns book identifies recurring design problems that OOP practitioners encounter; the very necessity of these patterns suggests that raw OOP mechanisms inadequately support common design requirements [14]. Anti-patterns such as the God Object problem reflect systematic tendency toward violation of single responsibility principles.

Chidamber and Kemerer's metrics suite for measuring OOP design quality reveals correlations between high metric values and undesirable properties: high coupling between object classes indicates fault-proneness and maintenance difficulty; excessive inheritance tree depth indicates reduced comprehensibility; high weighted methods per class indicates complexity [15]. These metrics-based findings empirically validate concerns about OOP's inherent tendency toward

```
abstract class AbstractEnterpriseProcessor {

    protected AuditService audit;
    protected PricingService pricing;
    protected ValidationService validation;

    public final Order process(OrderRequest request) {

        validate(request);
        preProcess(request);

        double price = pricing.calculate(request);
        Order order = createOrder(request, price);

        postProcess(order);
        audit.log("order_created");

        return order;
    }

    protected void validate(OrderRequest request) {
        validation.validateUser(request.userId());
        validation.validateItems(request.items());
    }

    protected abstract void preProcess(OrderRequest request);
    protected abstract Order createOrder(OrderRequest request, double price);
    protected abstract void postProcess(Order order);
}
```

Fig. 2.25. Base class implementing the Template Method pattern for order processing.

coupled, complex designs. Their research demonstrates that systems exhibiting high CBO values require more rigorous testing and experience higher defect rates, a pattern consistent across multiple industrial software projects [15].

The example presented in Figures 2.25,2.26,2.27,2.28,2.29 illustrates structural characteristics empirically associated with reduced maintainability and increased defect proneness in object-oriented systems.

First, the hierarchy exhibits a Depth of Inheritance Tree (DIT) equal to five. Understanding the runtime behavior of the process operation requires analysis across multiple abstraction layers. It has been empirically correlated with de-

```
abstract class PaymentProcessor extends AbstractEnterpriseProcessor {

    protected PaymentGateway payment;
    protected TransactionRepository transactions;

    @Override
    protected Order createOrder(OrderRequest request, double price) {

        payment.charge(request.userId(), price);
        transactions.save(request.userId(), price);

        return new Order(request.userId(), price);
    }
}
```

Fig. 2.26. Inheritance layer introducing payment and transaction persistence behavior.

creased comprehensibility and increased maintenance effort.

Second, the final concrete class demonstrates high Coupling Between Object classes (CBO). The EnterpriseOrderProcessor directly depends on numerous infrastructural and domain services. When combined with inherited dependencies from ancestor classes, the total number of external collaborators exceeds fifteen distinct services. High CBO values have been consistently associated with increased fault proneness and testing complexity.

Third, the concrete processor exhibits characteristics of the God Object antipattern. It assumes responsibility for heterogeneous business concerns. This concentration of responsibilities violates the Single Responsibility Principle and increases the likelihood of ripple effects during modification.

2.2.9 Systematic Classification of OOP Limitations

The critical examination conducted in the preceding subsections reveals a coherent set of fundamental limitations inherent in the object-oriented programming paradigm. These limitations can be systematically classified into several

```
abstract class FraudAwareProcessor extends PaymentProcessor {

    protected FraudService fraud;
    protected BlacklistService blacklist;

    @Override
    protected void preProcess(OrderRequest request) {

        double score = fraud.score(request);

        if (score > 0.8) {
            blacklist.add(request.userId());
            throw new RuntimeException("fraud suspected");
        }
    }
}
```

Fig. 2.27. Inheritance layer adding fraud detection and blacklist management.

```
abstract class RetryableProcessor extends FraudAwareProcessor {

    protected RetryService retry;

    @Override
    public final Order process(OrderRequest request) {

        return retry.execute(() -> super.process(request));
    }
}
```

Fig. 2.28. Inheritance layer introducing retry semantics around the processing algorithm.

interconnected categories, which collectively explain the challenges encountered in the design, implementation, and maintenance of large-scale OOP systems.

1. **Theoretical and Type-System Deficiencies:** At the most foundational level, OOP suffers from a conflation of distinct concepts, primarily inheritance (an implementation reuse mechanism) and subtyping (a type compatibility relation). This conflation, formalized by type theory, leads to inherent tensions between expressiveness and type safety, as exemplified by the need for complex workarounds.

2. **Architectural and Structural Deficiencies:** The inheritance mechanism itself introduces severe architectural flaws. It creates tight, often hidden, coupling between base and derived classes, fundamentally breaking encapsulation. This results in the well-documented *Fragile Base Class Problem*, where the modularity of a system is compromised because changes in one module (a base class) can unpredictably break functionality in dependent modules (derived classes), despite adherence to public interfaces.

3. **Compositional and Reusability Deficiencies:** Contrary to its core promises, OOP often hinders effective code reuse. The *Reusability Paradox* highlights that deep inheritance hierarchies bind classes to extensive implementation dependency graphs, making isolated reuse difficult. Furthermore, the challenges of *Multiple Inheritance*, such as the diamond problem, demonstrate the paradigm's struggle to manage complexity when composing behaviors from multiple sources.

4. **State Management and Concurrency Deficiencies:** The paradigm's emphasis on mutable state and object identity complicates reasoning about program behavior, especially in concurrent and distributed environments. The need to manage and synchronize shared mutable state stands in stark contrast to the simplicity offered by immutable data models, making OOP a suboptimal choice for highly concurrent systems.

5. **Design and Evolvability Deficiencies:** The absence of a clear separation between specification and implementation within the class construct impedes system evolution and maintenance. This, combined with the inherent coupling, leads to well-documented design anti-patterns. Empirical evi-

dence, including software metrics and the proliferation of design patterns, confirms a systematic tendency toward complex, tightly coupled designs that are fault-prone and difficult to comprehend.

```
class EnterpriseOrderProcessor extends RetryableProcessor {

    private InventoryService inventory;
    private RecommendationEngine recommendations;
    private ERPClient erp;
    private NotificationService notifications;
    private AnalyticsService analytics;
    private DiscountService discounts;
    private LoyaltyService loyalty;
    private ShippingService shipping;
    private InvoiceService invoice;
    private CRMClient crm;

    @Override
    protected void postProcess(Order order) {

        inventory.reserve(order);
        shipping.schedule(order);
        invoice.generate(order);
        erp.sync(order);
        crm.update(order);

        notifications.send(order);
        recommendations.update(order);
        analytics.track(order);
        loyalty.addPoints(order);

        applyDiscountPolicies(order);
        triggerCampaigns(order);
        reconcileAccounting(order);
    }

    private void applyDiscountPolicies(Order order) { }
    private void triggerCampaigns(Order order) { }
    private void reconcileAccounting(Order order) { }
}
```

Fig. 2.29. Concrete enterprise processor exhibiting extensive infrastructure coupling.

2.3 Review of Related Work

The problem of comparative analysis of programming languages is not new to computer science. Over recent decades, numerous works have been published dedicated to the classification and comparison of object-oriented languages. Through a systematic literature review, we have identified three main groups of studies: fundamental taxonomies, empirical performance comparisons, and narrowly focused comparative analyses of specific language pairs.

2.3.1 Fundamental taxonomies and theoretical foundations

The foundational works in the classification of object models were conducted in the late 1980s. The seminal work by P. Wegner, “Dimensions of Object-Oriented Language Design“ [16], laid the groundwork by introducing strict criteria for “object-based“ languages. However, contemporary research indicates that these hierarchical taxonomies are losing their relevance in the era of multi-paradigm languages.

In a recent work, M. Vandeloise [17] argues that traditional classifications fail to account for hybrid languages (e.g., Scala, Rust) and proposes a “compositional reconstruction“ of paradigms based on type theory, where the key factor becomes safety guarantees rather than the presence of classes. This is supported by research from Crichton et al. [18], who propose treating the ownership semantics in Rust as a new fundamental dimension of the object model, which ensures memory safety without a garbage collector—an aspect absent from earlier taxonomies like Armstrong’s [19].

2.3.2 Empirical Performance Comparisons

A second extensive group of works is dedicated to the quantitative comparison of languages. The well-known study by L. Prechelt [20] compares seven languages (C, C++, Java, Python, among others) based on criteria such as execution time and memory consumption. Similar contemporary studies, for instance, the work by Farooq and Khan [21], propose frameworks for assessing the popularity and technical efficiency of languages.

A key limitation of this group of works, in relation to the goals of our thesis, is their focus on *quantitative metrics* (speed, code volume) rather than on the *architectural distinctions* between object models. They answer the question “which language is faster,“ but do not explain how differences in the implementation of v-tables, prototypes (JavaScript), or structural typing (Go) influence the architecture of software systems.

2.3.3 Narrowly Focused and Engineering Comparisons

The third category encompasses works that compare specific language pairs or their individual mechanisms. Numerous publications exist that contrast Go’s interface model with the class-based model of Java/C++ [22], [23]. Hybrid functional-object-oriented models in Scala compared to Java have also been extensively documented [24].

Nevertheless, a significant gap remains in the academic literature regarding works that conduct a comprehensive, end-to-end analysis of the entire spectrum of modern models: ranging from “pure“ OOP (Smalltalk, Ruby) to prototype-based (JavaScript), hybrid (Scala, C++), and active object models (Zonnon). The Zonnon language and its unique model, based on active objects and dialogs as

described by Gutknecht [25], receives virtually no attention in broad comparative surveys, remaining confined to specialized niche literature.

2.3.4 Conclusion

The analysis of the literature demonstrates that, despite the abundance of comparative studies, a comprehensive investigation is lacking—one that would:

1. Unify both classical and modern languages within a single comparative framework.

2. Examine not only syntax or performance but also the internal implementation of object models (dispatch mechanisms, memory management, object layouts).

3. Identify the architectural consequences of choosing a particular model for designing complex systems.

This thesis aims to fill this gap by proposing a systematization that includes both industry standards and academic languages, with the goal of developing recommendations for an enhanced object model.

2.4 Methodology of Comparative Analysis

When developing object models for programming language, a systematic and reproducible approach to their analysis must be employed. This section provides a list of criteria for comparing the implementation of object models.

2.4.1 Rationale for the Chosen Methodology

The selection of comparison criteria is justified by a number of fundamental works on programming language design. Cardelli and Wegner [1] established three key requirements for defining an object-oriented language: support for objects as data abstractions, the association of objects with a specific type, and type inheritance. These requirements form a basic classification of languages and allow one to distinguish object-oriented languages from those that merely support some object-based features.

Booch [2] defined four core elements of the object model — abstraction, encapsulation, modularity, and hierarchy — as well as three secondary elements — as well as three secondary elements—typing, concurrency, and persistence. According to Booch, without the core elements, a model ceases to be object-oriented; however, the secondary elements significantly impact the model's practical applicability. This hierarchical approach to identifying elements helps determine which language characteristics are critical.

The methodology for evaluating programming languages is based on a comprehensive approach discussed in works on language assessment criteria. Sebesta [26] proposed four primary criteria for evaluating languages: readability, writability, reliability, and cost. Furthermore, readability and writability depend on a wide range of language characteristics, including syntactic simplicity, support for

abstraction, and orthogonality. Modern works on code quality and software maintainability emphasize the importance of metrics such as modularity, reusability, analyzability, and modifiability [27], which are directly influenced by the characteristics of a specific language's object model.

2.4.2 Object Model Comparison Criteria

Based on the fundamental works mentioned above and an analysis of programming language standards, the following criteria have been identified for the systematic comparison of object models:

1. **Type System:** allows to evaluate how the language ensures code reliability and builds its infrastructure through data validation rules.

2. **Inheritance Mechanism:** is necessary to compare how languages support code reuse and hierarchy organization, which directly affects maintainability and extensibility.

3. **Abstraction and Encapsulation:** helps analyze how language manages complexity by hiding implementation details and protecting the internal state of objects.

4. **Polymorphism:** comparing the trade-off between performance (static dispatching) and code flexibility (dynamic dispatching) in different languages.

5. **Object Creation Mechanism:** Compare fundamental approaches to defining objects: based on classes (instances) or prototypes (delegation).

6. **Metaclasses and Metaprogramming:** assessments of the language's ability to introspect and modify its own structure during execution.

7. **Multiple Inheritance and Mixins:** Compare alternative mechanisms for composition and extending functionality beyond a strict class hierarchy.

8. **Exception Handling:** Compare the philosophies of languages regarding error handling guarantees: strict control at the compilation stage (checked exceptions) or a more flexible runtime model.

9. **Modularity and Packages:** assess how the language supports the organization and scaling of large projects through code structuring mechanisms.

10. **Subtyping and Contract model:** Comparison of how languages relate inheritance and subtyping, and what means of contract formalization they provide (invariants, preconditions, postconditions).

11. **Composition mechanisms and alternatives to inheritance:** Comparison of the degree of composition support (delegation, decorators, first-class interfaces) as an alternative to inheritance, it shows how much the model really supports reuse without hierarchies.

12. **Managing the mutability and evolution of hierarchies:** Languages protect clients from base class changes in different ways: final/sealed constructs, closed hierarchies, explicit extension points, and redefinition rules.

13. **Multi-paradigm integration:** Comparison of how the object model is consistent with other paradigms (immutability-first, ownership, actors, active objects).

14. **Support for formal verifiability and specification:** Compare the extent to which the object model is suitable for formal methods: the presence of strict type semantics, restrictions on side effects, explicit specification interfaces.

2.4.3 Methodology of Comparative Analysis

Based on the reviewed literature and the theoretical foundations presented above, the comparative methodology in this thesis is reduced to a set of eight core criteria. Each criterion reflects a structural dimension of the object model rather than superficial syntactic or performance characteristics.

1. Type System and Subtyping Model

The type system constitutes the primary semantic framework of an object model. Cardelli and Wegner emphasize that understanding object-oriented languages requires distinguishing between data abstraction, polymorphism, and subtyping [1]. They explicitly separate inheritance from subtyping, demonstrating that "inheritance is not subtyping" and that type safety depends on formal subtyping relations rather than reuse mechanisms.

Therefore, the comparison must evaluate: (i) nominal vs. structural typing, (ii) variance rules, and (iii) the formal relation between inheritance and subtyping.

Relevance: The type system defines semantic correctness boundaries and impacts modular reasoning.

2. Abstraction and Encapsulation Mechanisms

Abstraction and encapsulation are identified by Booch as core elements of the object model [2]. Parnas' principle of information hiding states that modules should conceal design decisions likely to change [4].

Encapsulation mechanisms (visibility modifiers, module boundaries, interface constructs) determine whether internal representation is truly protected or merely conventionally hidden. Weak encapsulation leads to fragile hierarchies

and tight coupling, as documented in design pattern literature.

Relevance: Encapsulation quality determines modular stability and evolvability.

3. Inheritance Semantics and Hierarchy Management

Hierarchy is one of Booch's four major components, yet inheritance introduces well-documented structural risks. Mikhajlov and Sekerinski formally analyze the fragile base class problem, demonstrating that "apparently safe modifications to a base class can cause derived classes to malfunction" [10].

Consequently, the methodology evaluates: (i) single vs. multiple inheritance, (ii) linearization rules, (iii) mechanisms such as sealed/final classes, and (iv) protection against fragile base class effects.

Relevance: Inheritance directly affects modular soundness and long-term maintainability.

4. Composition and Alternatives to Inheritance

Because class inheritance exposes the implementation details of superclasses to subclasses, many programmers prefer composition. Therefore, object models must be compared with respect to delegation mechanisms, interface-based composition, mixins, and traits. This criterion evaluates whether the language provides first-class constructs enabling reuse without hierarchical fragility.

Relevance: Composition mechanisms mitigate structural deficiencies of inheritance-based reuse.

5. Object Representation and Creation Model

The languages diverge fundamentally in how objects are constructed and represented: class-based instantiation, prototype-based delegation, or hybrid models.

This dimension determines object identity semantics, dispatch strategy, and layout constraints. It also affects meta-level extensibility and runtime adaptability.

Relevance: Object construction semantics shape the entire runtime model.

6. Polymorphism and Dispatch Mechanisms

Polymorphism is central to OOP theory. The comparison must distinguish between static (compile-time) and dynamic (runtime) dispatch and evaluate trade-offs between flexibility and predictability.

Relevance: Dispatch rules determine behavioral substitutability and performance characteristics.

7. State Model, Mutability, and Concurrency Guarantees

Reynolds distinguishes between values and entities, highlighting the conceptual implications of mutable object identity [13]. Mutable shared state complicates reasoning, particularly under concurrency.

Languages differ in whether they enforce immutability by default, provide ownership models, or rely on synchronization primitives. This dimension is essential for evaluating architectural scalability and suitability for concurrent systems.

Relevance: Increasingly critical in modern multi-core environments.

8. Support for Formal Specification and Verifiability

The theoretical separation between behavioral subtyping and implementation inheritance [8] implies the need for explicit contracts. Languages that support invariants, preconditions, and postconditions allow formal reasoning about substitutability.

The absence of clear separation between specification and implementation has been identified as a structural weakness of mainstream OOP systems [10]. Thus, the methodology assesses the availability of contract models and formal type semantics.

Relevance: Essential for long-term correctness, static verification, and architectural reliability.

Excluded Criteria

Certain criteria from the initial list were excluded:

- *Exception handling* — although relevant to language philosophy, it does not directly define the object model architecture.
- *Packages and modular organization* — considered secondary to encapsulation and hierarchy, already captured under modular abstraction.
- *Persistence* — treated as an orthogonal runtime concern rather than a defining structural property.
- *Metaprogramming* — important for language flexibility but not foundational for object model semantics.

Chapter 3

Methodology

This chapter provides a comparative analysis of object models in modern programming languages. The study covers a representative group of languages — C#, C++, C, Golang, Java, Python, Ruby, JavaScript, Scala, Smalltalk, and Zonnon— to encompass a wide range of object-oriented paradigms, from classical, class-based, to prototyping and structural typing systems. The analytical method includes a systematic comparison of the object model of each language in accordance with an established set of criteria based on the theoretical foundation presented in the previous chapter. In addition, to substantiate the practical consequences of theoretical differences and identified potential shortcomings, the methodology includes qualitative verification of documented cases of errors. This verification includes the analysis of real software defects and anti-patterns that can be directly related to specific characteristics or limitations within the frame of the implementation of the object model of the corresponding language.

3.1 Comparison Techniques

The criteria defined in the previous chapter establish the basis for the analysis of object models at the level of principles of language design. However, to ensure a strict and reproducible comparison between several programming languages, these qualitative parameters must be implemented in measurable constructs. Thus, the methodology moves from a theoretical classification to a formalized evaluation model in which each criterion is expressed as a quantitative metric. The following subsections provide a set of definitions of metrics corresponding to each previously identified criterion, providing both formal equations and decomposition at the component level.

3.1.1 Type System and Subtyping Model

The TSSI metric is based on research on language metrics and subtyping theory: the idea of representing language properties as normalized measurable components is taken from the work of [28], and the distinction between nominal and structural typing, as well as the difference between inheritance and subtyping, is based on the study of [1]. The component related to variance conceptually goes back to the formalization of parametric polymorphism and generic security generics in research on generic systems [29].

Metric Definition

Let the *Type System and Subtyping Soundness Index (TSSI)* be defined as:

$$TSSI(L) = \alpha \cdot S_{nom}(L) + \beta \cdot S_{var}(L) + \gamma \cdot S_{rel}(L)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Nominal–Structural Typing Score $S_{nom}(L)$

$$S_{nom}(L) = \frac{I_{struct}(L)+I_{nom}(L)}{2}$$

where:

- $I_{struct}(L) = 1$ if the language supports structural subtyping with formal typing rules; 0 otherwise.
- $I_{nom}(L) = 1$ if the language supports nominal typing with explicit subtype declarations; 0 otherwise.

Hybrid systems receive 1 for both components. Purely nominal or purely structural systems receive 0.5. The evaluation is performed at the language specification level by analyzing typing judgments and formal rules.

2) Variance Formalization Score $S_{var}(L)$

$$S_{var}(L) = \frac{V_{exp}(L)+V_{sound}(L)}{2}$$

where:

- $V_{exp}(L) \in [0, 1]$ measures the explicitness of variance annotations (0 = no generics; 0.5 = implicit or restricted variance; 1 = explicit declaration of co/contra/invariance).

- $V_{sound}(L) \in [0, 1]$ measures formal soundness guarantees (1 if variance rules are formally defined and proven type-safe in specification; 0.5 if partially restricted; 0 if unsound or unspecified).

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

$$S_{rel}(L) = \frac{I_{dec}(L)+I_{ind}(L)+I_{form}(L)}{3}$$

where:

$$I_{dec}(L) = \begin{cases} 1, & \text{if the language allows explicit subtyping without inheritance constr} \\ 0.5, & \text{if such separation exists but is restricted (e.g., limited to specific co} \\ 0, & \text{if subtyping is defined only through inheritance} \end{cases}$$

$$I_{ind}(L) = \begin{cases} 1, & \text{if subtyping rules are defined independently of inheritance semanti} \\ 0.5, & \text{if independence is partial or implicit} \\ 0, & \text{if subtyping is fully derived from inheritance} \end{cases}$$

$$I_{form}(L) = \begin{cases} 1, & \text{if inheritance and subtyping are formally specified as distinct relat} \\ 0.5, & \text{if separation is stated but не полностью формализована} \\ 0, & \text{if distinction отсутствует или не определена в спецификации} \end{cases}$$

This metric measures the proportion of subtype relationships that are not caused by implementation inheritance. If all subtypes exist in the language only

through inheritance, then the value of the metric approaches 0, which indicates the absence of conceptual separation. If subtypes are determined in most cases separately from inheritance (e.g., using interfaces or structural typing rules), the metric approaches 1, reflecting a strong semantic separation between reuse and interchangeability. Measurement is performed at the specification level by constructing abstract inheritance and subtype relation graphs from the formal language definition.

3.1.2 Abstraction and Encapsulation Mechanisms

The Abstraction and Encapsulation score is based on the approach to comparative analysis of programming languages proposed in [28], where individual language properties are translated into quantitative indicators. The idea of highlighting visibility levels, modular boundaries, and encapsulation mechanisms is based on work on object-oriented design and modularity [4], [30]. The component reflecting encapsulation protection at the specification and runtime levels is conceptually related to research on controlled access to the state of an object and restriction of unauthorized reflection [31].

Metric Definition

Let the *Abstraction and Encapsulation score (AE)* be defined as:

$$AE(L) = \alpha \cdot V(L) + \beta \cdot M(L) + \gamma \cdot I(L) + \delta \cdot E(L)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma + \delta = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Visibility Granularity Index $V(L)$

Normalized measure of the access control strictness modifiers (e.g., private, protected, file, module-level).

$$V(L) = \frac{n_{levels}(L)}{n_{max}}$$

where $n_{levels}(L)$ is the number of semantically distinct access scopes supported by the language.

2) Module Isolation Strength $M(L)$ Metric aimed for comparison of the number of module or namespace boundaaries that restrict access to a view:

$$M(L) = \frac{b_{strict}(L)}{b_{total}(L)}$$

where $b_{strict}(L)$ is the amount of boundaries with enforced export control (e.g., modules with explicit export lists), and $b_{total}(L)$ is the total number of modularization constructs.

3) Interface Abstraction Capacity $I(L)$ Measures the ability to separate specification from implementation (e.g., interfaces, abstract classes, protocols from classes).

$$I(L) = \frac{n_{abstraction_constructs}(L)}{n_{object}(L)}$$

where $n_{abstraction_constructs}(L)$ counts distinct language-level constructs dedicated to behavioral abstraction and $n_{object}(L)$ is the number of object-defining constructs.

4) Encapsulation Enforcement Coefficient $E(L)$ Binary measure reflecting

whether encapsulation violations are prevented by the type system and runtime (no unrestricted reflection, no unchecked field access, controlled unsafe mechanisms).

$E(L) = 1$ if the language does not supports mechanisms for uncontrolled access during runtime; 0 otherwise.

3.1.3 Inheritance Semantics and Hierarchy Management

The Inheritance Semantics Robustness Index metric develops the ideas about the difference between inheritance and subtyping outlined in [8] and complements them with formal aspects of standard systems from [1], [29]. The component related to deterministic parameterization and method resolution order is conceptually based on work on conflict resolution in inheritance hierarchies [32].

Metric Definition

Let the *Inheritance Semantics Robustness Index (ISRI)* be defined as:

$$ISRI(L) = \alpha \cdot H_s(L) + \beta \cdot L_d(L) + \gamma \cdot C_p(L) + \delta \cdot F_b(L),$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma + \delta = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Hierarchy Simplicity Factor $H_s(L)$

A normalized measure of inheritance model complexity:

$$H_s(L) = 1 - \frac{M(L)-1}{M_{max}-1},$$

where $M(L)$ is the maximum number of direct superclass declarations allowed (1 for single inheritance, > 1 for multiple inheritance), and M_{max} is a maximal value of M(L) for all comparing languages.

2) Linearization Determinism Score $L_d(L)$ A binary measure reflecting whether the language defines a formally specified, deterministic method resolution order (MRO):

$$L_d(L) = \begin{cases} 1, & \text{if linearization is formally defined and deterministic,} \\ 0.5, & \text{if partially specified,} \\ 0, & \text{if ambiguous or implementation-defined.} \end{cases}$$

3) Constraint Protection Coefficient $C_p(L)$ Measures availability of hierarchy restriction mechanisms:

$$C_p(L) = \frac{\sum_{i=1}^{k} p_i(L)}{k}$$

where $p_i(L)$ — binary indicator defined ШТ specification. Comparison includes:

- p_1: Finalization of classes/methods (e.g., final, non-inheritable declarations)
- p_2: Sealed or closed hierarchies (restricted subclassing sets)
- p_3: Explicit override enforcement (mandatory $\mathrm{override}$ annotation)
- p_4: Non-virtual-by-default semantics
- p_5: Abstract/interface separation constraints

- p_6: Compile-time inheritance restriction rules (e.g., prohibition of certain extensions)

4) Fragile Base Mitigation Factor $F_b(L)$ Reflects language-level safeguards against fragile base class effects:

$$F_b(L) = \frac{\sum_{i=1}^{k} s_i(L)}{k}$$

where $s_i(L)$ — binary variable reflecting whether safeguard i is explicitly defined at the language specification level. Comparison includes:

- s_1: Mandatory explicit override annotation (prevents accidental overriding)
- s_2: Strict method visibility control (prevents unintended exposure of overridable members)
- s_3: Requirement of explicit superclass invocation (controlled super/base calls)
- s_4: Variance restrictions ensuring type-safe overriding (covariance/contravariance constraints)
- s_5: Default non-overridable methods (opt-in polymorphism)
- s_6: Compile-time override consistency checks (signature and contract enforcement)

3.1.4 Composition and Alternatives to Inheritance

For the CRFI metric: the key idea of paying consideration flexibility through traits and mixins taken from the works of [32]. The component related to naming

conflicts and requirements for the using class is based on the formalization of traits and their composition mechanisms [32]. The idea of interpreting interfaces as carriers of behavior, not just contracts, is consistent with the evolution of default methods in languages with interfaces [33].

Metric Definition

Let the *Composition and Reuse Flexibility Index (CRFI)* be defined as:

$$CRFI(L) = \alpha \cdot D_{cls}(L) + \beta \cdot T_{exp}(L) + \gamma \cdot I_{comp}(L)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Class-Based Delegation Score $D_{cls}(L)$

$$D_{cls}(L) = \frac{K_{del}(L)+S_{fwd}(L)}{2}$$

where:

- $K_{del}(L)$ indicates available syntactic delegation constructs in the language.

$$K_{del}(L) = \begin{cases} 1, & \text{if available with keywords (e.g. } \mathrm{delegate}\text{),} \\ 0.5, & \text{if emulated via adapters,} \\ 0, & \text{if language does not provide delegation mechanisms.} \end{cases}$$

- $S_{fwd}(L)$ evaluates the presence of mechanisms that allows to automatically delegate method calls from one object to another without writing boilerplate code.

$$S_{fwd}(L) = \begin{cases} 1, & \text{if automatic delegation mechanisms are available,} \\ 0, & \text{if language does not provide automatic delegation.} \end{cases}$$

2) Trait Expressiveness Score $T_{exp}(L)$

$$T_{exp}(L) = \frac{R_{conf}(L)+O_{mod}(L)}{2}$$

where:

- $R_{conf}(L)$ evaluates name conflict resolution during mixing.

$$R_{conf}(L) = \begin{cases} 1, & \text{if resolution using explicit exclusion or aliasing,} \\ 0.5, & \text{if resolution using declaration order,} \\ 0, & \text{if compilation error.} \end{cases}$$

- $O_{mod}(L)$ evaluates the ability of composition mechanisms (tracks, mixins) to determine the requirements for the class that uses them.

$$O_{mod}(L) = \begin{cases} 1, & \text{if allows to declare requirements inside traits/mixins,} \\ 0, & \text{if traits/mixins should be completely self-constrained.} \end{cases}$$

3) Interface Composition Capacity $I_{comp}(L)$

$$I_{comp}(L) = \frac{N_{impl}(L)+D_{def}(L)}{2}$$

where:

- $N_{impl}(L)$ is the maximum number of interfaces a class can implement,

$$N_{impl}(L) = \begin{cases} 1, & \text{if number of interfaces is not restricted,} \\ 0.66, & \text{if the number of interfaces is limited, but it is greater than} \\ 0.33, & \text{if the number of interfaces is limited by 1 interface,} \\ 0, & \text{if there are no interfaces or their analogues.} \end{cases}$$

- $D_{def}(L)$ indicates the presence of default method implementations in interfaces.

$$D_{def}(L) = \begin{cases} 1, & \text{if allows to declare implementations inside interfaces.,} \\ 0, & \text{if interfaces contain only method signatures, constant, types.} \end{cases}$$

This parameter evaluates whether interfaces function solely as types or also as units of composable behavior, approaching the role of mixins.

3.1.5 Object Representation and Creation Model

The ORCI metric combines several lines of research: the distinction between object identity and value semantics goes back to classical works on object models [30], and the idea of multiple object creation paradigms goes back to research on prototype-based programming and meta-object models [34].

Metric Definition

Let the *Object Representation and Creation Index (ORCI)* be defined as:

$$ORCI(L) = \alpha \cdot S_{id}(L) + \beta \cdot S_{cr}(L) + \gamma \cdot S_{meta}(L)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Identity Semantics Score $S_{id}(L)$

Object identity is a core property distinguishing objects from values. This component measures the consistency of reference semantics across the language's type system.

$$S_{id}(L) = \frac{T_{ref}(L)}{T_{user}(L)}$$

where:

- $T_{user}(L)$ is the total number of user-definable type categories in the language specification.
- $T_{ref}(L)$ is the number of user-definable type categories that enforce reference semantics.

2) Instantiation Paradigm Diversity $S_{cr}(L)$

$$S_{cr}(L) = \frac{P_{supported}(L)}{P_{max}}$$

where:

- $P_{supported}(L)$ is the number of distinct instantiation paradigms supported natively (e.g., new Class, Prototype Clone, Literal Construction, Actor Spawn).
- P_{max} is the maximum number of paradigms observed across all compared languages.

3) Meta-level Extensibility Score $S_{meta}(L)$

$$S_{meta}(L) = \frac{I_{level}(L)}{I_{max}}$$

where: $I_{level}(L)$ represents the depth of intercession allowed (0 = none, 1 = introspection only, 2 = structural modification, 3 = creation protocol override).

$$I_{level}(L) = \begin{cases} 3, & \text{if allows overriding the object instantiation process via metaobjects,} \\ 2, & \text{if allows modification of existing object structures at runtime,} \\ 1, & \text{if allows querying object structure at runtime but prohibits modifica} \\ 0, & \text{if does not allows querying object structure and modification.} \end{cases}$$

I_{max} is the maximum intercession level defined for the comparison languages.

3.1.6 Polymorphism and Dispatch Mechanisms

The PDEI metric is based on research on polymorphism, multiple dispatch, and expression problem: the relationship between extensibility of behavior and the typical structure of language is discussed in [35]. The idea of separating secure

dispatch from just dynamic dispatch is based on the formal criteria of exhaustion and compile-time correctness, and the final/sealed/static hints component is associated with de-virtualization and limiting late binding [29].

Metric Definition

Let the *Polymorphism and Dispatch Efficiency Index (PDEI)* of a language L be defined as:

$$PDEI(L) = \alpha \cdot M_{scope}(L) + \beta \cdot S_{disp}(L) + \gamma \cdot O_{virt}(L)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Dispatch Scope Mechanism $M_{scope}(L)$

This component evaluates the richness of dispatch mechanisms supported by the language specification, distinguishing between single, multiple, and predicate dispatch.

$$M_{scope}(L) = \frac{w_{single} \cdot I_{single} + w_{multi} \cdot I_{multi} + w_{pred} \cdot I_{pred}}{w_{single} + w_{multi} + w_{pred}}$$

where:

- $I_{single} = 1$ if single dispatch (receiver-based) is supported; 0 otherwise.
- $I_{multi} = 1$ if multiple dispatch (argument-based) is natively supported; 0 otherwise.

- $I_{pred} = 1$ if predicate-based or condition-based dispatch is available; 0 otherwise.
- Weights are set as $w_{single} = 1$, $w_{multi} = 2$, $w_{pred} = 2$
  - Single dispatch weight is 1 because it often requires structural workarounds.
  - Multiple dispatch weight is 2 because it resolves the expression problem for operation extension without modifying existing ones.
  - Predicate dispatch weight is 2 since it extends polymorphism beyond type hierarchies into execution logic.

2) Dispatch Safety Guarantee $S_{disp}(L)$

$$S_{disp}(L) = \frac{I_{static}(L)+I_{exhaust}(L)}{2}$$

where:

- $I_{static}(L)$ captures whether the language enforces compile-time verification of dispatch correctness.
- $I_{exhaust}(L)$ evaluates whether the language enforces completeness of dispatch definitions.

$$I_{static}(L) = \begin{cases} 1, & \text{if the language guarantees dispatch safety at compile time} \\ 0, & \text{otherwise} \end{cases}$$

$$I_{exhaust}(L) = \begin{cases} 1, & \text{if the language enforces exhaustive dispatch definitions} \\ 0, & \text{otherwise} \end{cases}$$

3) Virtualization Optimization Potential $O_{virt}(L)$

$$O_{virt}(L) = \frac{K_{final}+K_{sealed}+K_{static}}{3}$$

where:

- $K_{final} = 1$ if language support final/non-virtual methods; 0 otherwise.
- $K_{sealed} = 1$ if language support sealed class hierarchies restricting subclassing; 0 otherwise.
- $K_{static} = 1$ if language support static dispatch hints (e.g., static methods, explicit devirtualization annotations); 0 otherwise.

3.1.7 State Model, Mutability, and Concurrency Guarantees

The SCSI metric is based on work on safe operation with the state and competitiveness of [36]. The idea to measure the default mutability constraint and static link constraints was based on research on ownership types, affine/linear reasoning, and aliasing control [36].

Metric Definition

Let the *State and Concurrency Safety Index (SCSI)* be defined as:

$$SCSI(L) = \alpha \cdot M_{def}(L) + \beta \cdot S_{dens}(L) + \gamma \cdot P_{conc}(L)$$

where L denotes the analyzed programming language, $\alpha + \beta + \gamma = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Default Mutability Score $M_{def}(L)$

$$M_{def}(L) = \begin{cases} 1, & \text{if objects are immutable by default} \\ 0.5, & \text{if mutability requires explicit annotation} \\ 0, & \text{if objects are mutable by default} \end{cases}$$

2) Static Reference Constraint Density $S_{dens}(L)$

$$S_{dens}(L) = \frac{N_{constr}(L)}{N_{ref}(L)}$$

where:

- $N_{constr}(L)$ is the number of distinct syntactic constructs in language L that impose static constraints on references.
- $N_{ref}(L)$ is the total number of distinct reference-related constructs in language L.

Formal Counting Rules

A construct is counted in $N_{ref}(L)$ if it:

- introduces, manipulates, or qualifies references (e.g., pointers, references, object handles),
- is explicitly represented in the language syntax or type system.

A construct is counted in $N_{constr}(L)$ if it:

- belongs to $N_{ref}(L)$, and
- enforces compile-time restrictions on aliasing, mutability, or usage of references.

3) Concurrency Primitive Safety $P_{conc}(L)$

$$P_{conc}(L) = \frac{S_{race}(L)+S_{model}(L)}{2}$$

where:

- $S_{race}(L) \in [0,1]$ measures data race freedom guarantees.

$$S_{race}(L) = \begin{cases} 1, & \text{if the type system proves data race freedom} \\ 0.5, & \text{if runtime checks exist} \\ 0, & \text{if reliance is purely on programmer discipline} \end{cases}$$

- $S_{model}(L) \in [0,1]$ measures the abstraction level of concurrency primitives.

$$S_{model}(L) = \begin{cases} 1, & \text{if provide high-level models(Actors, CSP channels)} \\ 0.5, & \text{if provide structured concurrency (tasks/futures)} \\ 0, & \text{if provide only low-level threads and locks} \end{cases}$$

3.1.8 Support for Formal Specification and Verifiability

The FSVI metric is based on Design by Contract, formulated in [37], where preconditions, postconditions, and invariants are first-class language concepts.

The distinction between native contracts, runtime checking, and static verification was added due to research that considers the program specification as a tool for formal correctness control [37].

Metric Definition

Let the *Formal Specification and Verifiability Index (FSVI)* be defined as:

$$FSVI(L) = \alpha \cdot C_{nat}(L) + \beta \cdot V_{enf}(L)$$

where L denotes the analyzed programming language, $\alpha + \beta = 1$ and each component is normalized to the interval $[0, 1]$.

Components

1) Native Contract Constructs Score $C_{nat}(L)$

$$C_{nat}(L) = \frac{n_{pre}+n_{post}+n_{inv}}{3}$$

- n_{pre} indicate the presence of first-class language constructs for preconditions.
- n_{post} indicate the presence of first-class language constructs for postconditions.
- n_{inv} indicate the presence of first-class language constructs for invariants.
- Values are assigned based on the language specification: 1 if the construct is syntactically native and semantically defined, 0 if reliant on external libraries or comments.

2) Verification Enforcement Level $V_{enf}(L)$

$$V_{enf}(L) = \begin{cases} 1.0, & \text{if static verification is supported (compile-time proof)} \\ 0.5, & \text{if only runtime checking is available} \\ 0.0, & \text{if no enforcement mechanism exists} \end{cases}$$

3.2 Parameter Estimation for Metric Coefficients

The selection of weighting coefficients for the proposed metrics is guided by the primary objective of this work: to emphasize those aspects of object models that most significantly affect extensibility, safety, and semantic clarity in object-oriented programming. The coefficients are calibrated to prioritize semantic soundness over syntactic convenience, explicitness over implicit behavior, and composability over rigid hierarchy structures. Additionally, separation of concerns—particularly between subtyping, inheritance, and composition—is critical for scalable software architecture [8], [38]. Therefore, higher weights are systematically assigned to components reflecting formal guarantees and semantic orthogonality.

3.2.1 Type System and Subtyping Model

$$\alpha = 0.3, \beta = 0.3, \gamma = 0.4$$

The highest weight is assigned to the inheritance–subtyping separation component ($\gamma = 0.45$). This decision is grounded in extensive literature identifying the conflation of inheritance and subtyping as a fundamental flaw in mainstream object-oriented languages [8]. Failure to distinguish them leads to violations of the Liskov Substitution Principle and undermines sound modular reasoning. Liskov and Wing further formalize this issue, noting that subtype relationships must preserve behavioral specifications independently of implementation structure [5]. This separation is treated as the most critical factor influencing the overall soundness of the object model.

The variance formalization component is assigned the second-highest weight

($\beta = 0.30$), as it directly impacts type safety in the presence of parametric polymorphism [29]. Historical issues in Java generics, such as the need for use-site variance (wildcards), illustrate the complexity introduced by insufficiently expressive or partially formalized variance systems.

The nominal–structural typing balance component receives a slightly lower weight ($\alpha = 0.25$). While important for flexibility and expressiveness, its impact on soundness is more indirect compared to the other components. Cardelli and Wegner argue that no single typing discipline suffices for all abstraction needs [1], motivating hybrid approaches that combine nominal and structural typing. Structural typing improves composability and reduces coupling, while nominal typing enhances readability and explicitness of design intent. Therefore, α is assigned a moderate weight, reflecting its role as an enabler of expressive type design rather than a primary determinant of soundness.

3.2.2 Abstraction and Encapsulation Mechanisms

$$\alpha = 0.20, \beta = 0.25, \gamma = 0.25, \delta = 0.30$$

The highest weight is assigned to the encapsulation enforcement coefficient ($\delta = 0.30$), as enforcement mechanisms represent the final guarantee that abstraction boundaries are respected at runtime. Encapsulation, as originally formulated by Parnas, is fundamentally about information hiding to reduce system complexity and improve maintainability [4]. However, this principle is only effective if the language prevents unauthorised access to the internal state.

The module isolation strength component is assigned $\beta = 0.25$, reflecting its importance in large-scale system design. Modular boundaries serve as the primary

mechanism for controlling visibility across compilation units and deployment artefacts. Parnas emphasises that modularisation should be based on design decisions which are likely to change [4], requiring strict control over exported interfaces. Languages with explicit export lists and strong module systems provide stronger guarantees than those with weaker namespace mechanisms.

The interface abstraction capacity component is also assigned $\gamma = 0.25$, as the ability to separate specification from implementation is fundamental to object-oriented design. Liskov and Wing emphasise that abstraction mechanisms must support behavioural subtyping independently of implementation details [5]. Languages that provide multiple abstraction constructs (interfaces, abstract classes, traits, protocols) enable more flexible and precise modelling of contracts.

The visibility granularity index receives the lowest weight ($\alpha = 0.20$), as fine-grained access modifiers contribute to encapsulation but are insufficient on their own. The presence of multiple visibility levels does not prevent violations if the language permits reflection or unsafe access.

3.2.3 Inheritance Semantics and Hierarchy Management

$$\alpha = 0.20, \beta = 0.25, \gamma = 0.30, \delta = 0.25$$

The highest weight is assigned to the constraint protection coefficient ($\gamma = 0.30$), as explicit language-level restrictions on inheritance are the primary mechanism for preventing incorrect or unsafe extension of class hierarchies. Unconstrained inheritance has long been recognised as a source of design fragility and unintended coupling. Meyer notes that unrestricted inheritance enables improper extension and redefinition, leading to violations of class invariants and behavioural

contracts [30].

The linearization determinism score is assigned $\beta = 0.25$, reflecting the importance of a formally defined and predictable method resolution order (MRO), especially in languages that support multiple inheritance or mixin composition. Ambiguity in method lookup leads to non-local reasoning and unpredictable behaviour.

The fragile base mitigation factor is assigned $\delta = 0.25$, as it captures safeguards against one of the most well-documented problems in object-oriented design. The fragile base class problem arises when modifications to a superclass unintentionally break derived classes due to hidden dependencies. Mikhajlov and Sekerinski describe this issue as a consequence of implicit assumptions about inherited behaviour that are not enforced by the type system [10].

The hierarchy simplicity factor receives the lowest weight ($\alpha = 0.20$), as limiting the number of direct superclasses reduces structural complexity but does not, by itself, guarantee correctness or robustness. Single inheritance eliminates ambiguity and simplifies reasoning, as noted by Booch, who argues that simpler hierarchies improve comprehensibility and maintainability [2]. However, multiple inheritance and mixin-based designs can provide expressive power when combined with proper linearization and constraint mechanisms.

3.2.4 Composition and Alternatives to Inheritance

$\alpha = 0.30, \beta = 0.35, \gamma = 0.35$

The highest weights are assigned to trait expressiveness and interface composition capacity ($\beta = 0.35$, $\gamma = 0.35$), as these mechanisms provide the most

semantically rich and scalable alternatives to classical inheritance. The shift toward composition over inheritance has been widely advocated in the literature. This principle is motivated by the need to reduce tight coupling and increase flexibility in system design. Trait and mixin systems represent a formalization of composition-based reuse. Similarly, interface composition capacity is highly significant, as modern languages increasingly extend interfaces beyond pure type declarations. As Gamma et al. state, favor object composition over class inheritance [6].

The class-based delegation score receives a slightly lower weight ($\alpha = 0.30$). Delegation is a well-established alternative to inheritance, particularly in prototype-based and dynamically typed languages. However, in many statically typed languages, delegation is often implemented through design patterns (e.g., Adapter, Decorator) rather than first-class language constructs, resulting in boilerplate code and reduced clarity. Languages that provide built-in delegation mechanisms reduce this overhead. However, delegation alone does not provide the same level of compositional structure and conflict resolution as traits or advanced interface systems. For this reason, Class-Based Delegation Score is assigned a slightly lower weight compared to the other components.

3.2.5 Object Representation and Creation Model

$$\alpha = 0.35, \beta = 0.30, \gamma = 0.35$$

The highest weights are assigned to identity semantics and meta-level extensibility ($\alpha = 0.35$, $\gamma = 0.35$), as these components directly determine the conceptual integrity and expressive power of the object model.

The identity semantics score ($\alpha = 0.35$) is critical because object identity is a defining property of object-oriented programming. As emphasized by Abadi and Cardelli, objects are characterized by identity, state, and behavior [39]. Inconsistent treatment of identity across user-defined types leads to semantic fragmentation, where some objects behave as values while others behave as references. This inconsistency complicates reasoning about aliasing, mutability, and side effects.

The meta-level extensibility score ($\gamma = 0.35$) is equally significant, as it captures the ability of a language to support reflection, intercession, and customization of object behavior. Metaobject protocols (MOPs), as described by Kiczales et al., enable programmers to adapt the language implementation to suit the needs of specific applications [34]. This includes modifying object layout, method dispatch, and instantiation semantics.

The instantiation paradigm diversity component receives a slightly lower weight ($\beta = 0.30$). The availability of multiple object creation paradigms—such as constructor-based instantiation, prototype cloning, literals, and actor-based spawning—enhances flexibility and supports different programming styles. However, while multiple instantiation mechanisms increase expressiveness, they do not necessarily improve semantic clarity or correctness. Excessive diversity may even introduce inconsistency if paradigms are not well integrated. Therefore, Instantiation Paradigm Diversity is assigned a substantial but secondary weight compared to identity semantics and meta-level control.

3.2.6 Polymorphism and Dispatch Mechanisms

$\alpha = 0.35, \beta = 0.35, \gamma = 0.30$

The highest weights are assigned to dispatch scope and dispatch safety ($\alpha = 0.35$, $\beta = 0.35$), as these components fundamentally determine both the expressive capabilities and the semantic reliability of polymorphic behavior.

The dispatch scope mechanism ($\alpha = 0.35$) captures the range of polymorphic abstractions supported by the language. Traditional object-oriented languages rely primarily on single dispatch, which restricts method selection to the dynamic type of the receiver. However, this model is widely recognized as insufficient for many extensibility scenarios. As Wadler describes in the context of the expression problem, single dispatch makes it difficult to extend operations without modifying existing code [40].

Multiple dispatch directly addresses this limitation by allowing method selection based on the dynamic types of all arguments, improving modular extensibility. Predicate dispatch further generalizes this concept by enabling selection based on arbitrary conditions, extending polymorphism beyond type hierarchies. Given its direct impact on the expressive power of object interaction, Dispatch Scope Mechanism is assigned a dominant weight.

The dispatch safety guarantee is assigned $\beta = 0.35$, reflecting the importance of correctness in method selection. Type-safe dispatch is essential for ensuring that method calls do not result in runtime failures. Therefore, Dispatch Safety Guarantee is weighted equally with dispatch scope.

The virtualization optimization potential receives a slightly lower weight ($\gamma = 0.30$), as it primarily affects performance rather than expressiveness or correctness. Virtual method dispatch introduces runtime overhead, and modern languages provide mechanisms to mitigate this through devirtualization techniques. While these mechanisms are important for performance-critical systems, they do not alter the fundamental semantics of polymorphism.

3.2.7 State Model, Mutability, and Concurrency Guarantees

$$\alpha = 0.35, \beta = 0.25, \gamma = 0.40$$

The highest weight is assigned to the concurrency primitive safety component ($\gamma = 0.40$), as concurrency-related errors constitute one of the most critical and difficult classes of defects in modern software systems. As emphasized by *Adve and Boehm*, data races lead to undefined or unpredictable behavior in most programming language memory models [41]. Languages that provide formal guarantees of race freedom or enforce structured concurrency significantly reduce the cognitive burden on developers and improve system reliability. Consequently, the ability of a language to provide safe and expressive concurrency primitives directly impacts its suitability for large-scale object-oriented systems.

The default mutability score ($\alpha = 0.35$) is assigned the second highest weight due to its strong influence on both reasoning about program state and interaction with concurrency. Immutability has been widely recognized as a key mechanism for reducing complexity in concurrent systems. As noted by *Bloch*, immutable objects are inherently thread-safe [42]. Therefore, default immutability or explicit mutability annotations significantly enhance both safety and predictability.

The static reference constraint density component ($\beta = 0.25$) is assigned a slightly lower, yet still substantial, weight. Static constraints on references—such as ownership types, borrowing systems, or const-correctness—play an important role in preventing aliasing-related errors and enforcing safe usage patterns. Clarke et al. demonstrate that ownership type systems can statically guarantee encapsulation and alias control [36]. However, in practice, many mainstream object-oriented languages either lack such mechanisms or provide them in limited

form (e.g., const in C++), reducing their overall effectiveness. Furthermore, while static reference constraints improve safety, they are often complex and may hinder usability if not carefully designed. As a result, their impact, though important, is considered secondary to the more fundamental concerns of mutability defaults and concurrency safety.

3.2.8 Support for Formal Specification and Verifiability

$$\alpha = 0.45, \beta = 0.55$$

The verification enforcement component ($\beta = 0.55$) is assigned a slightly higher weight, as the mere presence of specification constructs does not guarantee their practical effectiveness without enforcement mechanisms. Formal specifications only contribute to software reliability if they are systematically checked. Static verification, in particular, enables compile-time guarantees about program correctness, significantly reducing the risk of runtime failures. Meyer further argues that contracts are only meaningful when they are both documented and automatically checked [37]. Therefore, enforcement—especially at compile time—plays a decisive role in ensuring that formal specifications are not merely descriptive but operational.

The native contract constructs component ($\alpha = 0.45$) is assigned a slightly lower, yet still substantial, weight. The presence of built-in constructs for preconditions, postconditions, and invariants reflects the language's commitment to formal reasoning and design-by-contract principles. Meyer introduced Design by Contract as a fundamental methodology for object-oriented programming, stating that software elements should be specified by precise and verifiable interface

conditions [30]. Languages that natively support such constructs (e.g., Eiffel) provide clearer semantics and better tool integration compared to those relying on external libraries or annotations. However, the developers often neglect or inconsistently apply contract annotations when they are not enforced or integrated into the compilation process.

Chapter 4

Implementation

This chapter provides calculations based on the methodological framework developed in the previous sections and describes in detail the application of the proposed assessment approach. Earlier, in the methodology chapter, the criteria were formalised as a set of measurable metrics. This transformation allows for a structured and reproducible comparison between programming languages, transforming the conceptual properties of object models to quantitative indicators.

In particular, this chapter describes the implementation of the metric calculation process and its application to a selected set of ten modern programming languages: C#, C++, C, Golang, Java, Python, Ruby, JavaScript, Scala, Smalltalk, and Zonnon. For each language, certain indicators are calculated based on the analysis of language specifications. This stage of implementation serves as the empirical basis of the research. The results of the calculations will allow for a comparative assessment of object models, identify the strengths and weaknesses of different languages.

4.1 Metrics Evaluation

4.1.1 Metrics Evaluation for C#

Introduction

C# is a statically typed, object-oriented programming language standardized as ECMA-334 and commonly used in conjunction with the Common Language Runtime (CLR) [43]. The type system and object model include nominal typing, interface-based abstraction, and managed runtime semantics.

In this section, we evaluate the language against the defined metrics, grounding each assessment in the formal specification and relevant academic literature.

Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

C# employs a strictly nominal type system in which subtypes are explicitly declared using type names, inheritance relationships, and the implementation of the interface [43]. Structural typing is not supported at the language level.

$I_{struct}(\text{C\#}) = 0, I_{nom}(\text{C\#}) = 1 \therefore S_{nom}(\text{C\#}) = \frac{0+1}{2} = 0.5$

2) Variance Formalization Score $S_{var}(L)$

C# supports generics with explicit variance annotations (in, out) for interfaces and delegates, introduced in C# 4.0. These are formally constrained to preserve type safety. However, it only applies to interfaces and delegates but does not apply to classes.

$V_{exp}(\text{C\#}) = 1, V_{sound}(\text{C\#}) = 1 \therefore S_{var}(\text{C\#}) = 1$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, C# explicitly supports subtyping independently of class inheritance

through interface implementation: a class or struct may implement one or more interfaces without inheriting implementation, thereby introducing subtyping relations decoupled from class inheritance.

$I_{dec}(\text{C\#}) = 1$

Second, subtyping rules in C# are not strictly derived from inheritance semantics. The type system defines subtyping via multiple mechanisms, including interface implementation, variance in generics, and implicit reference conversions, indicating independence from purely inheritance-based relations.

$I_{ind}(\text{C\#}) = 1$

Third, although the C# specification distinguishes between inheritance (class derivation) and interface-based subtyping, this distinction is not fully formalized as separate mathematical relations but rather described operationally within the type system rules.

$I_{form}(\text{C\#}) = 0.5$

$S_{rel}(\text{C\#}) = \frac{1+1+0.5}{3} \approx 0.83$

Final metric value:

$$\begin{aligned} TSSI(\text{C\#}) = 0.3 \cdot S_{nom}(\text{C\#}) + 0.3 \cdot S_{var}(\text{C\#}) + 0.4 \cdot S_{rel}(\text{C\#}) \\ = 0.3 \cdot 0.5 + 0.3 \cdot 1 + 0.4 \cdot \frac{5}{6} \\ \approx 0.7833 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

C# defines seven access modifiers: private, protected, internal, protected

internal, private protected, public, file [43].

$$V(\text{C\#}) = \frac{7}{7}$$

2) Module Isolation Strength $M(L)$

Namespaces do not enforce strict encapsulation (no export lists) File enforce strict encapsulation, but there is not export mechanism and partially extends access Assembly fits all criteria except the export list. The export list is not explicitly specified, but is set using access modifiers.

$$M(\text{C\#}) = 0$$

3) Interface Abstraction Capacity $I(L)$

C# provides interfaces, abstract classes and delegate for construction dedecited to behavior abstraction C# provides class, struct, record and record struct for construction dedecited to behavior abstraction

$$I(\text{C\#}) = \frac{3}{4} = 0.75$$

4) Encapsulation Enforcement Coefficient $E(L)$

C# provide encapsulation bypass using: reflection and unsafe code, flag InternalsVisibleTo, Invoke mechanism:

$$E(\text{C\#}) = 0$$

Final metric value:

$$\begin{aligned} AE(\text{C\#}) &= 0.2 \cdot V(\text{C\#}) + 0.25 \cdot M(\text{C\#}) + 0.25 \cdot I(\text{C\#}) + 0.3 \cdot E(\text{C\#}) \\ &= 0.2 \cdot 1 + 0.25 \cdot 0 + 0.25 \cdot 0.75 + 0.3 \cdot 0 \\ &= 0.3875 \end{aligned}$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

C# supports only single class inheritance, but supports multiple interface inheritance:

$$H_s(\text{C\#}) = 1 - \frac{1-1}{M_{max}-1} = 1$$

2) Linearization Determinism Score $L_d(L)$

In C#, class inheritance is strictly single (i.e., a class may inherit from exactly one base class), and interface implementation does not introduce ambiguity in method resolution because interfaces do not provide inherited implementation in the same sense (default interface methods are resolved with well-defined precedence rules). The specification defines a deterministic lookup process: member resolution proceeds along the class inheritance chain, and in the presence of interfaces, explicit rules govern conflict resolution.

$L_d(\text{C\#}) = 1$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 1$: C# provides explicit finalization mechanisms via the sealed modifier for classes and methods, preventing further inheritance or overriding.

$p_2 = 0$: while sealed prevents extension, C# does not support fully closed hierarchies with enumerated subclasses (as in algebraic data types), thus lacking strict sealed hierarchy sets.

$p_3 = 1$: C# enforces explicit override semantics through the mandatory override keyword for redefining virtual members.

$p_4 = 1$: methods are non-virtual by default and must be explicitly marked virtual, abstract, or override to participate in polymorphism.

$p_5 = 1$: the language enforces a clear separation between abstract classes and interfaces, with distinct constraints on implementation and inheritance.

$p_6 = 1$: the compiler enforces multiple inheritance restrictions (e.g., no multiple class inheritance, constraints on base class selection, and rules for interface implementation).

$$C_p(\text{C\#}) = \frac{1+0+1+1+1+1}{6} \approx 0.83$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 1$: C# requires explicit $\mathrm{override}$ annotations, preventing accidental overriding.

$s_2 = 1$: strict access modifiers (e.g., $\mathrm{private}$, $\mathrm{protected}$, $\mathrm{internal}$) control the visibility and overridability of members, limiting unintended exposure.

$s_3 = 0$: invocation of base class implementations via base is optional rather than mandatory, thus not enforcing controlled superclass delegation.

$s_4 = 1$: C# enforces variance rules (covariance and contravariance) in generics and method overriding to preserve type safety.

$s_5 = 1$: methods are non-virtual by default, requiring explicit opt-in for polymorphic behavior.

$s_6 = 1$: the compiler enforces strict override consistency, including signature matching and return type compatibility.

$$F_b(\text{C\#}) = \frac{1+1+0+1+1+1}{6} = \frac{5}{6} \approx 0.83$$

Final metric value:

$$\begin{aligned} ISRI(\text{C\#}) &= 0.2 \cdot H_s(\text{C\#}) + 0.25 \cdot L_d(\text{C\#}) + 0.3 \cdot C_p(\text{C\#}) + 0.25 \cdot F_b(\text{C\#}) \\ &= 0.2 \cdot 1 + 0.25 \cdot 1 + 0.3 \cdot \frac{5}{6} + 0.25 \cdot \frac{5}{6} \\ &\approx 0.9083 \end{aligned}$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

C# does not provide native delegation constructs: The delegate keyword exists in C# [43]. It is a reference type that encapsulates a reference to a method with a specific signature. It is actively used for events, callbacks, and asynchronous patterns. There is no built-in syntax for automatic forwarding in C#. The developer must manually proxy the calls.

$$K_{del}(\text{C\#}) = 1, S_{fwd}(\text{C\#}) = 0$$

$$D_{cls}(\text{C\#}) = 0.5$$

2) Trait Expressiveness Score $T_{exp}(L)$

When using interfaces with DIM, if a class implements two interfaces containing methods with the same signature and default implementation, the C# compiler does not resolve the conflict automatically. C# does not allow an interface/trait to require the implementing class to have arbitrary methods, properties, or fields that are not included in the interface's contract.

$$R_{conf}(\text{C\#}) = 0, O_{mod}(\text{C\#}) = 0 \therefore T_{exp}(\text{C\#}) = 0$$

3) Interface Composition Capacity $I_{comp}(L)$

In C#, a class can implement multiple interfaces, and there is no Strict upper limit. Starting with C# 8.0, default interface methods have been introduced. Now the interface can contain: method implementations, static methods, private methods (can be reused inside the interface).

$$I_{comp}(\text{C\#}) = \frac{N_{impl}(\text{C\#}) + D_{def}(\text{C\#})}{2} = \frac{1+1}{2} = 1$$

Final metric value:

$$\begin{aligned} CRFI(\text{C\#}) &= 0.3 \cdot D_{cls}(\text{C\#}) + 0.35 \cdot T_{exp}(\text{C\#}) + 0.35 \cdot I_{comp}(\text{C\#}) \\ &= 0.3 \cdot 0.5 + 0.35 \cdot 0 + 0.35 \cdot 1 \\ &= 0.5 \end{aligned}$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

Type categories that can be declared in C# [43]:

- class - reference type
- struct - value type
- interface - reference type
- enum - value type
- delegate - reference type

$$S_{id}(\text{C\#}) = \frac{3}{5} = 0.6$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

In the C# language, four semantically different paradigms of object creation are distinguished:

- $Constructor-based(new)$ defined by the specification as the main mechanism for creating instances of reference and meaningful types.
- *Literalinitialization* allows the creation of objects without explicitly calling the constructor at the syntax level.
- $Copyinginstantiation(\mathrm{with}expressionsfor\mathrm{record})$ implementing non-destructive mutation and representing a separate functional creation model.
- $Stackmemory(stackalloc)$ provides an alternative model for managing the lifetime of objects outside of heap allocation

Other mechanisms, such as object initializers, reflection, or $\mathrm{ICloneable}$, do not form separate paradigms, since they are either syntactic extensions of new, or library level constructions rather than to linguistic semantics.

$$S_{cr}(\text{C\#}) = \frac{4}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

The .NET platform specification provides advanced introspection through the $\mathrm{System.Reflection}$ namespace, which corresponds to level $I_{level} = 1$, allowing the structure of types and objects to be queried at runtime. Additionally, the language allows limited modification of structures at runtime via $\mathrm{ExpandoObject}$, dynamic typing ($\mathrm{dynamic}$), and type generation through $\mathrm{System.Reflection.Emit}$, which

satisfies the criterion for level $I_{level} = 2$. However, C# does not support the meta-object protocol and does not allow the object creation process to be redefined (e.g., the semantics of the new operator), which precludes achieving level 3.

$$I_{level}(\text{JS/Python/Ruby}) = 3$$
$$S_{meta}(\text{C\#}) = \frac{2}{3}$$

Final metric value:

$$\begin{aligned} ORCI(\text{C\#}) &= 0.35 \cdot S_{id}((\text{C\#})) + 0.3 \cdot S_{cr}((\text{C\#})) + 0.35 \cdot S_{meta}((\text{C\#})) \\ &= 0.35 \cdot 0.6 + 0.3 \cdot 1 + 0.35 \cdot \frac{2}{3} \\ &\approx 0.7433 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

C# natively supports single dispatch via virtual method invocation, where method resolution depends on the runtime type of the receiver.

$I_{single}(\text{C\#}) = 1$

However, it does not provide true multiple dispatch, as method overloading is resolved at compile time and does not consider the runtime types of all arguments.

$I_{multi}(\text{C\#}) = 0$

The language supports a form of predicate-based dispatch through pattern matching constructs (e.g., switch expressions with guards and is-patterns), which enable branching based on both type and additional logical conditions.

$I_{pred}(\text{C\#}) = 1$

$$M_{scope}(\text{C\#}) = \frac{1*1+2*0+2*1}{5} = 0.6$$

2) Dispatch Safety Guarantee $S_{disp}(L)$

First, C# enforces compile-time verification of method dispatch correctness through static type checking: method calls, virtual dispatch resolution, and overload selection must be type-safe and valid at compile time, with the compiler rejecting ambiguous or invalid dispatch scenarios.

$I_{static}(\text{C\#}) = 1$

Second, C# does not enforce exhaustiveness of dispatch definitions. The virtual method overriding does not require covering all possible derived types, and pattern matching (e.g., switch expressions) is not universally required to be exhaustive unless specific constructs (like enums or sealed types) are used, thus lacking a general guarantee.

$I_{exhaust}(\text{C\#}) = 0$

$$S_{disp}(\text{C\#}) = \frac{1+0}{2} = 0.5$$

3) Virtualization Optimization Potential $O_{virt}(L)$

First, C# supports non-virtual methods by default and allows explicitly sealing overridden methods using the sealed modifier, satisfying the condition.

$K_{final}(\text{C\#}) = 1$

Second, the language provides sealed class hierarchies via the sealed keyword, preventing further inheritance and enabling more aggressive compile-time and JIT optimizations, thus $K_{sealed} = 1$. Third, C# supports static dispatch through static methods, which are resolved at compile time, as well as recent extensions such as static abstract interface members that further strengthen compile-time

polymorphism.

$K_{static} = 1$

$O_{virt}(\text{C\#}) = 1$

Final metric value:

$$\begin{aligned} PDEI(\text{C\#}) &= 0.35 \cdot M_{scope}(\text{C\#}) + 0.35 \cdot S_{disp}(\text{C\#}) + 0.3 \cdot O_{virt}(\text{C\#}) \\ &= 0.35 \cdot 0.6 + 0.35 \cdot 0.5 + 0.3 \cdot 1 \\ &= 0.685 \end{aligned}$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

All instances of reference and value types are mutable by default: fields can be modified after initialization, and property accessors do not restrict modification without additional modifiers. Immutability is achieved exclusively through explicit language constructs, such as readonly for fields, init for properties, or the use of the record type, which confirms the absence of immutability by default at the language level.

$M_{def}(\text{C\#}) = 0$

2) Static Reference Constraint Density $S_{dens}(L)$

Reference-related constructs in C# language include [43]:

- reference types (classes, interfaces, arrays, delegates)
- ref

- out
- in
- unsafe pointers (*, &)
- nullable reference types (T?)
- ref struct (e.g., Span<T>)
- this
- base
- dynamic

Among these, constructs imposing compile-time constraints:

- ref aliasing and definite assignment rules
- out mandatory assignment before return
- in (read-only enforcement)
- nullable reference types static - flow analysis for null safety
- ref struct stack-only lifetime and escape restrictions
- unsafe pointers - restricted to unsafe context

$S_{dens}(\text{C\#}) = \frac{6}{10} = 0.6$

3) Concurrency Primitive Safety $P_{conc}(L)$

According to the C# specification and .NET memory model, C# does not provide compile-time guarantees of data race freedom. Instead, it relies on programmer discipline with constructs such as lock, Monitor, and volatile, without enforcing race freedom via the type system or runtime detection:

$S_{race}(\text{C\#}) = 0$

C# provides structured concurrency through Task, async/await, and the Task Parallel Library, which encapsulate threads and support composable asynchronous workflows but do not enforce actor-style isolation or channel-based communication as core language primitives:

$S_{model}(\text{C\#}) = 0.5$

$P_{conc}(\text{C\#}) = 0.25$

Final metric value:

$$\begin{aligned} SCSI(\text{C\#}) &= 0.35 \cdot M_{def}(\text{C\#}) + 0.25 \cdot S_{dens}(\text{C\#}) + 0.4 \cdot P_{conc}(\text{C\#}) \\ &= 0.35 \cdot 0 + 0.25 \cdot 0.6 + 0.4 \cdot 0.25 \\ &= 0.25 \end{aligned}$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

C# does not include first-class syntactic constructs for design-by-contract elements such as preconditions, postconditions, or invariants. Such functionality is historically provided via external mechanisms like the .NET Code Contracts library or manual assertions, which are not part of the core language syntax or semantics. $n_{pre} = 0, n_{post} = 0, n_{inv} = 0$

$$C_{nat}(\text{C\#}) = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

C# does not provide built-in static verification mechanisms capable of proving program correctness properties at compile time, as its type system is not dependent or refinement-based. However, the language and runtime support extensive runtime checking, including type safety, array bounds checking, null reference checks, and optional contract-like validation via assertions or guard clauses, all enforced during execution by the CLR. Since these checks occur at runtime rather than as compile-time proofs:

$$V_{enf}(\text{C\#}) = 0.5$$

Final metric value:

$$\begin{aligned} FSVI(\text{C\#}) &= 0.45 \cdot C_{nat}(\text{C\#}) + 0.55 \cdot V_{enf}(\text{C\#}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0.5 \\ &= 0.275 \end{aligned}$$

Conclusion

The evaluation shows that C# demonstrates strong performance in inheritance semantics and encapsulation, reflecting its design as a pragmatic industrial OO language. However, limitations arise in composition mechanisms, concurrency safety, and formal verifiability, confirming observations in prior studies that mainstream OO languages prioritize usability and backward compatibility over formal rigor [29], [44]. These limitations motivate the need for improved object

model designs in next-generation languages.

4.1.2 Metrics Evaluation for Java

Introduction

Java represents a canonical example of a statically typed, nominally structured object-oriented language with a formally specified type system and execution model defined in the Java Language Specification (JLS) [45]. Its design emphasizes type safety, backward compatibility, and controlled extensibility, making it a suitable subject for evaluating object model characteristics. In this section, the previously defined metrics are instantiated for Java based strictly on its formal specification and supported by established research literature on its type system and object semantics.

Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

As defined by the *Java Language Specification (JLS)* [45]: subtype relationships in Java are established exclusively through explicit declarations such as $\mathrm{extends}$ and $\mathrm{implements}$, and not by structural equivalence. All subtype relations must be declared in class or interface definitions according to the formal typing judgments in the specification, which sutisfy explicit subtype declarations.

$$I_{struct}(\text{Java}) = 0, I_{nom}(\text{Java}) = 1 \Rightarrow S_{nom}(\text{Java}) = \frac{0+1}{2} = 0.5$$

2) Variance Formalization Score $S_{var}(L)$

Java does not allow declaration-site variance (i.e., no explicit in/out anno-

tations on type parameters), but supports use-site variance via wildcards (e.g., $?\ \text{extends}\ \text{T}$, $?\ \text{super}\ \text{T}$), which constitutes a restricted and indirect form of variance. Regarding the soundness component, JLS provides precise typing judgments and constraints on wildcard capture and subtyping that ensure type safety, albeit through restrictions such as invariance of generic types and limitations on covariance.

$$V_{exp}(\text{Java}) = 0.5, V_{sound}(\text{Java}) = 0.5$$
$$S_{var}(\text{Java}) = \frac{0.5 + 0.5}{2} = 0.5$$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, Java allows explicit subtyping without class inheritance through interfaces: a class may implement one or more interfaces without inheriting implementation, which establishes subtype relations independently of class extension. However, this mechanism is restricted to interfaces.

$I_{dec}(\text{Java}) = 0.5$

Second, subtyping in Java is partially independent from inheritance semantics: while class inheritance (extends) directly induces subtyping, interface implementation (implements) defines subtype relations without sharing implementation, indicating partial independence.

$I_{ind}(\text{Java}) = 0.5$

Third, the JLS distinguishes between subclassing and subtyping at the level of types (e.g., class types, interface types, and their assignment compatibility rules), but this distinction is not fully formalized as two completely separate relations in the specification.

$I_{form}(\text{Java}) = 0.5$

$$S_{rel}(\text{Java}) = \frac{0.5+0.5+0.5}{3} = 0.5$$

Final metric value:

$$\begin{aligned} TSSI(\text{Java}) &= 0.3 \cdot S_{nom}(\text{Java}) + 0.3 \cdot S_{var}(\text{Java}) + 0.4 \cdot S_{rel}(\text{Java}) \\ &= 0.3 \cdot 0.5 + 0.3 \cdot 0.5 + 0.4 \cdot 0.5 \\ &= 0.5 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

Java defines four access scopes: private (accessible only within the declaring class), default/package-private (accessible within the same package), protected (accessible within the package and by subclasses), and public (accessible from any context)[45].

$$n_{levels}(\text{C\#}) = 7 \Rightarrow V(\text{Java}) = \frac{4}{7}$$

2) Module Isolation Strength $M(L)$

The Java Platform Module System (JPMS) introduces explicit module boundaries with controlled exports. Java provides two primary modular boundaries: (i) packages, defined in JLS §7, which organize types but do not enforce explicit export control (all public types are accessible unless restricted externally), and (ii) modules, defined in the JPMS (module-info.java), which enforce strict encapsulation via explicit exports and requires directives.

$$b_{strict}(\text{Java}) = 1, b_{total}(\text{Java}) = 2$$
$$M(\text{Java}) = \frac{1}{2} = 0.5$$

3) Interface Abstraction Capacity $I(L)$

Java provides two primary abstraction constructs: (i) interface, which defines pure behavioral contracts with method signatures (JLS §9), and (ii) abstract class, which allows partial implementation alongside abstract methods (JLS §8.1.1.1). The object-defining constructs in Java are classes records, enums and annotations.

$$n_{abstraction_constructs}(\text{Java}) = 2, n_{object}(\text{Java}) = 4$$
$$I(\text{Java}) = \frac{2}{4} = 0.5$$

4) Encapsulation Enforcement Coefficient $E(L)$

While Java enforces access control statically via modifiers such as private, protected, and public (JLS §6.6), it also provides reflection capabilities through the core reflection API (java.lang.reflect), which allows runtime inspection and, critically, the ability to override access checks via mechanisms such as setAccessible(true) [42]. Furthermore, low-level facilities (e.g., Unsafe and deep reflection) enable direct memory or field access outside standard type safety guarantees.

$$E(\text{Java}) = 0$$

Final metric value:

$$\begin{aligned} AE(\text{Java}) &= 0.2 \cdot V(\text{Java}) + 0.25 \cdot M(\text{Java}) + 0.25 \cdot I(\text{Java}) + 0.3 \cdot E(\text{Java}) \\ &= 0.2 \cdot \frac{4}{7} + 0.25 \cdot 0.5 + 0.25 \cdot 0.5 + 0.3 \cdot 0 \\ &= 0.3643 \end{aligned}$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

Java enforces single inheritance for classes, allowing each class to declare at most one direct superclass via the $\mathrm{extends}$ clause. Although Java supports multiple inheritance of type through interfaces, this does not affect $M(L)$, as the metric explicitly concerns superclass declarations.

$$H_s(\text{Java}) = 1$$

2) Linearization Determinism Score $L_d(L)$

According to the Java Language Specification (JLS), method lookup is strictly defined through a well-ordered procedure: class inheritance follows a single inheritance model for classes, ensuring a unique linear superclass chain, while multiple inheritance of behavior is permitted only via interfaces, whose default methods are resolved using explicitly defined rules (JLS §8.4.8, §9.4.1). In particular, Java enforces deterministic conflict resolution by prioritizing class methods over interface default methods, and requiring explicit disambiguation when multiple interfaces provide conflicting defaults.

$$L_d(\text{Java}) = 1$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 1$: Java supports finalization of classes and methods via the final keyword, preventing inheritance or overriding.

$p_2 = 1$: It also introduces sealed classes and interfaces (since Java 17), enabling explicitly restricted subclassing sets.

$p_3 = 0$: Override enforcement is not mandatory: the $\mathrm{@Override}$ annotation is optional and not required by the compiler.

$p_4 = 0$: Methods in Java are virtual by default (except when declared final, $\mathrm{private}$, or static), so non-virtual-by-default semantics are absent.

$p_5 = 1$: Java enforces a clear distinction between abstract classes and interfaces at the syntactic and semantic level, including rules for implementation and inheritance.

$p_6 = 1$: the JLS defines compile-time inheritance restrictions (e.g., a class cannot extend multiple classes, cannot extend a final class, and must implement all abstract methods).

$$C_p(\text{Java}) = \tfrac{1+1+0+0+1+1}{6} \approx 0.67$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 0$: Java does not require mandatory explicit override annotations, as $\mathrm{@Override}$ is optional.

$s_2 = 1$: It enforces strict method visibility via access modifiers ($\mathrm{private}$, $\mathrm{protected}$, public), limiting exposure of overridable members.

$s_3 = 1$: Java requires explicit superclass constructor invocation either implicitly or explicitly using super, ensuring controlled initialization.

$s_4 = 1$: The JLS enforces type-safe overriding through return-type covari-

ance and parameter invariance, preventing unsafe variance.

$s_5 = 0$: methods are virtual by default (except final, private, static), so non-overridable-by-default semantics are absent.

$s_6 = 1$: Java performs compile-time checks for overriding consistency (method signatures, return types, checked exceptions), ensuring correctness.

$$F_b(\text{Java}) = \tfrac{0+1+1+1+0+1}{6} \approx 0.67$$

Final metric value:

$$\begin{aligned} \mathit{ISRI}(\text{Java}) &= 0.2 \cdot H_s(\text{Java}) + 0.25 \cdot L_d(\text{Java}) + 0.3 \cdot C_p(\text{Java}) + 0.25 \cdot F_b(\text{Java}) \\ &= 0.2 \cdot \frac{4}{7} + 0.25 \cdot 1 + 0.3 \cdot \frac{4}{6} + 0.25 \cdot \frac{4}{6} \\ &\approx 0.731 \end{aligned}$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

Java does not provide native syntactic delegation keywords (such as delegate in other languages). The delegation is typically implemented manually via composition and adapter patterns, where methods explicitly forward calls to a contained object. The Java Language Specification does not define any mechanism for automatic method forwarding. All delegation logic must be written explicitly by the programmer, and no built-in feature generates forwarding methods implicitly.

$$\begin{aligned} &K_{del}(\text{Java}) = 0.5, S_{fwd}(\text{Java}) = 0 \\ &\Rightarrow D_{cls}(\text{Java}) = \frac{0.5 + 0}{2} = 0.25 \end{aligned}$$

2) Trait Expressiveness Score $T_{exp}(L)$

In the presence of multiple inherited default methods with the same signature, Java does not support conflict resolution via aliasing or exclusion. It enforces a compile-time error unless the implementing class explicitly overrides the conflicting method and resolves the ambiguity (JLS §8.4.8, §9.4.1.3). The interfaces cannot declare explicit requirements on the implementing class beyond method signatures. While abstract methods define obligations, there is no mechanism to express constraints such as required fields or dependent behavior within the interface itself.

$$R_{conf}(\text{Java}) = 0, O_{mod}(\text{Java}) = 0$$
$$\Rightarrow T_{exp}(\text{Java}) = 0$$

3) Interface Composition Capacity $I_{comp}(L)$

Java allows a class to implement an arbitrary number of interfaces without any formal upper bound (JLS §8.1.5), which corresponds to unrestricted composition. Since Java 8, interfaces may include *default methods* that provide concrete implementations (JLS §9.4.1), enabling interfaces to encapsulate reusable behavior in addition to type abstraction.

$$N_{impl}(\text{Java}) = 1, D_{def}(\text{Java}) = 1$$
$$\Rightarrow I_{comp}(\text{Java}) = 1$$

Final metric value:

$$\begin{aligned} \mathit{CRFI}(\text{Java}) &= 0.3 \cdot D_{cls}(\text{Java}) + 0.35 \cdot T_{exp}(\text{Java}) + 0.35 \cdot I_{comp}(\text{Java}) \\ &= 0.3 \cdot 0.25 + 0.35 \cdot 0 + 0.35 \cdot 1 \\ &= 0.425 \end{aligned}$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

The primary user-definable type categories are classes, interfaces, enums, records, and annotation types. All of these are reference types, meaning variables hold references to objects rather than values, and identity is preserved via reference equality.

$$S_{id}(\text{Java}) = 1$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

The primary instantiation mechanism is explicit construction via the new operator, which applies uniformly to classes, arrays, and records. Additionally, Java supports object creation via reflective instantiation (e.g., $\mathrm{Class::newInstance}$, $\mathrm{Constructor::newInstance}$), which constitutes a distinct paradigm as it defers type binding to runtime. Furthermore, Java provides cloning through the $\mathrm{clone()}$ method, representing a prototype-based instantiation style, albeit with restricted and non-uniform semantics. Other paradigms such as literal object construction (beyond strings and primitives) or actor-based spawning are not supported at the language level.

$$P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{Java}) = \tfrac{3}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

The Java Language Specification, together with the core reflection API (java.lang.reflect), allows comprehensive introspection of classes, methods, fields, and annotations at runtime, enabling programs to query structural properties dynamically. However, Java does not permit direct structural modification of existing classes or objects at runtime (e.g., adding fields or methods), as the type system remains statically fixed after class loading. Furthermore, Java does not support overriding the object instantiation protocol via metaobjects.

$$I_{level}(\text{Java}) = 1, I_{level}(\text{JS/Python/Ruby}) = 3 \Rightarrow S_{meta}(\text{Java}) = \frac{1}{3}$$

Final metric value:

$$\begin{aligned} ORCI(\text{Java}) &= 0.35 \cdot S_{id}(\text{Java}) + 0.3 \cdot S_{cr}(\text{Java}) + 0.35 \cdot S_{meta}(\text{Java}) \\ &= 0.35 \cdot 1 + 0.3 \cdot \frac{3}{4} + 0.35 \cdot \frac{1}{3} \\ &\approx 0.6917 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

Java natively supports single dispatch, where method selection is based on the dynamic type of the receiver object.

$I_{single}(\text{Java}) = 1$.

However, Java does not support multiple dispatch, as method overloading is resolved statically at compile time based on declared types rather than dynamically

across all arguments.

$I_{multi}(\text{Java}) = 0$

Similarly, predicate-based dispatch—where method selection depends on arbitrary runtime conditions—is not part of the language specification and must be emulated through explicit conditional logic.

$I_{pred}(\text{Java}) = 0$

$M_{scope}(\text{Java}) = \frac{1}{5} = 0.2$

2) Dispatch Safety Guarantee $S_{disp}(L)$

First, Java ensures compile-time verification of method dispatch correctness through static type checking: method calls are validated against the declared type, and overload resolution is performed at compile time, guaranteeing that only valid methods can be invoked.

$I_{static}(\text{Java}) = 1$

Second, Java does not enforce exhaustiveness of dispatch definitions in general polymorphic scenarios (e.g., method overriding or dynamic dispatch across class hierarchies). Although newer features such as pattern matching in switch with sealed types introduce partial exhaustiveness checks, this is limited and not a general property of the dispatch system.

$I_{exhaust}(\text{Java}) = 0$

$S_{disp}(\text{Java}) = \frac{1+0}{2} = 0.5$

3) Virtualization Optimization Potential $O_{virt}(L)$

First, Java supports the final keyword for methods and classes, preventing

overriding and enabling static binding.

$K_{final}(\text{Java}) = 1$

Second, since Java 17, the language includes sealed classes and interfaces, which restrict subclassing to a known finite set and allow more precise dispatch analysis.

$K_{sealed}(\text{Java}) = 1$

Third, Java provides static dispatch mechanisms through static methods and compile-time binding of non-virtual calls (e.g., private, final methods), which serve as explicit or implicit devirtualization hints.

$K_{static}(\text{Java}) = 1$

$O_{virt}(\text{Java}) = 1$

Final metric value:

$$\begin{aligned} PDEI(\text{Java}) &= 0.35 \cdot M_{scope}(\text{Java}) + 0.35 \cdot S_{disp}(\text{Java}) + 0.3 \cdot O_{virt}(\text{Java}) \\ &= 0.35 \cdot 0.2 + 0.35 \cdot 0.5 + 0.3 \cdot 1 \\ &= 0.545 \end{aligned}$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

In Java, all objects are mutable by default unless explicitly designed otherwise by the programmer, for example by declaring fields as final or creating immutable classes (e.g., String). The JLS specifies that instance fields are mutable unless restricted, and there is no language-level requirement or annotation enforcing immutability by default. Moreover, immutability in Java is a design pattern rather

than a built-in default behavior, requiring explicit developer effort. Therefore, Java does not satisfy the condition of immutability by default nor the condition where mutability requires explicit annotation. The mutability is inherent and unrestricted unless constrained.

$$M_{def}(\text{Java}) = 0$$

2) Static Reference Constraint Density $S_{dens}(L)$

Java employs references for all non-primitive types, and the primary constructs contributing to $N_{ref}(L)$ include: object references (class types), array references, interface-typed references, generic type references, and the null reference.

$N_{ref}(\text{Java}) = 5$

Next, we identify constructs that impose compile-time constraints on reference usage: the final keyword, which restricts reassignment of references, generic type bounds (e.g., $<\mathrm{T}\ \mathrm{extends}\ ...>$), which constrain admissible reference types, and access modifiers (e.g., $\mathrm{private}$, $\mathrm{protected}$) that statically restrict reference visibility and usage contexts.

$N_{constr}(\text{Java}) = 3$

$$S_{dens}(\text{Java}) = 0.6$$

3) Concurrency Primitive Safety $P_{conc}(L)$

First, Java does not provide compile-time guarantees of data race freedom. It relies on programmer discipline using constructs such as $\mathrm{synchronized}$, $\mathrm{volatile}$, and higher-level utilities from $\mathrm{java.util.concurrent}$. Although the Java Memory

Model formally defines happens-before relationships, it does not prevent data races statically nor enforce them via runtime checks [46].

$S_{race}(\text{Java}) = 0$

Second, Java primarily exposes low-level concurrency primitives such as threads (Thread) and locks (synchronized, Lock), while also offering structured abstractions like Future, ExecutorService, and task-based concurrency. However, these do not constitute fully high-level models like Actors or CSP channels.

$S_{model}(\text{Java}) = 0.5$

$P_{conc}(\text{Java}) = 0.25$

Final metric value:

$$SCSI(\text{Java}) = 0.35 \cdot M_{def}(\text{Java}) + 0.25 \cdot S_{dens}(\text{Java}) + 0.4 \cdot P_{conc}(\text{Java}) = 0.35 \cdot 0 + 0$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

Java does not include native language-level constructs for specifying preconditions (e.g., requires), postconditions (e.g., ensures), or class invariants as part of its syntax or type system. While developers can express such constraints using assertions (assert) or external frameworks (e.g., JML), these are not integrated as formal contract constructs with guaranteed enforcement semantics in the core language specification.

$C_{nat}(\text{Java}) = 0$

2) Verification Enforcement Level $V_{enf}(L)$

Java does not support static verification of program correctness in the sense of compile-time proofs for contracts or general behavioral properties. The type system ensures type safety but does not verify semantic correctness such as preconditions or invariants. However, Java provides runtime checking mechanisms, most notably through the assert statement, which enables optional runtime validation of boolean conditions. Additionally, exceptions (e.g., $\mathrm{IllegalArgumentException}$) are commonly used to enforce conditions dynamically during execution. These mechanisms are explicitly defined in the language and standard runtime but operate only at runtime rather than compile time.

$$V_{enf}(\text{Java}) = 0.5$$

Final metric value:

$$\begin{aligned} FSVI(\text{Java}) &= 0.45 \cdot C_{nat}(\text{Java}) + 0.55 \cdot V_{enf}(\text{Java}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0.5 \\ &= 0.275 \end{aligned}$$

4.1.3 Metrics Evaluation for Smalltalk

Smalltalk represents one of the earliest and most influential realizations of a pure object-oriented programming paradigm, where “everything is an object” and computation is performed exclusively via message passing [47]. Its design deliberately prioritizes uniformity and reflection over static guarantees, which significantly impacts the quantitative evaluation of object model properties. As noted by Goldberg and Robson, “Smalltalk is a dynamically typed language in

which all operations are expressed as messages sent to objects" [47, p. 9]. The following subsection evaluates Smalltalk against the proposed metrics, grounding each numerical assignment in the formal and de facto language specification as documented in canonical literature and subsequent analyses.

Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

Smalltalk is a purely dynamically typed language[48] whose semantics are defined by message passing rather than static typing judgments. It does not provide formal structural subtyping rules.

$I_{struct}(\text{Smalltalk}) = 0$

Although Smalltalk employs classes and inheritance, subtype relationships are not enforced through a static type system with explicit nominal typing judgments, but instead emerge dynamically at runtime. It is lack of formal nominal typing as defined by static type theory.

$I_{nom}(\text{Smalltalk}) = 0$

$$S_{nom}(\text{Smalltalk}) = \frac{0+0}{2} = 0$$

2) Variance Formalization Score $S_{var}(L)$

Smalltalk does not include generics or parametric type constructors in its core language definition. All polymorphism is achieved dynamically via message passing without static type parameters, hence no variance annotations (covariance, contravariance, or invariance) can be expressed.

$V_{exp}(\text{Smalltalk}) = 0$

Additionally, since variance is not part of the language semantics, there are

no formally defined typing rules or proofs of type soundness related to variance in the specification.

$V_{sound}(\text{Smalltalk}) = 0$

$$V_{exp}(\text{Smalltalk}) = 0, V_{sound}(\text{Smalltalk}) = 0 \Rightarrow S_{var}(\text{Smalltalk}) = 0$$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, Smalltalk does not provide any explicit notion of subtyping separate from class inheritance. The type compatibility is entirely governed by the class hierarchy, and there are no interfaces, traits, or structural typing mechanisms that would allow subtyping without inheritance.

$I_{dec}(\text{Smalltalk}) = 0$

Second, the subtype relation is not defined independently but is implicitly identical to the inheritance relation: an object is considered substitutable only if it belongs to a subclass, reflecting purely nominal and inheritance-based typing.

$I_{ind}(\text{Smalltalk}) = 0$

Third, the specification does not formally distinguish between inheritance and subtyping as separate relations. The behavioral compatibility is informally tied to method availability within the class hierarchy, without a formal type system that separates these concepts.

$I_{form}(\text{Smalltalk}) = 0$

$$S_{rel}(\text{Smalltalk}) = \frac{0+0+0}{3} = 0$$

Final metric value:

$$TSSI(\text{Smalltalk}) = 0.3 \cdot S_{nom}(\text{Smalltalk}) + 0.3 \cdot S_{var}(\text{Smalltalk}) + 0.4 \cdot S_{rel}(\text{Smalltalk}$$
$$= 0.3 \cdot 0 + 0.3 \cdot 0 + 0.4 \cdot 0 = 0$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

Smalltalk does not provide formal access modifiers such as *private*, *protected*, or module-level visibility. All methods are publicly accessible via message sending, and encapsulation is achieved by convention (e.g., method categories) rather than enforced by the language semantics.

$$n_{levels}(\text{Smalltalk}) = 1$$

$$n_{levels}(\text{C\#}) = 7 \Rightarrow V(\text{Smalltalk}) = \tfrac{1}{7}$$

2) Module Isolation Strength $M(L)$

Smalltalk systems organize code primarily through classes and method categories, and in some environments through packages. However, these constructs do not enforce strict export control at the language specification level, as all classes and methods remain globally accessible via message passing. Thus, while modularization constructs exist, none impose enforced visibility restrictions or explicit export lists.

$$M(\text{Smalltalk}) = 0$$

3) Interface Abstraction Capacity $I(L)$

Smalltalk defines objects exclusively through classes.

$n_{object}(\text{Smalltalk}) = 1$

The language does not include distinct constructs such as interfaces, abstract classes with enforced method signatures, or formal protocols in its core specification. Behavioral abstraction is achieved implicitly through duck typing and message protocols, which are not separate syntactic or semantic entities.

$n_{abstraction_constructs}(\text{Smalltalk}) = 0$

$$I(\text{Smalltalk}) = \frac{0}{1} = 0$$

4) Encapsulation Enforcement Coefficient $E(L)$

Smalltalk provides fully reflective capabilities, including the ability to inspect and modify objects, classes, and method dictionaries at runtime. Additionally, instance variables can be accessed indirectly through reflective operations and message sending without enforced access restrictions. Since the language lacks a static type system and does not restrict reflective access or enforce encapsulation boundaries at runtime, it permits uncontrolled access to object internals.

$$E(\text{Smalltalk}) = 0$$

Final metric value:

$$AE(\text{Smalltalk}) = 0.2 \cdot V(\text{Smalltalk}) + 0.25 \cdot M(\text{Smalltalk}) + 0.25 \cdot I(\text{Smalltalk}) + 0.$$
$$= 0.2 \cdot \frac{1}{7} + 0.25 \cdot 0 +$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

Smalltalk enforces a *single inheritance* paradigm, meaning that each class has exactly one direct superclass (except for the root class, typically Object).

$$H_s(\text{Smalltalk}) = 1$$

2) Linearization Determinism Score $L_d(L)$

Smalltalk employs a strictly single inheritance model, where each class has exactly one superclass, forming a linear inheritance chain. Method resolution is therefore performed via a deterministic upward traversal of this chain [47], starting from the receiver's class and proceeding through its superclasses until a matching method is found.

$$L_d(\text{Smalltalk}) = 1$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 0$: Smalltalk does not support final classes or methods—all classes and methods can be subclassed or overridden.

$p_2 = 0$: There is no notion of sealed or closed hierarchies, and any class can be extended freely.

$p_3 = 0$ as method overriding does not require explicit annotation (method replacement is implicit via dynamic dispatch).

$p_4 = 0$: All methods are effectively virtual by default, with no non-virtual dispatch semantics.

$p_5 = 0$: The language lacks a formal distinction between abstract classes and

interfaces in its original specification, relying instead on conventions.

$p_6 = 0$: There are no compile-time restrictions on inheritance due to the absence of static type checking and the system's live, reflective nature.

$$C_p(\text{Smalltalk}) = \frac{0+0+0+0+0+0}{6} = 0$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 0$: Overriding does not require explicit annotation and occurs implicitly through method redefinition.

$s_2 = 0$: Method visibility is not strictly enforced (all methods are effectively accessible within the system, with only conventional categories such as "private" lacking formal enforcement).

$s_3 = 0$: There is no requirement to explicitly invoke superclass implementations—sending a super message is optional and not enforced.

$s_4 = 0$: The language lacks a static type system and thus does not define variance constraints for safe overriding.

$s_5 = 0$: All methods are dynamically dispatched and overridable by default, with no mechanism for non-overridable declarations.

$s_6 = 0$: There are no compile-time checks ensuring override consistency due to the absence of static compilation constraints.

$$F_b(\text{Smalltalk}) = \frac{0+0+0+0+0+0}{6} = 0$$

Final metric value:

$$ISRI(\text{Smalltalk}) = 0.2 \cdot H_s(\text{Smalltalk}) + 0.25 \cdot L_d(\text{Smalltalk}) + 0.3 \cdot C_p(\text{Smalltalk}) +$$
$$= 0.2 \cdot 1 + 0.25 \cdot$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

Smalltalk does not provide dedicated language-level keywords for delegation (such as delegate), but delegation can be systematically emulated using composition and message forwarding patterns (e.g., via $\text{doesNotUnderstand:}$). The language includes a reflective message interception mechanism through $\text{doesNotUnderstand:}$, which enables developers to implement dynamic forwarding of messages to another object, effectively supporting automatic delegation without explicit boilerplate per method.

$$K_{del}(\text{Smalltalk}) = 0.5, S_{fwd}(\text{Smalltalk}) = 1 \Rightarrow D_{cls}(\text{Smalltalk}) = 0.75$$

2) Trait Expressiveness Score $T_{exp}(L)$

Smalltalk traits provide explicit mechanisms for resolving method name conflicts through both exclusion and aliasing operators, which are formally defined in the trait composition semantics. The traits can declare required methods that must be implemented by the consuming class, enabling explicit specification of behavioral dependencies.

$$R_{conf}(\text{Smalltalk}) = 1, O_{mod}(\text{Smalltalk}) = 1 \Rightarrow T_{exp}(\text{Smalltalk}) = 0$$

3) Interface Composition Capacity $I_{comp}(L)$

Smalltalk does not define interfaces as a separate language feature. It relies on implicit typing (duck typing) and behavioral conformance, where any object responding to a set of messages satisfies the expected protocol. Consequently, there are no formal interface declarations or constructs analogous to interfaces, yielding Since interfaces do not exist as entities, there is no notion of default method implementations within them, thus

$$N_{impl}(\text{Smalltalk}) = 0, D_{def}(\text{Smalltalk}) = 0 \Rightarrow I_{comp}(\text{Smalltalk}) = 0$$

Final metric value:

$$CRFI(\text{Smalltalk}) = 0.3 \cdot D_{cls}(\text{Smalltalk}) + 0.35 \cdot T_{exp}(\text{Smalltalk}) + 0.35 \cdot I_{comp}(\text{Smal}$$
$$= 0.3 \cdot 0.75 + 0.35 \cdot 0 + 0.$$
$$= 0$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

In Smalltalk, all user-definable types are classes, and every instance is an object accessed via references, with no distinction between value types and reference types at the language level.

$$S_{id}(\text{Smalltalk}) = 1$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

Smalltalk primarily supports class-based instantiation via message sending

(e.g., $\mathrm{MyClass\ new}$), which internally invokes standardized allocation and initialization protocols (such as $\mathrm{basicNew}$ and $\mathrm{initialize}$). While additional patterns like cloning or literal construction may exist in specific implementations, they are not part of the core, formally defined instantiation paradigms of the language.

$$P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{Smalltalk}) = \tfrac{1}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

Smalltalk provides a fully reflective environment in which classes are themselves objects (instances of metaclasses), and both structure and behavior can be modified at runtime. Moreover, the object instantiation process can be customized by overriding class-side methods such as new, $\mathrm{basicNew}$, and initialization protocols, effectively enabling control over allocation and initialization semantics through metaobjects.

$$I_{level}(\text{Smalltalk}) = 3, I_{level}(\text{JS/Python/Ruby}) = 3$$
$$\Rightarrow S_{meta}(\text{Smalltalk}) = \frac{1}{3}$$

Final metric value:

$$ORCI(\text{Smalltalk}) = 0.35 \cdot S_{id}(\text{Smalltalk}) + 0.3 \cdot S_{cr}(\text{Smalltalk}) + 0.35 \cdot S_{meta}(\text{Smallta}$$
$$= 0.35 \cdot 1 + 0.3 \cdot \frac{1}{4} + 0.35$$
$$\approx 0.5$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

Smalltalk natively supports single dispatch, where method selection depends solely on the dynamic type of the message receiver.

$I_{single}(\text{Smalltalk}) = 1$

However, it does not provide native multiple dispatch, as method resolution does not consider the dynamic types of arguments beyond the receiver.

$I_{multi}(\text{Smalltalk}) = 0$

Additionally, predicate-based dispatch is not part of the language specification. The conditional behavior must be encoded explicitly within method bodies rather than through the dispatch mechanism itself.

$I_{pred}(\text{Smalltalk}) = 0$

$M_{scope}(\text{Smalltalk}) = \frac{1}{5} = 0.2$

2) Dispatch Safety Guarantee $S_{disp}(L)$

First, Smalltalk employs purely dynamic message passing with no static type checking. The correctness of a message send (i.e., whether the receiver understands the selector) is verified only at runtime, potentially resulting in a $\text{doesNotUnderstand:}$ exception.

$I_{static}(\text{Smalltalk}) = 0$

Second, the language does not require exhaustive definitions of dispatch cases—method lookup proceeds dynamically through the class hierarchy, and missing methods are handled at runtime rather than enforced statically.

$I_{exhaust}(\text{Smalltalk}) = 0$

$S_{disp}(\text{Smalltalk}) = \frac{0+0}{2} = 0$

3) Virtualization Optimization Potential $O_{virt}(L)$

Smalltalk does not support final or non-virtual methods, as all methods are dynamically dispatched and can be overridden.

$K_{final}(\text{Smalltalk}) = 0$

Similarly, it does not provide sealed class hierarchies or mechanisms to restrict subclassing.

$K_{sealed}(\text{Smalltalk}) = 0$

Furthermore, there are no static dispatch hints or constructs such as static methods or annotations for devirtualization. All message sends are uniformly dynamic.

$K_{static}(\text{Smalltalk}) = 0$

$O_{virt}(\text{Smalltalk}) = 0$

Final metric value:

$$PDEI(\text{Smalltalk}) = 0.35 \cdot M_{scope}(\text{Smalltalk}) + 0.35 \cdot S_{disp}(\text{Smalltalk}) + 0.3 \cdot O_{virt}(\text{Sm}$$
$$= 0.35 \cdot 0.2 + 0.35 \cdot 0 +$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

In Smalltalk, all entities are objects and their state is inherently mutable: instance variables can be freely modified through message passing without requiring any special annotations or qualifiers. There is no built-in notion of immutability enforced by default, nor a requirement to explicitly mark objects as mutable,

mutability is the standard behavior.

$$M_{def}(\text{Smalltalk}) = 0$$

2) Static Reference Constraint Density $S_{dens}(L)$

In Smalltalk, all variables (including instance, temporary, and class variables) uniformly denote references to objects, and message passing is the sole mechanism for interaction. The distinct reference-related constructs include: variable bindings, assignment operation, and message sends that operate on object references.

$N_{ref}(\text{Smalltalk}) = 3$

However, Smalltalk is a dynamically typed language with no static type system and no compile-time enforcement of aliasing, mutability, or reference usage constraints, all such checks occur at runtime. Consequently, none of the reference-related constructs impose static constraints.

$N_{constr}(\text{Smalltalk}) = 0$

$$S_{dens}(\text{Smalltalk}) = 0$$

3) Concurrency Primitive Safety $P_{conc}(L)$

First, Smalltalk provides lightweight processes (green threads) and synchronization primitives (e.g., semaphores), but lacks a static type system or runtime mechanisms that enforce data race freedom, correctness depends entirely on programmer discipline when coordinating shared mutable state.

$S_{race}(\text{Smalltalk}) = 0$

Second, Smalltalk offers low-level concurrency constructs such as processes

and semaphores rather than structured concurrency or high-level models like Actors or CSP channels in its standard specification.

$$S_{model}(\text{Smalltalk}) = 0$$

$$P_{conc}(\text{Smalltalk}) = 0$$

Final metric value:

$$SCSI(\text{Smalltalk}) = 0.35 \cdot M_{def}(\text{Smalltalk}) + 0.25 \cdot S_{dens}(\text{Smalltalk}) + 0.4 \cdot P_{conc}(\text{Sma}$$
$$= 0.35 \cdot 0 + 0.25 \cdot 0 + 0.4 \cdot$$

8) Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

Smalltalk does not natively support preconditions, postconditions, or invariants as part of its syntax or semantics, such checks must be implemented manually using message sends (e.g., assertions) or via external frameworks, and are not enforced by the language itself.

$$C_{nat}(\text{Smalltalk}) = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

Smalltalk is a dynamically typed language without any support for static verification or compile-time proofs of correctness. While developers may implement runtime checks (e.g., via assertions or conditional message sends), these are not part of a formal, built-in verification framework and are neither enforced nor standardized by the language semantics.

$$V_{enf}(\text{Smalltalk}) = 0$$

Final metric value:

$$\begin{aligned} FSVI(\text{Smalltalk}) &= 0.45 \cdot C_{nat}(\text{Smalltalk}) + 0.55 \cdot V_{enf}(\text{Smalltalk}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0 = 0 \end{aligned}$$

4.1.4 Metrics Evaluation for C++

Introduction

C++ represents a multi-paradigm language with a predominantly nominal and statically checked type system, enriched by templates, multiple inheritance, and low-level memory control. Its object model has evolved significantly from early descriptions in [49] to the formalized ISO standard [50]. Prior studies emphasize that C++ combines abstraction mechanisms with explicit control over performance and representation, often at the cost of formal safety guarantees [51]. This section evaluates C++ using the defined metrics, grounding each numerical assignment in the language specification and established literature.

Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

C++ does not define structural subtyping[49] ((ISO/IEC 14882)) in core type system: type compatibility is not determined by the structure but rather by explicit declarations, and while templates and concepts enable compile-time constraints resembling structural checks, they do not constitute formal structural subtyping judgments in the specification. C++ is fundamentally nominally typed: subtype relationships are explicitly declared via class inheritance (e.g., $\mathrm{class\ B\ :\ public\ A}$), and the standard formally specifies these relationships through inheritance and access control rules.

$$I_{nom}(\text{C++}) = 1, I_{struct}(\text{C++}) = 0 \Rightarrow S_{nom}(\text{C++}) = 0.5$$

2) Variance Formalization Score $S_{var}(L)$

C++ supports parametric polymorphism via templates, but variance (covariance or contravariance) is neither explicitly declared nor syntactically annotated. Any variance behavior arises implicitly through pointer/reference conversions and template instantiation rules, without first-class variance constructs. The language enforces type safety through well-defined template instantiation and conversion rules, but does not provide a formally specified or proven variance system. Moreover, certain constructs demonstrate that variance is only partially restricted rather than formally sound [29].

$$V_{exp}(\text{C++}) = 0.5, V_{sound}(\text{C++}) = 0.5 \Rightarrow S_{var}(\text{C++}) = 0.5$$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, the language does not provide constructs for declaring subtyping independently of inheritance (e.g., no native interface or structural typing mechanism separate from class inheritance).

$I_{dec}(\text{C++}) = 0$

Second, subtyping rules—such as implicit conversion from derived to base class pointers or references—are directly derived from inheritance semantics, meaning there is no independent subtyping relation.

$I_{ind}(\text{C++}) = 0$

Third, the specification does not formally distinguish inheritance and subtyping as separate relations. The substitutability is an emergent property of inheritance rules and type conversion mechanisms, without an explicit formal separation.

$I_{form}(\text{C++}) = 0$

$$S_{rel}(\text{C++}) = \frac{0+0+0}{3} = 0$$

Final metric value:

$$\begin{aligned} TSSI(\text{C++}) &= 0.3 \cdot S_{nom}(\text{C++}) + 0.3 \cdot S_{var}(\text{C++}) + 0.4 \cdot S_{rel}(\text{C++}) \\ &= 0.3 \cdot 0.5 + 0.3 \cdot 0.5 + 0.4 \cdot 0 \\ &= 0.3 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

The language provides three semantically distinct access levels: private, protected, and public, each formally specified to constrain member accessibility in class hierarchies and across translation units.

$$n_{levels}(\text{C\#}) = 7 \Rightarrow V(\text{C++}) = \frac{3}{7}$$

2) Module Isolation Strength $M(L)$

The language provides two primary boundary mechanisms: namespaces and modules. Namespaces serve as organizational scopes but do not enforce strict access control, as all members are visible once declared. They are not counted as strict boundaries. In contrast, C++20 modules introduce explicit export control via export declarations, forming well-defined interface boundaries that restrict visibility across translation units.

$$\begin{aligned} b_{strict}(\text{C++}) = 1, b_{total}(\text{C++}) = 2 \\ \Rightarrow M(\text{C++}) = 0.5 \end{aligned}$$

3) Interface Abstraction Capacity $I(L)$

The language does not provide a dedicated *interface* keyword. Abstraction is achieved primarily through abstract classes and, more recently, through concepts, which define compile-time behavioral constraints. Regarding object-defining constructs, C++ includes class, struct, and union, all formally specified as distinct type constructors.

$$n_{abstraction_constructs}(\text{C++}) = 2, n_{object}(\text{C++}) = 3$$
$$\Rightarrow I(\text{C++}) = \frac{2}{3} = 0.67$$

4) Encapsulation Enforcement Coefficient $E(L)$

C++ enforces multiple mechanisms that allow bypassing encapsulation constraints: pointer arithmetic, type casting (e.g., reinterpret_cast), direct memory manipulation, and undefined behavior that can expose or modify object internals without restriction. Furthermore, the absence of runtime enforcement and the presence of low-level operations mean that encapsulation is not strictly preserved under all conditions.

$$E(\text{C++}) = 0$$

Final metric value:

$$AE(\text{C++}) = 0.2 \cdot V(\text{C++}) + 0.25 \cdot M(\text{C++}) + 0.25 \cdot I(\text{C++}) + 0.3 \cdot E(\text{C++})$$
$$= 0.2 \cdot \frac{3}{7} + 0.25 \cdot 0.5 + 0.25 \cdot \frac{2}{3} + 0.3 \cdot 0$$
$$\approx 0.3774$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

C++ class may inherit from multiple base classes using a comma-separated base-specifier list, which implies that the maximum number of direct superclasses $M(L)$ is unbounded in practice (i.e., $M(L) > 1$ and limited only by implementation constraints). Since there is no fixed upper bound in the specification, C++ effectively reaches the maximal considered value in comparative analysis.

$$M(\text{C++}) = M_{max} \Rightarrow H_s(\text{C++}) = 1 - \frac{M(\text{C++})-1}{M_{max}-1} = 0$$

2) Linearization Determinism Score $L_d(L)$

The C++ standard does not define a global, formally specified method resolution order (MRO) comparable to linearization algorithms, instead, name lookup and overload resolution are governed by a combination of rules including scope-based lookup, unqualified and qualified name lookup, and class member access control. In multiple inheritance scenarios, ambiguities may arise (e.g., the diamond problem), and the programmer must explicitly disambiguate member access using scope resolution or employ virtual inheritance to alter object layout and lookup behavior. Since the standard specifies lookup procedures but does not define a single deterministic linearization sequence for all cases, the effective resolution strategy is only partially specified.

$$L_d(\text{C++}) = 0.5$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 1$: The language provides final for classes and virtual functions, pre-

venting further inheritance or overriding.

$p_2 = 0$: It does not support sealed or closed hierarchies with an explicitly fixed set of subclasses.

$p_3 = 0$: The $\mathrm{override}$ specifier exists but is not mandatory, hence no enforced override requirement.

$p_4 = 1$: Member functions are non-virtual by default unless explicitly declared $\mathrm{virtual}$.

$p_5 = 0$: Although abstract classes exist via pure virtual functions, there is no strict language-level separation of interfaces as a distinct construct.

$p_6 = 0$: Compile-time inheritance restrictions are partially present (e.g., final), but since this is already accounted for in p_1 and no additional independent restriction system exists.

$$C_p(\text{C++}) = \frac{1+0+0+1+0+0}{6} \approx 0.33$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 0$: The language does not require mandatory override annotations, as $\mathrm{override}$ is optional.

$s_2 = 1$: It provides strict method visibility control via access specifiers ($\mathrm{private}$, $\mathrm{protected}$, public), which constrain exposure of overridable members.

$s_3 = 0$: There is no requirement for explicit superclass method invocation when overriding (calls to base class implementations via qualified names are optional).

$s_4 = 1$: C++ enforces variance restrictions in overriding, allowing only limited covariance (e.g., return types for pointers/references) and disallowing unsafe contravariance, which ensures type safety at compile time.

$s_5 = 1$: Member functions are non-virtual by default and require explicit virtual to enable overriding.

$s_6 = 1$: Finally, the compiler performs strict compile-time checks on overriding consistency (e.g., signature matching, virtual dispatch rules, and diagnostics with override).

$$F_b(\text{C++}) = \tfrac{0+1+0+1+1+1}{6} \approx 0.67$$

Final metric value:

$$\begin{aligned} ISRI(\text{C++}) &= 0.2 \cdot H_s(\text{C++}) + 0.25 \cdot L_d(\text{C++}) + 0.3 \cdot C_p(\text{C++}) + 0.25 \cdot F_b(\text{C++}) \\ &= 0.2 \cdot 0 + 0.25 \cdot 0.5 + 0.3 \cdot 0.33 + 0.25 \cdot \frac{4}{6} \\ &\approx 0.3907 \end{aligned}$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

There is no dedicated keyword or built-in construct (such as delegate) for delegation in C++. Instead, delegation must be implemented manually using composition and forwarding functions (e.g., wrapper or adapter patterns). Therefore, delegation is only emulated. Furthermore, C++ does not provide automatic method forwarding mechanisms, meaning developers must explicitly define forwarding functions.

$$K_{del}(\text{C++}) = 0.5, S_{fwd}(\text{C++}) = 0 \Rightarrow D_{cls}(\text{C++}) = 0.25$$

2) Trait Expressiveness Score $T_{exp}(L)$

In cases of multiple inheritance, name conflicts between base classes result in ambiguity errors unless explicitly resolved by the programmer using qualified names. However, the language does not provide trait-style exclusion or aliasing mechanisms. C++ templates and mixin patterns do not support explicit declaration of required methods or properties within the mixin itself.

$$T_{exp}(\text{C++}) = 0$$

3) Interface Composition Capacity $I_{comp}(L)$

C++ allows a class to inherit from an arbitrary number of base classes, including abstract classes (i.e., classes with pure virtual functions), with no language-imposed upper bound. Unlike traditional interface-only constructs, abstract classes in C++ may contain fully defined member functions alongside pure virtual functions, effectively supporting default method implementations within interface-like structures.

$$I_{comp}(\text{C++}) = 1$$

Final metric value:

$$\begin{aligned} CRFI(\text{C++}) &= 0.3 \cdot D_{cls}(\text{C++}) + 0.35 \cdot T_{exp}(\text{C++}) + 0.35 \cdot I_{comp}(\text{C++}) \\ &= 0.3 \cdot 0.25 + 0.35 \cdot 0 + 0.35 \cdot 1 \\ &= 0.425 \end{aligned}$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

The primary user-definable type categories include classes, structs, unions and enums. In C++, all these types follow value semantics by default: objects are copied on assignment and passed by value unless explicitly handled via pointers or references, which are not independent user-defined type categories but rather type modifiers.

$$S_{id}(\text{C++}) = \frac{0}{4} = 0$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

First, C++ supports direct construction via constructors, including stack allocation and dynamic allocation using the new operator, which together represent a single class-based instantiation paradigm. Second, the language supports copy and move construction (including cloning via copy constructors), which forms a prototype-like paradigm grounded in object copying semantics. Third, C++ provides aggregate and uniform initialization (including brace-initialization), which constitutes a literal-style construction mechanism for certain types.

$$P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{C++}) = \frac{3}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

The language provides limited runtime introspection via mechanisms such as typeid and $\mathrm{dynamic\_cast}$, which allow querying certain aspects of an object's dynamic type within polymorphic hierarchies. However, the language does not support structural modification of objects at runtime, nor does it provide a metaobject protocol for overriding object instantiation or altering class definitions dynamically. All type structures are fixed at compile time, and templates operate

purely at compile time without enabling runtime intercession.

$$I_{level}(\text{C++}) = 1, I_{level}(\text{JS/Python/Ruby}) = 3 \Rightarrow S_{meta}(\text{C++}) = \tfrac{1}{3}$$

Final metric value:

$$\begin{aligned} ORCI(\text{C++}) &= 0.35 \cdot S_{id}(\text{C++}) + 0.3 \cdot S_{cr}(\text{C++}) + 0.35 \cdot S_{meta}(\text{C++}) \\ &= 0.35 \cdot 0 + 0.3 \cdot \frac{3}{4} + 0.35 \cdot \frac{1}{3} \\ &\approx 0.3417 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

First, C++ supports single dispatch via virtual functions, where method selection is determined by the dynamic type of the receiver object.

$I_{single}(\text{C++}) = 1$

Second, C++ does not natively support multiple dispatch (i.e., dispatch based on the runtime types of multiple arguments). Such behavior must be emulated using patterns like double dispatch or visitor.

$I_{multi}(\text{C++}) = 0$

Third, predicate-based dispatch (selection based on arbitrary runtime conditions or guards integrated into the dispatch mechanism) is not part of the language semantics and must be implemented manually using conditional logic.

$I_{pred}(\text{C++}) = 0$

$M_{scope}(\text{C++}) = \frac{1}{5} = 0.2$

2) Dispatch Safety Guarantee $S_{disp}(L)$

The compiler checks function signatures and virtual override consistency, dynamic dispatch via virtual functions depends on runtime object types and does not ensure complete safety.

$I_{static}(\text{C++}) = 0$

Furthermore, C++ does not enforce exhaustive dispatch definitions: constructs such as virtual method hierarchies or switch statements over enums do not require covering all possible dynamic types or cases at compile time, and missing cases are not universally diagnosed as errors.

$I_{exhaust}(\text{C++}) = 0$

$$S_{disp}(\text{C++}) = \frac{0+0}{2} = 0$$

3) Virtualization Optimization Potential $O_{virt}(L)$

First, C++ supports non-virtual methods by default and provides the final specifier for both classes and virtual functions, preventing further overriding and enabling compiler optimizations.

$K_{final}(\text{C++}) = 1$

Second, the final specifier applied to classes effectively creates sealed hierarchies by prohibiting inheritance.

$K_{sealed}(\text{C++}) = 1$

Third, C++ supports static dispatch through non-virtual member functions, free functions, and templates, all of which are resolved at compile time and provide strong hints for devirtualization.

$K_{static}(\text{C++}) = 1$

$$O_{virt}(\text{C++}) = \frac{1+1+1}{3} = 1$$

Final metric value:

$$\begin{aligned} PDEI(\text{C++}) &= 0.35 \cdot M_{scope}(\text{C++}) + 0.35 \cdot S_{disp}(\text{C++}) + 0.3 \cdot O_{virt}(\text{C++}) \\ &= 0.35 \cdot 0.2 + 0.35 \cdot 0 + 0.3 \cdot 1 \\ &= 0.37 \end{aligned}$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

By default, any variable declared without the const qualifier permits modification of its stored value after initialization, which directly implies that mutability does not require any additional annotation, whereas immutability does. The const qualifier must be explicitly applied to enforce immutability, and this requirement is purely opt-in rather than default behavior.

$$M_{def}(\text{C++}) = 0$$

2) Static Reference Constraint Density $S_{dens}(L)$

In C++, the set of reference-related constructs includes syntactic forms that introduce or manipulate references and pointers as defined by the ISO C++ standard: namely raw pointers (T), lvalue references (T&), rvalue references (T&&), pointer qualifiers such as const and volatile, smart pointers (e.g., std unique_ptr, std shared_ptr), and reference-related type modifiers in templates and type deduction.

$N_{ref}(\text{C++}) = 6$

Among these, constructs that impose static constraints include:

- const qualification, which enforces compile-time immutability constraints on referenced objects.
- lvalue references, which must bind to valid lvalues and cannot be reseated.
- rvalue references, which restrict binding to temporaries and enable move semantics
- std::unique_ptr, which enforces exclusive ownership semantics at compile time via deleted copy constructors.

$N_{constr}(\text{C++}) = 4$

$S_{dens}(\text{C++}) = \frac{4}{6} \approx 0.67$

3) Concurrency Primitive Safety $P_{conc}(L)$

In C++, concurrency semantics are defined by the ISO C++ standard (e.g., §1.10 and §30 in C++17 and later), where the memory model formally specifies data races as undefined behavior but does not provide compile-time guarantees of race freedom. The language relies on programmer discipline, supported by low-level primitives such as std::thread, std::mutex, and std::atomic, without a type system capable of proving absence of data races.

$S_{race}(\text{C++}) = 0$

Regarding abstraction level, C++ primarily offers low-level threading and synchronization constructs, although it also includes std::future and std::async, which provide limited structured concurrency. However, these do not constitute a

fully structured or high-level concurrency model such as actors or CSP channels, and the dominant paradigm remains explicit threads and locks.

$$S_{model}(\text{C++}) = 0$$

$$P_{conc}(\text{C++}) = 0$$

Final metric value:

$$\begin{aligned} SCSI(\text{C++}) &= 0.35 \cdot M_{def}(\text{C++}) + 0.25 \cdot S_{dens}(\text{C++}) + 0.4 \cdot P_{conc}(\text{C++}) \\ &= 0.35 \cdot 0 + 0.25 \cdot \frac{4}{6} + 0.4 \cdot 0 \\ &\approx 0.1667 \end{aligned}$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

The ISO C++ standard does not include fully adopted and semantically enforced language-level support for contracts. Consequently, there are no first-class syntactic constructs for specifying preconditions, postconditions, or invariants within the core language. The developers rely on mechanisms such as assert, comments, or external libraries, which do not satisfy the requirement of being both syntactically native and semantically defined by the language specification.

$$C_{nat}(\text{C++}) = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

In C++, verification enforcement must be assessed based on mechanisms defined by the ISO C++ standard, which does not provide built-in facilities for

formal static verification. While the language supports runtime checking mechanisms such as assert and exception handling, these operate exclusively at runtime and are not part of a formal verification system guaranteed by the compiler.

$$V_{enf}(\text{C++}) = 0.5$$

Final metric value:

$$\begin{aligned} FSVI(\text{C++}) &= 0.45 \cdot C_{nat}(\text{C++}) + 0.55 \cdot V_{enf}(\text{C++}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0.5 \\ &= 0.275 \end{aligned}$$

4.1.5 Metrics Evaluation for C

Introduction

The C programming language, standardized by ISO/IEC 9899, represents a minimalistic procedural paradigm with a weak notion of abstraction and no native support for object-oriented constructs. As noted by Kernighan and Ritchie, "C provides no direct support for object-oriented programming" [52]. Consequently, evaluating C under object-model-oriented metrics requires interpreting its type system, memory model, and modularization mechanisms through the lens of emulated object-oriented patterns. This subsection computes the defined metrics strictly based on the formal language specification and authoritative analyses [29], [53].

1) Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

Structural subtyping is absent in C, since type compatibility is not based on structural equivalence with subtyping rules, but rather on strict compatibility rules without a formal subtype relation (ISO/IEC 9899).

$I_{struct}(\mathrm{C}) = 0.$

Nominal typing with explicit subtype declarations is also absent: although C provides named types via struct, union, and $\mathrm{typedef}$, it does not support inheritance or explicit subtype relationships between named types, as required by nominal subtyping systems.

$I_{nom}(\mathrm{C}) = 0.$

$$S_{nom}(\mathrm{C}) = 0$$

2) Variance Formalization Score $S_{var}(L)$

C does not support generics in the sense required for variance (i.e., no parameterized types with subtype relations), as its type system lacks constructs such as type parameters. Therefore, variance annotations are inapplicable.

$V_{exp}(\mathrm{C}) = 0.$

Since variance is not defined at all in the language specification, there are no formal rules governing covariance, contravariance, or invariance, nor any associated soundness proofs.

$V_{sound}(\mathrm{C}) = 0.$

$$S_{var}(\mathrm{C}) = 0$$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

C is a procedural language with a type system based on compatibility rather than subtype polymorphism. The constructs such as `struct` composition and pointer conversions do not constitute explicit subtyping relations defined by the language semantics.

$I_{dec}(\mathrm{C}) = 0$ because the language does not provide explicit subtyping independent of inheritance (nor inheritance itself).

$I_{ind}(\mathrm{C}) = 0$ since no independent subtyping rules are specified.

$I_{form}(\mathrm{C}) = 0$ because there is no formal distinction between inheritance and subtyping in the specification.

$$S_{rel}(\mathrm{C}) = \tfrac{0+0+0}{3} = 0$$

Final metric value:

$$\begin{aligned} TSSI(\mathrm{C}) &= 0.3 \cdot S_{nom}(\mathrm{C}) + 0.3 \cdot S_{var}(\mathrm{C}) + 0.4 \cdot S_{rel}(\mathrm{C}) \\ &= 0.3 \cdot 0 + 0.3 \cdot 0 + 0.4 \cdot 0 = 0 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

C supports two semantically distinct visibility scopes via storage-class specifiers and linkage rules: *external linkage* (identifiers visible across translation units) and *internal linkage* (restricted to a single translation unit using the `static` keyword).

$$n_{levels}(\mathrm{C\#}) = 7, n_{levels}(\mathrm{C}) = 2 \Rightarrow V(\mathrm{C}) = \tfrac{2}{7}$$

2) Module Isolation Strength $M(L)$

C provides a single primary modularization construct: the *translation unit*, formed after preprocessing, which serves as the compilation and visibility boundary. However, this boundary does not enforce explicit export control lists: visibility across translation units is governed implicitly by linkage rules (external vs. internal) and declarations in header files, rather than by a formal mechanism requiring explicit export specifications. Consequently, there are no boundaries with enforced export control.

$$b_{strict}(\mathrm{C}) = 0, b_{total}(\mathrm{C}) = 1 \Rightarrow M(\mathrm{C}) = 0$$

3) Interface Abstraction Capacity $I(L)$

C does not provide dedicated language-level constructs for behavioral abstraction such as interfaces, abstract classes, or protocols. Abstraction is achieved idiomatically using header files and function declarations, which are not distinct semantic constructs in the type system.

$$I(\mathrm{C}) = 0$$

4) Encapsulation Enforcement Coefficient $E(L)$

C does not enforce encapsulation at the language level: it allows unrestricted memory access via pointers, including arbitrary casting between incompatible types (subject only to undefined behavior constraints), direct field access to struct members, and manipulation of object representations through pointer arithmetic. Additionally, there are no runtime checks preventing access violations, nor controlled unsafe regions distinct from safe code.

$$E(\mathrm{C}) = 0$$

Final metric value:

$$\begin{aligned} AE(\mathrm{C}) &= 0.2 \cdot V(\mathrm{C}) + 0.25 \cdot M(\mathrm{C}) + 0.25 \cdot I(\mathrm{C}) + 0.3 \cdot E(\mathrm{C}) \\ &= 0.2 \cdot \frac{2}{7} + 0.25 \cdot 0 + 0.25 \cdot 0 + 0.3 \cdot 0 \\ &\approx 0.0571 \end{aligned}$$

Inheritance Semantics and Hierarchy Management

According to the ISO C standard, C does not support inheritance:

$$H_s(\mathrm{C}) = 0, L_d(\mathrm{C}) = 0, C_p(\mathrm{C}) = 0, F_b(\mathrm{C}) = 0$$

Final metric value:

$$ISRI(\mathrm{C}) = 0.2 \cdot H_s(\mathrm{C}) + 0.25 \cdot L_d(\mathrm{C}) + 0.3 \cdot C_p(\mathrm{C}) + 0.25 \cdot F_b(\mathrm{C}) = 0$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

C provides neither keywords nor built-in constructs for delegation. However, delegation-like behavior can be manually emulated using function pointers and struct composition (e.g., forwarding calls through embedded structures), which corresponds to adapter-based emulation. The language requires explicit function calls and does not include any facility for automatic method delegation or forwarding without boilerplate code.

$$K_{del}(\mathrm{C}) = 0.5, S_{fwd}(\mathrm{C}) = 0 \Rightarrow D_{cls}(\mathrm{C}) = 0.25$$

2) Trait Expressiveness Score $T_{exp}(L)$

Since C does not support traits or multiple composition units with overlapping method namespaces, such conflicts cannot be resolved via language-defined mechanisms and instead typically result in compilation errors (e.g., duplicate symbol definitions). C lacks traits or mixins entirely, and thus provides no facility to specify required methods or fields for composition.

$$T_{exp}(\mathrm{C}) = 0$$

3) Interface Composition Capacity $I_{comp}(L)$

No interfaces:

$$I_{comp}(\mathrm{C}) = 0$$

Final metric value:

$$\begin{aligned} CRFI(\mathrm{C}) &= 0.3 \cdot D_{cls}(\mathrm{C}) + 0.35 \cdot T_{exp}(\mathrm{C}) + 0.35 \cdot I_{comp}(\mathrm{C}) \\ &= 0.3 \cdot 0.25 + 0.35 \cdot 0 + 0.35 \cdot 0 \\ &= 0.075 \end{aligned}$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

The set of user-definable type categories includes struct, union, and enum. These types are defined with value semantics: objects of these types are copied

on assignment and passed by value unless explicitly manipulated via pointers, meaning the language does not enforce reference semantics for any user-defined type category.

$$T_{user}(\mathrm{C}) = 3, T_{ref}(\mathrm{C}) = 0 \Rightarrow S_{id}(\mathrm{C}) = 0$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

C does not provide high-level instantiation constructs such as new, prototype cloning, or actor-based spawning. It supports two fundamental paradigms: static or automatic allocation and dynamic allocation via standard library functions such as malloc. These mechanisms correspond to low-level memory allocation rather than abstract object instantiation models, but they are the only native paradigms available in the language.

$$P_{supported}(\mathrm{C\#}) = 4 \Rightarrow S_{cr}(\mathrm{C}) = \tfrac{2}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

C does not provide any built-in mechanisms for runtime introspection or structural modification of objects. All type information is resolved at compile time, and no standardized runtime type system exists.

$$I_{level}(\mathrm{C}) = 0 \Rightarrow S_{meta}(\mathrm{C}) = 0$$

Final metric value:

$$\begin{aligned} ORCI(\mathrm{C}) &= 0.35 \cdot S_{id}(\mathrm{C}) + 0.3 \cdot S_{cr}(\mathrm{C}) + 0.35 \cdot S_{meta}(\mathrm{C}) \\ &= 0.35 \cdot 0 + 0.3 \cdot \frac{2}{4} + 0.35 \cdot 0 \\ &= 0.15 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

C does not support single dispatch in the object-oriented sense, as there are no methods or implicit receiver-based calls, function calls are resolved statically at compile time.

$I_{single}(\mathrm{C}) = 0$

Similarly, C does not provide multiple dispatch (i.e., dynamic selection based on multiple argument types).

$I_{multi}(\mathrm{C}) = 0$

Predicate-based dispatch is also not natively supported. Although conditional logic (e.g., if, switch) can be used to manually implement behavior selection, this is not part of the language's dispatch semantics.

$I_{pred}(\mathrm{C}) = 0$

$M_{scope}(\mathrm{C}) = 0$

2) Dispatch Safety Guarantee $S_{disp}(L)$

C does not provide built-in polymorphic dispatch mechanisms. Instead, dispatch is performed either statically via direct function calls or manually through function pointers. Although direct calls are type-checked, the use of function

pointers does not guarantee full compile-time verification of dispatch correctness.

$I_{static}(\mathrm{C}) = 0$

C does not enforce exhaustive dispatch definitions: constructs like switch statements do not require covering all possible cases (even for enumerations), and missing cases are not mandated to be diagnosed by the compiler.

$I_{exhaust}(\mathrm{C}) = 0$

$$S_{disp}(\mathrm{C}) = \frac{0+0}{2} = 0$$

3) Virtualization Optimization Potential $O_{virt}(L)$

Since C does not define virtual methods or dynamic dispatch, the notion of final methods is not applicable.

$K_{final}(\mathrm{C}) = 0$

Likewise, C lacks class hierarchies entirely, and therefore provides no mechanism for restricting subclassing (i.e., sealed hierarchies).

$K_{sealed}(\mathrm{C}) = 0$

However, all function calls in C are statically bound by default (except via function pointers), and the static keyword allows internal linkage and potential compiler optimizations, which can be interpreted as a form of static dispatch hint within the language specification.

$K_{static}(\mathrm{C}) = 1$

$$O_{virt}(\mathrm{C}) = \frac{1}{3}$$

Final metric value:

$$\begin{aligned} \mathit{PDEI}(\mathrm{C}) &= 0.35 \cdot \mathit{M}_{\mathit{scope}}(\mathrm{C}) + 0.35 \cdot \mathit{S}_{\mathit{disp}}(\mathrm{C}) + 0.3 \cdot \mathit{O}_{\mathit{virt}}(\mathrm{C}) \\ &= 0.35 \cdot 0 + 0.35 \cdot 0 + 0.3 \cdot \frac{1}{3} \\ &= 0.1 \end{aligned}$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

In C, all objects are mutable by default unless explicitly qualified with the const type qualifier, as defined in the ISO C standard (e.g., ISO/IEC 9899), which states that the absence of const permits modification of stored values through lvalues.

$$\mathit{M}_{\mathit{def}}(\mathrm{C}) = 0$$

2) Static Reference Constraint Density $S_{dens}(L)$

In C, references are represented exclusively via pointers: pointer types (T^*), pointer qualifiers such as const, volatile, and restrict, and array-to-pointer decay in expressions.

$\mathit{N}_{\mathit{ref}}(\mathrm{C}) = 3$

Next, distinct syntactic constructs include: the const qualifier enforces compile-time immutability of the referenced object, and restrict imposes aliasing constraints that enable compiler optimizations by guaranteeing exclusive access within a scope, volatile does not impose aliasing or mutability constraints but rather affects optimization semantics, and pointer types and decay introduce no restrictions.

$N_{constr}(\mathrm{C}) = 2$

$S_{dens}(\mathrm{C}) = \frac{2}{3}$

3) Concurrency Primitive Safety $P_{conc}(L)$

First, C provides no type-system or runtime enforcement of data race freedom. Although the standard defines a memory model and introduces atomic operations via _Atomic and stdatomic.h, correct synchronization is entirely the programmer's responsibility, and violations result in undefined behavior.

$S_{race}(\mathrm{C}) = 0$

Second, C only offers low-level concurrency primitives, such as threads (via threads.h) and mutexes, without higher-level abstractions like actors, channels, or structured task systems.

$S_{model}(\mathrm{C}) = 0$

$P_{conc}(\mathrm{C}) = 0$

Final metric value:

$$\begin{aligned} SCSI(\mathrm{C}) = 0.35 \cdot M_{def}(\mathrm{C}) + 0.25 \cdot S_{dens}(\mathrm{C}) + 0.4 \cdot P_{conc}(\mathrm{C}) \\ = 0.35 \cdot 0 + 0.25 \cdot \frac{2}{3} + 0.4 \cdot 0 \\ \approx 0.1667 \end{aligned}$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

The language provides no native support for design-by-contract features:

preconditions and postconditions are not part of the function signature or type system, and invariants are not expressible as built-in semantic constructs.

$$C_{nat}(\mathrm{C}) = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

C does not support static verification in the sense of compile-time proof of program correctness. The type system is not expressive enough to encode and verify formal specifications.

$$V_{enf}(\mathrm{C}) = 0$$

Final metric value:

$$\begin{aligned} FSVI(\mathrm{C}) &= 0.45 \cdot C_{nat}(\mathrm{C}) + 0.55 \cdot V_{enf}(\mathrm{C}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0 = 0 \end{aligned}$$

4.1.6 Metrics Evaluation for Golang

Introduction

The Go programming language (Golang) represents a statically typed, compiled language that deliberately departs from classical object-oriented paradigms [54]. Unlike languages such as C++ or C#, Go does not provide inheritance, subtype polymorphism via class hierarchies, or explicit object constructs. Instead, it adopts a composition-based design philosophy combined with structural typing through interfaces. Pike et al., that Go is not an object-oriented language, because it has no classes, inheritance, and explicit method hierarchies [22].

1) Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

Go supports structural typing through its interface system: a type satisfies an interface implicitly if it implements the required method set, without explicit declaration, which corresponds to structural subtyping governed by formal assignability and method-set rules in the specification.

$I_{struct}(\text{Golang}) = 1$

Go does not support nominal subtyping with explicit subtype declarations (e.g., no extends or inheritance hierarchy). The type relationships are not declared but inferred structurally, and even named types are distinct unless explicitly converted.

$I_{nom}(\text{Golang}) = 0$

$S_{nom}(\text{Golang}) = \frac{1+0}{2} = 0.5$

2) Variance Formalization Score $S_{var}(L)$

Go does not provide explicit variance annotations (such as covariance or contravariance keywords). All type parameters are invariant, and variance behavior is implicitly restricted by the type system design, particularly through constraint interfaces and the absence of subtyping between instantiated generic types.

$V_{exp}(\text{Golang}) = 0.5$

The specification defines clear and conservative typing and instantiation rules that prevent common variance-related unsoundness (e.g., no array covariance), but it does not include formally proven variance theorems or an explicit formalization of variance properties. The soundness is achieved through restriction and simplicity.

$V_{sound}(\text{Golang}) = 0.5$

$S_{var}(\text{Golang}) = \frac{0.5+0.5}{2} = 0.5$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

Go allows subtyping (in the form of interface satisfaction) without any inheritance constructs such as class extension. The types implement interfaces implicitly based on method sets, independent of declaration.

$I_{dec}(\text{Golang}) = 1$

The subtyping rules are defined entirely through assignability and interface implementation semantics, which are specified independently of any inheritance mechanism.

$I_{ind}(\text{Golang}) = 1$

Although the specification clearly distinguishes named types, interface types, and implementation rules, it does not formalize inheritance as a relation at all, and the separation is therefore implicit rather than expressed as two formally distinct relations.

$I_{form}(\text{Golang}) = 0.5$

$S_{rel}(\text{Golang}) = \frac{1+1+0.5}{3} \approx 0.83,$

Final metric value:

$$\begin{aligned} TSSI(\text{Golang}) &= 0.3 \cdot S_{nom}(\text{Golang}) + 0.3 \cdot S_{var}(\text{Golang}) + 0.4 \cdot S_{rel}(\text{Golang}) \\ &= 0.3 \cdot 0.5 + 0.3 \cdot 0.5 + 0.4 \cdot \frac{5}{6} \\ &\approx 0.6334 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

Go provides a single mechanism for visibility based on identifier capitalization: identifiers starting with an uppercase letter are exported (public at the package level), while those starting with a lowercase letter are unexported (private to the package), with no additional scopes such as protected, file-level, or module-level visibility.

$$n_{levels}(\text{C\#}) = 7, n_{levels}(\text{Golang}) = 2 \Rightarrow V(\text{Golang}) = \tfrac{2}{7}$$

2) Module Isolation Strength $M(L)$

Go provides two primary boundaries: packages and modules. However, only packages enforce access control via export rules (identifiers are visible outside the package only if exported), while modules (defined via $\mathrm{go.mod}$) serve as dependency and versioning units without introducing additional visibility restrictions.

$$b_{strict}(\text{Golang}) = 1, b_{total}(\text{Golang}) = 2 \Rightarrow M(\text{Golang}) = \tfrac{1}{2} = 0.5$$

3) Interface Abstraction Capacity $I(L)$

Go provides a single dedicated abstraction construct—interfaces—which define behavior via method sets independently of implementation.

$n_{abstraction_constructs}(\text{Golang}) = 1$

For object-defining constructs, Go includes concrete struct types as the primary means of defining composite data objects, Go does not include classes or alternative object forms. Interfaces are satisfied implicitly by any type whose method set matches, enabling strong separation of specification from implemen-

tation without inheritance.

$$n_{object}(\text{Golang}) = 1$$

$$I(\text{Golang}) = \tfrac{1}{1} = 1$$

4) Encapsulation Enforcement Coefficient $E(L)$

Although Go enforces encapsulation at the language level via package-based visibility rules, it also provides mechanisms that can bypass these guarantees. In particular, the unsafe package allows arbitrary memory access and pointer manipulation beyond type safety, and the reflect package enables inspection and, in limited cases, modification of otherwise inaccessible data at runtime. Because these mechanisms permit circumvention of encapsulation constraints, even if explicitly marked and discouraged, the condition of strictly preventing uncontrolled access is not satisfied.

$$E(\text{Golang}) = 0$$

Final metric value:

$$AE(\text{Golang}) = 0.2 \cdot V(\text{Golang}) + 0.25 \cdot M(\text{Golang}) + 0.25 \cdot I(\text{Golang}) + 0.3 \cdot E(\text{Gol}$$
$$= 0.2 \cdot \frac{2}{7} + 0.25 \cdot 0.5 + 0.25 \cdot 1 + 0$$
$$\approx 0.$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

Go does not support class-based inheritance or superclass declarations. It

relies on composition and interface-based polymorphism, meaning that the notion of direct superclasses is not defined.

$$M(\text{Golang}) = 1 \Rightarrow H_s(\text{Golang}) = 1 - \frac{1-1}{M_{max}-1} = 1$$

2) Linearization Determinism Score $L_d(L)$

Go does not implement inheritance or multiple inheritance, and therefore does not require a method resolution order (MRO) or linearization algorithm. Instead, method selection is determined deterministically via static rules over method sets and interface satisfaction, with no ambiguity or need for precedence resolution among multiple parent types.

$$L_d(\text{Golang}) = 1$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 0$: no final constructs.

$p_2 = 0$: no sealed hierarchies.

$p_3 = 0$: no $\mathrm{override}$ annotation due to absence of overriding.

$p_4 = 1$: methods are non-virtual by default in the sense that dispatch is static unless via interfaces.

$p_5 = 1$: clear separation between interfaces and concrete types is enforced by the type system.

$p_6 = 0$: no inheritance restriction rules exist because inheritance itself is absent.

$$C_p(\text{Golang}) = \frac{2}{6} \approx 0.33$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 0$: there is no override mechanism and thus no requirement for explicit override annotations.

$s_2 = 1$: Go enforces strict visibility via export rules (identifiers starting with uppercase are exported, otherwise package-private), preventing unintended external overriding.

$s_3 = 0$: there is no superclass or super invocation model.

$s_4 = 1$: type system enforces strict method set matching for interface satisfaction, disallowing unsafe variance in overriding-like behavior.

$s_5 = 1$: methods are not overridable by default due to the absence of inheritance—polymorphism is explicitly opt-in via interfaces.

$s_6 = 1$: the compiler performs static checks ensuring exact signature compatibility when a type satisfies an interface.

$$F_b(\text{Golang}) = (0 + 1 + 0 + 1 + 1 + 1)/6 \approx 0.67$$

Final metric value:

$$\begin{aligned} \mathit{ISRI}(\text{Golang}) &= 0.2 \cdot H_s(\text{Golang}) + 0.25 \cdot L_d(\text{Golang}) + 0.3 \cdot C_p(\text{Golang}) + 0.25 \cdot F \\ &= 0.2 \cdot 1 + 0.25 \cdot 1 + 0.3 \cdot \frac{2}{6} \end{aligned}$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

The delegation is not provided via explicit keywords but is partially supported through *type embedding*, which promotes methods of embedded fields to the outer

type. However, this mechanism is not a first-class delegation construct but rather a composition feature.

$K_{del}(\text{Golang}) = 0.5$

Go does not provide fully automatic forwarding of all method calls in the general case (e.g., no universal forwarding construct comparable to language-level delegation keywords), since method promotion only applies to embedded types and does not dynamically forward arbitrary calls.

$S_{fwd}(\text{Golang}) = 0$

$$K_{del}(\text{Golang}) = 0.5, S_{fwd}(\text{Golang}) = 0 \Rightarrow D_{cls}(\text{Golang}) = 0.25$$

2) Trait Expressiveness Score $T_{exp}(L)$

The Go Language Specification does not define traits or mixins as first-class constructs. Composition is achieved via interfaces and struct embedding (*Interfaces*, *Method sets*, *Embedding*). When multiple embedded types introduce methods with identical names, Go does not provide aliasing or exclusion mechanisms. Such ambiguity leads to a compile-time error unless explicitly disambiguated by qualification.

$R_{conf}(\text{Golang}) = 0$

Go interfaces can specify required method sets that a type must implement, effectively expressing behavioral requirements. However, these requirements are defined externally (in interfaces) rather than within reusable composition units analogous to traits, so composition units themselves (embedded structs) cannot declare constraints.

$O_{mod}(\text{Golang}) = 0$

$$T_{exp}(\text{Golang}) = (0 + 0)/2 = 0$$

3) Interface Composition Capacity $I_{comp}(L)$

In the Go Language Specification (*Interfaces*, *Type sets*), a type implicitly implements any number of interfaces by satisfying their method sets without explicit declaration, implying no restriction on the number of interfaces.

$N_{impl}(\text{Golang}) = 1$

Go interfaces are purely behavioral contracts and may only contain method signatures (and, since Go 1.18, type sets for constraints), but they do not allow method bodies or default implementations.

$D_{def}(\text{Golang}) = 0$

$$I_{comp}(\text{Golang}) = (1 + 0)/2 = 0.5$$

Final metric value:

$$\begin{aligned} CRFI(\text{Golang}) = 0.3 \cdot D_{cls}(\text{Golang}) + 0.35 \cdot T_{exp}(\text{Golang}) + 0.35 \cdot I_{comp}(\text{Golang}) \\ = 0.3 \cdot 0.25 + 0.35 \cdot 0 + 0.35 \cdot 0.5 \\ = 0.25 \end{aligned}$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

According to the Go Language Specification, user-definable type categories include *struct*, *array*, *slice*, *map*, *channel*, *function*, and *interface* types.

$T_{user}(\text{Golang}) = 7$

Among these, only a subset enforces reference semantics by design: slices,

maps, channels, functions, and interfaces exhibit reference-like behavior (they internally contain descriptors referencing underlying data structures), while structs and arrays are strictly value types copied on assignment.

$T_{ref}(\text{Golang}) = 5$

$S_{id}(\text{Golang}) = \frac{5}{7} \approx 0.71$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

The Go Language Specification (*Composite literals*, *Allocation*, *Making slices, maps, and channels*) defines several native instantiation paradigms:

- Allocation via new returning a pointer to zero-initialized storage.
- Literal construction using composite literals for structs, arrays, slices, and maps.
- Specialized initialization via make for slices, maps, and channels with internal runtime setup.

$P_{supported}(\text{Golang}) = 3, P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{Golang}) = \frac{3}{4}$

3) Meta-level Extensibility Score $S_{meta}(L)$

According to the Go Language Specification and its *reflect* package model (*Reflection*), Go provides runtime introspection capabilities that allow programs to query type information, method sets, and value properties dynamically. However, it does not permit structural modification of existing types or objects at runtime, nor does it support overriding object creation protocols via metaobjects.

$$I_{level}(\text{JS/Python/Ruby}) = 3, I_{level}(\text{Golang}) = 1 \Rightarrow S_{meta}(\text{Golang}) = \tfrac{1}{3}$$

Final metric value:

$$\begin{aligned} ORCI(\text{Golang}) &= 0.35 \cdot S_{id}(\text{Golang}) + 0.3 \cdot S_{cr}(\text{Golang}) + 0.35 \cdot S_{meta}(\text{Golang}) \\ &= 0.35 \cdot \frac{5}{7} + 0.3 \cdot 0.75 + 0.35 \cdot \frac{1}{3} \\ &\approx 0.5917 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

According to the Go Language Specification (*Method sets*, *Interfaces*), Go supports receiver-based method invocation, which corresponds to single dispatch determined by the dynamic type of the receiver in interface calls.

$I_{single}(\text{Golang}) = 1$

However, Go does not natively support multiple dispatch, as method selection is not based on the runtime types of multiple arguments but solely on the receiver.

$I_{multi}(\text{Golang}) = 0$

Similarly, predicate-based dispatch is not part of the language specification, conditional logic must be expressed explicitly via control flow constructs rather than dispatch mechanisms.

$I_{pred}(\text{Golang}) = 0$

$$M_{scope}(\text{Golang}) = (1 \cdot 1 + 2 \cdot 0 + 2 \cdot 0)/(1 + 2 + 2) = 1/5 = 0.2$$

2) Dispatch Safety Guarantee $S_{disp}(L)$

According to the Go Language Specification (*Interfaces*, *Method sets*, *Type*

assertions, *Type switches*), dispatch correctness is partially verified at compile time: the compiler ensures that a concrete type satisfies an interface before assignment or usage, guaranteeing method set compatibility.

$I_{static}(\text{Golang}) = 1$

However, Go does not enforce exhaustiveness of dispatch constructs.

$I_{exhaust}(\text{Golang}) = 0$

$$S_{disp}(\text{Golang}) = (1 + 0)/2 = 0.5$$

3) Virtualization Optimization Potential $O_{virt}(L)$

The Go Language Specification (*Method declarations*, *Interfaces*) does not define virtual methods or inheritance-based dispatch. The method calls on concrete types are statically bound, and dynamic dispatch occurs only through interfaces without user-level control over virtualization.

$K_{final}(\text{Golang}) = 0$: there is no notion of marking methods as final or non-virtual.

$K_{sealed}(\text{Golang}) = 0$: Go lacks class hierarchies and therefore provides no mechanism for sealing inheritance.

$K_{static}(\text{Golang}) = 0$: there are no explicit language-level annotations or keywords for influencing dispatch optimization.

$$O_{virt}(\text{Golang}) = (0 + 0 + 0)/3 = 0$$

Final metric value:

$$\begin{aligned} PDEI(\text{Golang}) &= 0.35 \cdot M_{scope}(\text{Golang}) + 0.35 \cdot S_{disp}(\text{Golang}) + 0.3 \cdot O_{virt}(\text{Golang}) \\ &= 0.35 \cdot 0.2 + 0.35 \cdot 0.5 + 0.3 \cdot 0 \\ &= 0.245 \end{aligned}$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

In Go, variables declared using the var keyword or short declaration operator $:=$ are mutable unless explicitly constrained otherwise. There is no built-in language construct to declare variables as immutable after initialization. The Go specification explicitly defines variables as storage locations whose values can be changed during program execution, and while constants (declared via const) are immutable, they represent a distinct category of compile-time values rather than general-purpose objects.

$$M_{def}(\text{Golang}) = 0$$

2) Static Reference Constraint Density $S_{dens}(L)$

According to the Go specification, reference-related constructs include: pointers, slices (descriptor referencing an underlying array), maps (reference to runtime hash table), channels (reference to communication object), and function values (which internally reference closures).

$N_{ref}(\text{Golang}) = 5$

The type system does not provide constructs that restrict aliasing (no ownership, borrowing, or uniqueness types) nor mutability of referenced data at compile

time. All listed constructs allow unrestricted aliasing and mutation subject only to basic type safety.

$N_{constr}(\text{Golang}) = 0$

$S_{dens}(\text{Golang}) = \frac{0}{5} = 0$

3) Concurrency Primitive Safety $P_{conc}(L)$

Go does not provide compile-time guarantees of data race freedom via its type system. The language offers a runtime data race detector (enabled via tooling such as -race) that dynamically identifies conflicting memory accesses, implying partial enforcement rather than formal prevention.

$S_{race}(\text{Golang}) = 0.5$

Go natively implements a high-level concurrency model based on Communicating Sequential Processes (CSP), with goroutines and typed channels as first-class language constructs enabling structured communication and synchronization without explicit shared-memory coordination, as specified in its memory model and concurrency semantics.

$S_{model}(\text{Golang}) = 1$

$P_{conc}(\text{Golang}) = \frac{0.5+1}{2} = 0.75$

Final metric value:

$$SCSI(\text{Golang}) = 0.35 \cdot M_{def}(\text{Golang}) + 0.25 \cdot S_{dens}(\text{Golang}) + 0.4 \cdot P_{conc}(\text{Golang})$$
$$= 0.35 \cdot 0 + 0.25 \cdot 0 + 0.4 \cdot 0.75 = 0.3$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

The Go specification does not include any native language features for design-by-contract: there are no keywords or type-system mechanisms to declare pre-conditions, or invariants that are enforced either at compile time or runtime. Such constraints are typically expressed through explicit if-statements, error handling patterns, or documentation comments, which fall outside the definition of native constructs.

$$n_{pre} = 0, n_{post} = 0, n_{inv} = 0$$
$$C_{nat}(\text{Golang}) = \frac{0+0+0}{3} = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

Go does not support formal static verification within its type system or compiler. There are no dependent types, refinement types, or built-in verification frameworks. However, Go does support runtime checking through explicit error handling, panic/$\mathrm{recover}$ mechanisms, and testing frameworks, which allow developers to validate properties during execution, though these are not enforced as formal contracts by the language itself.

$$V_{enf}(\text{Golang}) = 0.5$$

Final metric value:

$$FSVI(\text{Golang}) = 0.45 \cdot C_{nat}(\text{Golang}) + 0.55 \cdot V_{enf}(\text{Golang})$$
$$= 0.45 \cdot 0 + 0.55 \cdot 0.5 = 0.275$$

4.1.7 Metrics Evaluation for Scala

Scala represents a hybrid object-functional programming language with a formally specified type system and a rich set of object-oriented abstractions. Its design, as described in the Scala Language Specification and foundational works such as Odersky et al. [38] and Emir et al. [55], explicitly aims to unify object-oriented and functional paradigms through a uniform object model and advanced type constructs. This subsection evaluates Scala according to the proposed metrics, grounding each value in language specification details and peer-reviewed research.

Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

Scala employs nominal subtyping through explicit declarations such as class, trait, and inheritance clauses (e.g., $\mathrm{extends}$), where subtype relations are defined by named entities and governed by formal typing judgments. Scala also supports structural types via refinement types, where type compatibility is determined by the presence of members rather than explicit inheritance, and this behavior is formally specified in its typing rules [38].

$$S_{nom}(\text{Scala}) = 1, I_{struct} = 1 \Rightarrow S_{nom} = 1$$

2) Variance Formalization Score $S_{var}(L)$

Scala provides fully explicit variance annotations, where type parameters can be declared covariant ($+\mathrm{A}$), contravariant ($\text{-}\mathrm{A}$), or invariant (default), and their usage is syntactically enforced by the compiler through well-defined typing rules.

$V_{exp}(\text{Scala}) = 1.$

Furthermore, the specification formalizes variance soundness via restrictions on positions of type parameters (e.g., covariant types cannot appear in contravariant positions), ensuring type safety through compile-time checks derived from formal subtyping and variance rules.

$V_{sound}(\text{Scala}) = 1.$

$S_{var}(\text{Scala}) = 1$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, although nominal subtyping is primarily introduced via class and trait inheritance, the language also permits limited explicit subtyping without direct inheritance through constructs such as refinement types and type bounds (e.g., $A <: B$ in type parameters), indicating partial but not fully general separation.

$I_{dec}(\text{Scala}) = 0.5$

Second, Scala's subtyping relation is formally richer than inheritance alone (including variance, compound types, and path-dependent types), yet still partially grounded in declared inheritance hierarchies, implying only partial independence.

$I_{ind}(\text{Scala}) = 0.5$

Third, the specification distinguishes subtyping ($<:$) from inheritance (class/-trait extension) at a conceptual level, but does not fully axiomatize them as completely orthogonal relations.

$I_{form}(\text{Scala}) = 0.5$

$S_{rel}(\text{Scala}) = \frac{0.5+0.5+0.5}{3} = 0.5$

Final metric value:

$$\begin{aligned} TSSI(\text{Scala}) &= 0.3 \cdot S_{nom}(\text{Scala}) + 0.3 \cdot S_{var}(\text{Scala}) + 0.4 \cdot S_{rel}(\text{Scala}) \\ &= 0.3 \cdot 1 + 0.3 \cdot 1 + 0.4 \cdot 0.5 \\ &= 0.8 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

Scala provides multiple semantically distinct access control levels: private[this] (object-private), private (class-private), qualified private[pkg] (package-restricted), protected[this] (object-protected), protected (subclass access), and unrestricted public access (default).

$$n_{levels}(\text{C\#}) = 7 \Rightarrow V(\text{Scala}) = \tfrac{6}{7}$$

2) Module Isolation Strength $M(L)$

Scala provides several modularization constructs, namely packages, package objects, singleton objects, classes, and traits, which together form $b_{total}(\text{Scala}) = 5$ distinct module or namespace boundaries. However, Scala does not enforce strict export lists at the module level. The visibility is controlled via access modifiers (e.g., private[pkg]) rather than explicit export declarations, meaning no boundary inherently requires an explicit interface of exported members. While newer Scala versions introduce selective imports/exports (e.g., export clauses), these do not constitute mandatory encapsulation barriers in the formal core specification: $b_{strict}(\text{Scala}) = 0$.

$$M(\text{Scala}) = \tfrac{0}{5} = 0$$

3) Interface Abstraction Capacity $I(L)$

Scala provides multiple constructs for separating specification from implementation: trait (the primary abstraction mechanism supporting abstract members and mixin composition) and $\mathrm{abstract\ class}$ (which allows partial implementation with abstract definitions).

$n_{abstraction\_constructs}(\text{Scala}) = 2$

The set of object-defining constructs includes class, trait, and object (singleton).

$n_{object}(\text{Scala}) = 3$

$$I(\text{Scala}) = \tfrac{2}{3} \approx 0.67$$

4) Encapsulation Enforcement Coefficient $E(L)$

Scala enforces encapsulation at the type system level via access modifiers (e.g., $\mathrm{private}$, $\mathrm{protected}$, and qualified visibility), it does not fully prevent encapsulation violations at runtime. Specifically, Scala inherits unrestricted reflection capabilities from the Java ecosystem (e.g., $\mathrm{java.lang.reflect}$) and allows access to private members through reflective APIs, as well as other unsafe mechanisms available on the JVM. These capabilities are not restricted by the Scala type system and are not prohibited in the specification.

$$E(\text{Scala}) = 0$$

Final metric value:

$$AE(\text{Scala}) = 0.2 \cdot V(\text{Scala}) + 0.25 \cdot M(\text{Scala}) + 0.25 \cdot I(\text{Scala}) + 0.3 \cdot E(\text{Scala})$$
$$= 0.2 \cdot \frac{6}{7} + 0.25 \cdot 0 + 0.25 \cdot \frac{2}{3} + 0.3 \cdot 0 \approx 0.3381$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

Scala enforces single inheritance for classes (each class may extend at most one superclass) while allowing multiple inheritance only through trait mixins, which are composed using a well-defined linearization algorithm that preserves a single inheritance chain at the class level. Thus, the maximum number of direct superclass declarations for classes is 1.

$$H_s(\text{Scala}) = 1$$

2) Linearization Determinism Score $L_d(L)$

Scala defines a precise and formally specified method resolution order based on a deterministic linearization algorithm for classes and traits. This algorithm constructs a single linear hierarchy by combining the superclass chain with mixin traits in a well-defined order, ensuring consistent resolution of overridden members and eliminating ambiguity in multiple inheritance scenarios [38].

$$L_d(\text{Scala}) = 1$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1(\text{Scala}) = 1$ since the language provides final for classes and methods, preventing further inheritance or overriding

$p_2(\text{Scala}) = 1$ due to sealed traits/classes restricting subclassing to the same

compilation unit

$p_3(\text{Scala}) = 1$ because overriding requires the explicit override modifier, enforced at compile time

$p_4(\text{Scala}) = 0$ as methods are virtual by default and dynamic dispatch is the standard semantics

$p_5(\text{Scala}) = 0$ because the language does not enforce a strict separation between abstract classes and interfaces, instead unifying them via traits

$p_6(\text{Scala}) = 1$ because the compiler enforces inheritance constraints such as prohibiting extension of final classes and enforcing linearization rules

$$C_p(\text{Scala}) = \tfrac{4}{6} \approx 0.67$$

4) Fragile Base Mitigation Factor $F_b(L)$

Scala incorporates several semantic safeguards that mitigate fragile base class effects. These include mandatory override annotations for redefining inherited members, fine-grained visibility control (e.g., private, protected, and qualified access), restrictions on variance positions ensuring type safety, and a well-defined linearization model governing super calls in mixin compositions.

$s_1(\text{Scala}) = 1$ since the override modifier is mandatory and prevents accidental overriding

$s_2(\text{Scala}) = 1$ because the language provides strict visibility modifiers (e.g., private, protected, and scoped qualifiers) that control exposure of overridable members.

$s_3(\text{Scala}) = 0$ as explicit superclass invocation is not universally required and overriding methods may omit super calls.

$s_4(\text{Scala}) = 1$ due to formally defined variance annotations ($+/-$) with

compile-time restrictions ensuring type-safe subtyping and overriding.

$s_5(\text{Scala}) = 0$ because methods are overridable by default unless marked final, so polymorphism is not supported.

$s_6(\text{Scala}) = 1$ since the compiler enforces strict override consistency checks on method signatures and types.

$$F_b(\text{Scala}) = \tfrac{1+1+0+1+0+1}{6} = \tfrac{4}{6} \approx 0.67$$

Final metric value:

$$\begin{aligned} \mathit{ISRI}(\text{Scala}) &= 0.2 \cdot H_s(\text{Scala}) + 0.25 \cdot L_d(\text{Scala}) + 0.3 \cdot C_p(\text{Scala}) + 0.25 \cdot F_b(\text{Scala}) \\ &= 0.2 \cdot 1 + 0.25 \cdot 1 + 0.3 \cdot \frac{4}{6} + 0.25 \cdot \frac{4}{6} = 0.8167 \end{aligned}$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

Scala does not provide built-in syntactic delegation constructs (i.e., no dedicated keyword such as delegate). The delegation is typically emulated via composition and forwarding methods or via library-level patterns. Furthermore, the language specification does not define automatic method forwarding mechanisms at the language level.

$$K_{del}(\text{Scala}) = 0.5, S_{fwd}(\text{Scala}) = 0 \Rightarrow D_{cls}(\text{Scala}) = 0.25$$

2) Trait Expressiveness Score $T_{exp}(L)$

Scala resolves name conflicts during trait composition using a deterministic linearization order rather than explicit exclusion or aliasing mechanisms. In cases

of ambiguity, the rightmost trait in the mixin order takes precedence, and conflicts are resolved through overriding rules defined in the specification. Furthermore, Scala traits can declare requirements on the classes that mix them in via self-types (e.g., this: SomeType =>), which are formally specified constraints on the composition context.

$$R_{conf}(\text{Scala}) = 0.5, O_{mod}(\text{Scala}) = 1 \Rightarrow R_{conf}(\text{Scala}) = 0.75$$

3) Interface Composition Capacity $I_{comp}(L)$

Scala does not have a separate interface construct. Instead, traits serve as their direct analogue and extend beyond traditional interfaces. A class may mix in an unrestricted number of traits using the with keyword. Traits can contain fully defined method implementations, fields, and even initialization logic, as formally specified in the language.

$$N_{impl}(\text{Scala}) = 1, D_{def}(\text{Scala}) = 1 \Rightarrow I_{comp} = 1$$

Final metric value:

$$\begin{aligned} CRFI(\text{Scala}) &= 0.3 \cdot D_{cls}(\text{Scala}) + 0.35 \cdot T_{exp}(\text{Scala}) + 0.35 \cdot I_{comp}(\text{Scala}) \\ &= 0.3 \cdot 0.25 + 0.35 \cdot 0.75 + 0.35 \cdot 1 = 0.6875 \end{aligned}$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

All user-defined types (classes, traits) are reference types: Scala distinguishes between reference types (instances of class, trait, and object) and value types (e.g., subclasses of AnyVal). But all user-definable type are reference types

$$S_{id} = 1$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

Scala natively supports standard constructor-based instantiation via the new keyword, singleton object instantiation through object declarations (module pattern with implicit initialization), and literal construction for built-in and case classes (e.g., apply methods enabling factory-style creation without new). Additionally, case classes provide a canonical factory mechanism via automatically generated companion objects.

$$P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{Scala}) = \tfrac{3}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

Scala supports runtime introspection through reflection APIs (e.g., scala.reflect and interoperability with java.lang.reflect), allowing programs to query type information and object structure dynamically. However, the language does not natively support full structural modification of objects (e.g., adding/removing fields or methods at runtime) nor overriding the object instantiation protocol via metaobjects within the core specification.

$$I_{level} = 1, I_{level}(\text{JS/Python/Ruby}) = 3 \Rightarrow S_{meta}(\text{Scala}) = \tfrac{1}{3}$$

Final metric value:

$$\begin{aligned} ORCI(\text{Scala}) &= 0.35 \cdot S_{id}(\text{Scala}) + 0.3 \cdot S_{cr}(\text{Scala}) + 0.35 \cdot S_{meta}(\text{Scala}) \\ &= 0.35 \cdot 1 + 0.3 \cdot \frac{3}{4} + 0.35 \cdot \frac{1}{3} \approx 0.6917 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

Scala natively supports single dispatch based on the dynamic type of the receiver object.

$I_{single}(\text{Scala}) = 1$

But, the language does not provide true multiple dispatch in the sense of dynamic selection based on all argument types. Method overloading is resolved statically at compile time rather than dynamically.

$I_{multi}(\text{Scala}) = 0$

Similarly, Scala does not include predicate-based dispatch as a formal language feature (dispatch based on arbitrary runtime conditions is expressed via pattern matching or control flow rather than method selection).

$I_{pred}(\text{Scala}) = 0$

$M_{scope}(\text{Scala}) = \frac{1}{5} = 0.2$

2) Dispatch Safety Guarantee $S_{disp}(L)$

First, method dispatch correctness is enforced statically through the type system, including method resolution, override checking, and type-safe dynamic dispatch constrained by static typing judgments.

$I_{static}(\text{Scala}) = 1$

Second, the language enforces exhaustiveness in pattern matching over sealed hierarchies and algebraic data types, with the compiler issuing errors or warnings when cases are incomplete, thus providing specification-level guarantees of dispatch completeness in such contexts.

$I_{exhaust}(\text{Scala}) = 1$

$S_{disp}(\text{Scala}) = \frac{1+1}{2} = 1$

3) Virtualization Optimization Potential $O_{virt}(L)$

Scala supports final declarations for classes and methods, preventing overriding and enabling devirtualization opportunities.

$K_{final}(\text{Scala}) = 1$

It also provides sealed classes and traits, which restrict subclassing to the same compilation unit and allow the compiler to reason about closed hierarchies.

$K_{sealed}(\text{Scala}) = 1$

Scala does not define explicit static methods or language-level static dispatch annotations in the same sense as some other languages (despite JVM-level optimizations), and dispatch remains primarily virtual unless restricted by final.

$K_{static}(\text{Scala}) = 0$

$O_{virt}(\text{Scala}) = \frac{1+1+0}{3} \approx 0.67$

Final metric value:

$$PDEI(\text{Scala}) = 0.35 \cdot M_{scope}(\text{Scala}) + 0.35 \cdot S_{disp}(\text{Scala}) + 0.3 \cdot O_{virt}(\text{Scala})$$
$$= 0.35 \cdot 0.2 + 0.35 \cdot 1 + 0.3 \cdot \frac{2}{3} = 0.62$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

although Scala promotes immutability idiomatically (e.g., via val bindings and immutable collections), mutability is explicitly available and controlled through the var keyword and mutable collection types, meaning that immutability

must be declared rather than being enforced by default at the object level.

$$M_{def}(\text{Scala}) = 0.5$$

2) Static Reference Constraint Density $S_{dens}(L)$

First, the constructs that introduce or manipulate references in Scala programming language: object references (all values are references except primitives under erasure), val bindings, var bindings, class/trait fields, parameter passing (by-value references), and path-dependent and refined types that qualify references.

$N_{ref}(\text{Scala}) = 6$

Second, the constructs that enforce static restrictions: val (immutability of the reference binding), path-dependent types (restricting reference usage via stable paths), and refined/structural types (constraining accessible members at compile time), while var, general object references, fields, and parameter passing do not impose strict aliasing or mutability constraints.

$N_{constr}(\text{Scala}) = 3$

$$S_{dens}(\text{Scala}) = \tfrac{3}{6} = 0.5$$

3) Concurrency Primitive Safety $P_{conc}(L)$

First, Scala's type system does not enforce data race freedom (e.g., shared mutable state via var is permitted without compile-time guarantees), and the Java Memory Model underlies its execution semantics. The correctness relies largely on programmer discipline and external libraries rather than intrinsic type safety.

$S_{race}(\text{Scala}) = 0$

Second, while the core language exposes low-level JVM threads, Scala's standard library provides Future and Promise abstractions (structured concurrency), and widely adopted frameworks such as Akka introduce Actor-based high-level concurrency models. However, since these are not strictly part of the core language specification.

$S_{model}(\text{Scala}) = 0.5.$

$P_{conc}(\text{Scala}) = 0.25$

Final metric value:

$$SCSI(\text{Scala}) = 0.35 \cdot M_{def}(\text{Scala}) + 0.25 \cdot S_{dens}(\text{Scala}) + 0.4 \cdot P_{conc}(\text{Scala})$$
$$= 0.35 \cdot 0.5 + 0.25 \cdot 0.5 + 0.4 \cdot 0.25 = 0.4$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

$n_{pre}(\text{Scala}) = 0$: Scala does not define native syntax for preconditions. Although developers may use require from the standard library, it is a conventional method rather than a first-class language construct with formal semantic enforcement.

$n_{post}(\text{Scala}) = 0$: Scala lacks intrinsic postcondition constructs (unlike languages with ensures clauses). While ensuring exists as a library extension method, it is not specified as a core semantic contract mechanism.

$n_{inv}(\text{Scala}) = 0$: Scala provides no native invariant constructs at the language level (e.g., no class invariants enforced by the compiler or runtime per specification).

$$C_{nat}(\text{Scala}) = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

Scala does not support built-in static verification or formal proof systems within its compiler (i.e., no native refinement types, dependent types, or contract verification as part of the specification). However, Scala's standard library includes runtime checking mechanisms such as require, assert, and ensuring, which allow developers to enforce conditions during execution, consistent with the operational semantics defined over the JVM.

$$V_{enf}(\text{Scala}) = 0.5$$

Final metric value:

$$\begin{aligned} FSVI(\text{Scala}) &= 0.45 \cdot C_{nat}(\text{Scala}) + 0.55 \cdot V_{enf}(\text{Scala}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0.5 = 0.275 \end{aligned}$$

4.1.8 Metrics Evaluation for JavaScript

JavaScript represents a dynamically typed, prototype-based object-oriented language standardized by ECMAScript (ECMA-262). Its object model fundamentally differs from class-based statically typed languages, emphasizing runtime flexibility, structural behavior, and reflective capabilities. As noted in [56], "objects are collections of properties" and inheritance is realized via prototype delegation rather than nominal hierarchies. Prior research highlights that JavaScript "lacks a formal static type system and relies on dynamic property lookup and late binding". This section evaluates JavaScript according to the proposed metrics, grounding

each numerical assignment in the language specification and established academic analyses.

Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

JavaScript does not provide a formal static type system with structural subtyping rules. It employs dynamic typing where type compatibility is resolved at runtime without formal typing judgments or compile-time subtyping relations. Although object shapes can be compared implicitly (duck typing), this behavior is not specified via formal structural typing rules in the language specification.

JavaScript lacks nominal typing: there are no explicit subtype declarations, and the prototype-based inheritance model does not constitute nominal subtyping in the formal type-theoretic sense. Subtyping relationships are not enforced through named type declarations but rather through dynamic property lookup.

$$I_{struct}(\text{JavaScript}) = 0, I_{nom}(\text{JavaScript}) = 0 \Rightarrow S_{nom}(\text{JavaScript}) = 0$$

2) Variance Formalization Score $S_{var}(L)$

ECMAScript does not include syntactic constructs for declaring parametric polymorphism or variance annotations (such as covariance or contravariance), and all values are dynamically typed without compile-time type parameterization. There are no formally defined variance rules, nor any specification-level guarantees of type soundness related to variance. The type behavior is governed entirely by runtime semantics without static verification or proofs of safety.

$$V_{exp}(\text{JavaScript}) = 0, V_{sound}(\text{JavaScript}) = 0 \Rightarrow S_{var}(\text{JavaScript}) = 0$$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, JavaScript allows subtyping (i.e., object compatibility and behavioral substitutability) independently of explicit inheritance constructs: objects can be created via object literals or functions and still be used interchangeably based on their structure, without requiring the [[Prototype]] linkage.

$I_{dec}(\text{JavaScript}) = 1$

Second, subtyping is not formally defined in the specification as an independent relation, but emerges implicitly through structural behavior (duck typing) and internal methods (e.g., [[Get]], [[Set]]). However, inheritance via the prototype chain influences property resolution, making the independence partial.

$I_{ind}(\text{JavaScript}) = 0.5$

Third, the ECMAScript specification does not formally distinguish subtyping from inheritance: it defines inheritance operationally via prototype chains, while subtyping is not explicitly formalized as a separate semantic relation.

$I_{form}(\text{JavaScript}) = 0$

$$S_{rel}(\text{JavaScript}) = \frac{1+0.5+0}{3} = 0.5.$$

Final metric value:

$$TSSI(\text{JavaScript}) = 0.3 \cdot S_{nom}(\text{JavaScript}) + 0.3 \cdot S_{var}(\text{JavaScript}) + 0.4 \cdot S_{rel}(\text{JavaScr}$$
$$= 0.3 \cdot 0 + 0.3 \cdot 0 + 0.4 \cdot 0.5 =$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

JavaScript supports: public visibility for all object properties by default,

class-private fields (denoted with the # syntax), which are enforced at the specification level via lexical scoping and inaccessible outside the declaring class[56].

$$n_{levels}(\text{JavaScript}) = 2, n_{levels}(\text{C\#}) = 7 \Rightarrow V(\text{JavaScript}) = \tfrac{2}{7}$$

2) Module Isolation Strength $M(L)$

JavaScript provides ES modules as the primary modularization construct, where bindings are explicitly exported and imported via export and import declarations, and only exported identifiers are accessible outside the module. These constitute strict boundaries with enforced export control. Additionally, other constructs such as script scope or function scope do not qualify as module systems in the specification-level sense of namespace isolation.

$$M(\text{JavaScript}) = 1$$

3) Interface Abstraction Capacity $I(L)$

JavaScript does not provide dedicated abstraction constructs such as interfaces, abstract classes, or protocols at the language specification level. While abstraction can be emulated through conventions (e.g., duck typing or documentation), these are not formalized as distinct syntactic or semantic constructs.

$$I(\text{JavaScript}) = 0$$

4) Encapsulation Enforcement Coefficient $E(L)$

JavaScript permits unrestricted runtime access to object properties via dynamic lookup (e.g., bracket notation), reflection-like mechanisms such as $\mathrm{Reflect}$ and Proxy, and mutation of object structure without compile-time restrictions.

These capabilities allow bypassing intended encapsulation boundaries for all non-private members.

$$E(\text{JavaScript}) = 0$$

Final metric value:

$$AE(\text{JavaScript}) = 0.2 \cdot V(\text{JavaScript}) + 0.25 \cdot M(\text{JavaScript}) + 0.25 \cdot I(\text{JavaScript}) +$$
$$= 0.2 \cdot \frac{2}{7} + 0.25 \cdot 1$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

JavaScript supports only single inheritance in terms of object delegation: each object has at most one internal $[[\text{Prototype}]]$ reference, and class-based syntax (extends) likewise allows only one direct superclass. Although mixin patterns can simulate multiple inheritance, they are not formal language-level inheritance mechanisms defined in the specification.

$$H_s(\text{JavaScript}) = 1$$

2) Linearization Determinism Score $L_d(L)$

JavaScript employs a prototype chain lookup algorithm, where property access is resolved deterministically by traversing the $[[\text{Prototype}]]$ chain in a strictly defined order until a matching property is found or the chain terminates. This lookup procedure is formally specified in ECMAScript (e.g., via the Get

and $[[\mathrm{Get}]]$ internal methods), ensuring a fully deterministic and unambiguous resolution order. Since JavaScript supports only single inheritance (one prototype link per object), there is no need for complex linearization strategies as in multiple inheritance systems, and the resolution order is uniquely defined by the chain structure.

$$L_d(\text{JavaScript}) = 1$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 0$: the language does not provide final classes or methods. The inheritance via the $[[\mathrm{Prototype}]]$ chain remains dynamically extensible.

$p_2 = 0$: ECMAScript lacks sealed or closed class hierarchies in the sense of restricting subclass sets (although $\mathrm{Object.seal}$ exists, it does not constrain inheritance relations).

$p_3 = 0$: method overriding is implicit and no mandatory $\mathrm{override}$ annotation is specified.

$p_4 = 0$: because all methods are effectively virtual by default due to dynamic dispatch through property lookup.

$p_5 = 0$: since JavaScript does not enforce a formal separation between abstract classes and interfaces at the language level.

$p_6 = 0$: because there are no compile-time rules prohibiting or restricting inheritance extensions, all checks are dynamic.

$$C_p(\text{JavaScript}) = \frac{0+0+0+0+0+0}{6} = 0,$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 0$: the language does not define any mandatory explicit override annotation, method overriding is implicit and unchecked by the specification.

$s_2 = 0$: JavaScript lacks strict method visibility controls at the specification level—while $\#\text{private}$ fields exist, method visibility does not prevent overriding or exposure in inheritance hierarchies.

$s_3 = 1$: ECMAScript explicitly requires invocation of $\text{super}()$ in derived class constructors before accessing this, enforcing controlled superclass initialization semantics.

$s_4 = 0$: the language specification does not impose variance restrictions or static type constraints ensuring type-safe overriding.

$s_5 = 0$: all methods are effectively overridable by default, with no built-in mechanism for non-overridable (final) methods.

$s_6 = 0$: there are no compile-time override consistency checks in the dynamically typed specification. Therefore, the metric evaluates to

$$F_b(\text{JavaScript}) = \tfrac{1}{6} \approx 0.167,$$

Final metric value:

$$ISRI(\text{JavaScript}) = 0.2 \cdot H_s(\text{JavaScript}) + 0.25 \cdot L_d(\text{JavaScript}) + 0.3 \cdot C_p(\text{JavaScript}$$
$$= 0.2 \cdot 1 + 0.2$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

JavaScript does not provide dedicated delegation keywords (such as dele-

gate). However, delegation can be emulated through object composition, prototype chaining, or manual forwarding methods, which are idiomatic but not syntactically enforced constructs. The language does not include automatic method forwarding mechanisms—developers must explicitly define wrapper methods or use patterns (e.g., proxies) to delegate calls, which are not equivalent to built-in delegation features tied to class definitions

$$K_{del}(\text{JavaScript}) = 0.5, S_{fwd}(\text{JavaScript}) = 0 \Rightarrow D_{cls}(\text{JavaScript}) = 0.25$$

2) Trait Expressiveness Score $T_{exp}(L)$

JavaScript resolves method name conflicts implicitly through property overwriting based on declaration or assignment order when mixin patterns (e.g., object spreading or prototype augmentation) are used. There are no formal mechanisms for explicit exclusion or aliasing, but neither does the language enforce compile-time errors. JavaScript mixin patterns cannot formally declare requirements (such as required methods or fields) within the language specification, any such constraints are informal and not enforced by the runtime or syntax.

$$R_{conf}(\text{JavaScript}) = 0, O_{mod}(\text{JavaScript}) = 0 \Rightarrow T_{exp}(\text{JavaScript}) = 0$$

3) Interface Composition Capacity $I_{comp}(L)$

The language does not include native interface constructs or formally defined analogues at the specification level. There is no facility for declaring default method implementations, because of absence of interfaces. While JavaScript supports behavioral composition through mixins and object composition, these are not formal interface-based mechanisms and are not specified as such.

$$I_{comp}(\text{JavaScript}) = 0$$

Final metric value:

$$CRFI(\text{JavaScript}) = 0.3 \cdot D_{cls}(\text{JavaScript}) + 0.35 \cdot T_{exp}(\text{JavaScript}) + 0.35 \cdot I_{comp}(\text{Jav}$$
$$= 0.3 \cdot 0.25 + 0.35 \cdot 0 + 0.35 \cdot 0$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

JavaScript allows the creation of objects via object literals and class-based syntax, both of which produce objects governed by reference semantics. All user-definable structured types are reference-based. Primitive types (e.g., Number, String, Boolean) are not user-definable in the specification sense and are excluded from $T_{user}(L)$.

$$S_{id}(\text{JavaScript}) = 1$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

JavaScript provides: constructor-based instantiation via the new operator with functions or classes, object literal construction using , and prototype-based creation via $\mathrm{Object.create}$, which directly specifies the prototype of a new object. Other patterns (e.g., factory functions) are reducible to these primitives and are not considered separate paradigms.

$$P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{JavaScript}) = \tfrac{3}{4} = 0.75$$

3) Meta-level Extensibility Score $S_{meta}(L)$

JavaScript provides: runtime structural modification of objects (e.g., adding, deleting, or redefining properties via $\mathrm{Object.defineProperty}$) and, more importantly, the ability to override fundamental object interaction semantics through Proxy objects, which intercept operations such as property access, assignment, and even object construction. This constitutes modification not only of structure but also of the object interaction protocol itself, corresponding to creation and behavior interception at a meta-level

$$S_{meta}(\text{JavaScript}) = \tfrac{3}{3}$$

Final metric value:

$$ORCI(\text{JavaScript}) = 0.35 \cdot S_{id}(\text{JavaScript}) + 0.3 \cdot S_{cr}(\text{JavaScript}) + 0.35 \cdot S_{meta}(\text{JavaS}$$
$$= 0.35 \cdot 1 + 0.3 \cdot 0.75 + 0.35 \cdot 1 =$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

JavaScript supports single dispatch through its object model, where method invocation is determined by the receiver object and resolved via the prototype chain.

$I_{single} = 1$

However, the language does not natively support multiple dispatch based on the runtime types of multiple arguments.

$I_{multi} = 0$

Similarly, predicate-based dispatch is not a built-in language feature—conditional logic can emulate it, but there is no formal dispatch mechanism based on predicates

in the specification.

$I_{pred} = 0$

$M_{scope}(\text{JavaScript}) = \frac{1}{5} = 0.2$

2) Dispatch Safety Guarantee $S_{disp}(L)$

The specification defines a dynamically typed language with no compile-time verification phase. The method dispatch correctness (e.g., property existence or callable status) is resolved at runtime via operations such as $[[\mathrm{Get}]]$ and $[[\mathrm{Call}]]$, and failures result in runtime exceptions rather than static rejection.

$I_{static} = 0$

The language does not enforce exhaustive dispatch definitions, constructs such as switch statements or property-based dispatch do not require coverage of all possible cases, and missing branches are permitted without specification-level errors. Substituting into the metric,

$I_{exhaust} = 0$

$S_{disp}(\text{JavaScript}) = \frac{0+0}{2} = 0,$

3) Virtualization Optimization Potential $O_{virt}(L)$

JavaScript does not support final or non-virtual methods, as all methods are dynamically dispatched and can be overridden.

$K_{final} = 0$

The language also lacks sealed class hierarchies that restrict subclassing.

$K_{sealed} = 0$

While JavaScript includes static methods in class syntax, these do not provide

static dispatch guarantees or devirtualization hints in the specification sense, as method resolution remains dynamic.

$K_{static} = 0$

$O_{virt}(\text{JavaScript}) = 0$

Final metric value:

$$PDEI(\text{JavaScript}) = 0.35 \cdot M_{scope}(\text{JavaScript}) + 0.35 \cdot S_{disp}(\text{JavaScript}) + 0.3 \cdot O_{virt}(\text{J}$$
$$= 0.35 \cdot 0.2 + 0.35 \cdot 0 + 0.3$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

All standard objects (including arrays and functions) are mutable unless explicitly restricted via mechanisms such as $\mathrm{Object.freeze}$ or by using immutable patterns at the application level. Therefore, mutability does not require explicit annotation, immutability must be enforced explicitly by the programmer.

$M_{def}(\text{JavaScript}) = 0$

2) Static Reference Constraint Density $S_{dens}(L)$

First, constructs that introduce or manipulate references: variable bindings (var, let, const), object property access, array indexing, and function parameter passing, all of which operate on references to objects in the heap via the specification's internal Reference Record model.

$N_{ref}(\text{JavaScript}) = 4$

Second, $N_{constr}(\text{JavaScript})$ counts only those constructs that enforce static

(compile-time) constraints on reference usage. JavaScript, being a dynamically typed language, does not impose compile-time aliasing or mutability constraints.

$$S_{dens}(\text{JavaScript}) = 0$$

3) Concurrency Primitive Safety $P_{conc}(L)$

First, JavaScript employs a single-threaded event loop for the main execution context, which avoids traditional data races by design. However, with the introduction of $\mathrm{SharedArrayBuffer}$ and $\mathrm{Atomics}$, true shared-memory concurrency is possible without static guarantees, and correctness relies on programmer discipline rather than type-system proofs or systematic runtime enforcement.

$S_{race}(\text{JavaScript}) = 0$

Second, JavaScript provides high-level concurrency primitives such as $\mathrm{Promise}$, $\mathrm{async/await}$, and event-driven callbacks, which correspond to structured concurrency (task-based asynchronous computation) rather than actor-style isolation or CSP channels.

$S_{model}(\text{JavaScript}) = 0.5.$

$$P_{conc}(\text{JavaScript}) = \tfrac{0+0.5}{2} = 0.25$$

Final metric value:

$$SCSI(\text{JavaScript}) = 0.35 \cdot M_{def}(\text{JavaScript}) + 0.25 \cdot S_{dens}(\text{JavaScript}) + 0.4 \cdot P_{conc}(\text{Ja}$$
$$= 0.35 \cdot 0 + 0.25 \cdot 0 + 0.4 \cdot 0.$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

JavaScript does not provide first-class language constructs for specifying preconditions, postconditions, or invariants, such checks are implemented manually using conditional statements or via external libraries (e.g., assertion frameworks), which are not part of the core language semantics. Specifically, there are no native syntactic forms analogous to $\mathrm{require}$, ensure, or invariant clauses within class or function definitions.

$$C_{nat}(\text{JavaScript}) = \tfrac{0+0+0}{3} = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

JavaScript does not support static verification or compile-time proof of correctness due to its dynamic type system and absence of a formal static contract or verification phase in the language standard. However, it does provide runtime checking mechanisms, such as dynamic type validation, exception handling (throw), and built-in runtime errors (e.g., $\mathrm{TypeError}$), which enforce certain correctness properties during execution. These mechanisms allow violations of expected behavior to be detected only at runtime rather than prevented statically.

$$V_{enf}(\text{JavaScript}) = 0.5$$

Final metric value:

$$\begin{aligned} FSVI(\text{JavaScript}) &= 0.45 \cdot C_{nat}(\text{JavaScript}) + 0.55 \cdot V_{enf}(\text{JavaScript}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0.5 = 0.275 \end{aligned}$$

4.1.9 Metrics Evaluation for Python

Introduction

Python is a dynamically typed, object-oriented language with a highly reflective runtime and a flexible object model rooted in message passing and late binding. Its design philosophy, as described in the Python Language Reference and further analyzed in works such as, prioritizes simplicity and runtime adaptability over formal static guarantees. Consequently, Python provides rich object-oriented expressiveness but relatively weak formalization of type safety, encapsulation enforcement, and verification. In this section, the previously defined metrics are computed for Python based strictly on its language specification and formally documented semantics.

1) Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

In the case of Python, nominal typing is clearly supported through its class system, where subtype relationships are explicitly declared via inheritance [57].

$I_{nom}(\text{Python}) = 1$

Structural typing is also supported through the introduction of *Protocols* [57], which define subtyping based on the presence of methods and properties rather than explicit inheritance.

$I_{struct}(\text{Python}) = 1$.

$S_{nom}(\text{Python}) = 1$

2) Variance Formalization Score $S_{var}(L)$

In Python, variance is supported in the type hinting system [58] through the use of generic type variables (e.g., $\mathrm{TypeVar}$) with explicit annotations such as $\mathrm{covariant}{=}\mathrm{True}$ and $\mathrm{contravariant}{=}\mathrm{True}$, hence co-, contra-, and invariance can be directly declared.

However, regarding soundness, Python's type system is not enforced at runtime and relies on external static type checkers (e.g., mypy), and the official specification does not provide a fully formalized, proven type-safe variance model. It imposes practical restrictions to mitigate unsoundness.

$$V_{exp}(\text{Python}) = 1, V_{sound}(\text{Python}) = 0.5$$
$$S_{var}(\text{Python}) = 0.75$$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, Python supports explicit subtyping without inheritance via structural typing mechanisms such as $\mathrm{typing.Protocol}$, allowing a class to be considered a subtype based on method signatures rather than inheritance hierarchy.

$I_{dec}(\text{Python}) = 1$

Second, subtyping rules in Python's static type system are defined independently of inheritance semantics: nominal inheritance (class-based) and structural subtyping (protocol-based) coexist, and type compatibility is determined by type checkers without requiring subclass relationships.

$I_{ind}(\text{Python}) = 1$

Third, although the distinction between inheritance and subtyping is clearly articulated in the typing PEPs, it is not fully formalized in a rigorous mathematical specification within the core language definition, remaining partly informal and tool-dependent. Thus, substituting these values into the formula yields.

$I_{form}(\text{Python}) = 0.5$

$S_{rel}(\text{Python}) = \frac{1+1+0.5}{3} = \frac{2.5}{3} \approx 0.83,$

Final metric value:

$$\begin{aligned} TSSI(\text{Python}) &= 0.3 \cdot S_{nom}(\text{Python}) + 0.3 \cdot S_{var}(\text{Python}) + 0.4 \cdot S_{rel}(\text{Python}) \\ &= 0.3 \cdot 1 + 0.3 \cdot 0.75 + 0.4 \cdot \frac{5}{6} \\ &\approx 0.8583 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

In Python, access control is not enforced through strict modifiers but follows a convention-based model as described in the language reference:

- public members (default)
- protected members indicated by a single leading underscore (e.g., _x)
- private members using name mangling with double leading underscores (e.g., __x)

Although these mechanisms do not enforce strict encapsulation at the compiler or runtime level, they are semantically distinct and consistently recognized by the specification and tooling.

$$n_{levels}(\text{Python}) = 3, n_{levels}(\text{C\#}) = 7$$
$$V(\text{Python}) = 0.\frac{3}{7}$$

2) Module Isolation Strength $M(L)$

In Python, the primary modularization construct is the module itself (i.e., a file), along with packages that group modules hierarchically.

$b_{total}(\text{Python}) = 2$ (modules and packages)

However, Python does not enforce strict export control: although the special variable $\_\_\text{all}\_\_$ can define an explicit export list, it only affects wildcard imports ($\text{from module import } *$) and does not prevent direct access to non-exported attributes. Additionally, all names remain accessible via standard attribute lookup, reflecting the specification's philosophy of open access rather than enforced encapsulation.

$b_{strict}(\text{Python}) = 0$

$M(\text{Python}) = 0.5$

3) Interface Abstraction Capacity $I(L)$

In Python, abstraction is supported through:

- abstract base classes provided by the abc module
- structural *Protocols* introduced in PEP 544, which define behavior independently of inheritance.

$n_{abstraction_constructs}(\text{Python}) = 2$

The primary object-defining construct in Python is the class (functions are not object-defining in the same structural sense).

$n_{object}(\text{Python}) = 1.$

$I(\text{Python}) = \frac{2}{1} = 2 \Rightarrow \text{normalized} \approx 1$

4) Encapsulation Enforcement Coefficient $E(L)$

In Python, encapsulation is not strictly enforced: all object attributes are accessible via standard mechanisms such as $\mathrm{getattr}$, $\mathrm{setattr}$, direct dictionary manipulation through $\_\_\mathrm{dict}\_\_$, and introspection/reflection facilities defined in the language specification. Python also lacks restrictions on reflective capabilities and does not provide a type system that enforces access boundaries at runtime. Therefore, encapsulation violations are not prevented, and the language explicitly permits unrestricted access to object internals.

$$E(\text{Python}) = 0.2$$

Final metric value:

$$AE(\text{Python}) = 0.2 \cdot V(\text{Python}) + 0.25 \cdot M(\text{Python}) + 0.25 \cdot I(\text{Python}) + 0.3 \cdot E(\text{Pyth}$$
$$= 0.2 \cdot \frac{3}{7} + 0.25 \cdot 0.5 + 0.25 \cdot 1 + 0.3 \cdot$$
$$\approx 0.5$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

In Python, the class definition syntax explicitly allows multiple inheritance, i.e., a class can inherit from an arbitrary number of base classes (e.g., $\mathrm{class\ C(A,\ B,\ ...)}$), implying that $M(\text{Python}) > 1$. Since there is no fixed upper bound in the specification, Python effectively reaches the maximal considered value in comparative analysis.

$$M(\text{Python}) = M_{max} \Rightarrow H_s(\text{Python}) = 1 - \frac{M(\text{Python})-1}{M_{max}-1} = 0$$

2) Linearization Determinism Score $L_d(L)$

In Python, the MRO is explicitly defined in the language specification via the C3 linearization algorithm, which ensures a consistent and monotonic resolution order for multiple inheritance hierarchies. This algorithm is formally described and guarantees determinism across all compliant implementations, eliminating ambiguity in method lookup. Since Python fully specifies its linearization strategy and enforces it uniformly, it satisfies the condition for a formally defined and deterministic MRO.

$$L_d(\text{Python}) = 1$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 0$ because, although $\mathrm{typing.final}$ exists, it is not enforced by the core language semantics and remains advisory for static type checkers only.

$p_2 = 0$ since Python does not provide sealed or closed class hierarchies.

$p_3 = 0$ because method overriding does not require an explicit $\mathrm{override}$ annotation.

$p_4 = 0$ as all methods are effectively virtual by default due to dynamic dispatch.

$p_5 = 1$ since Python formally distinguishes abstract base classes via the abc module, enforcing abstract/interface separation constraints at instantiation time.

$p_6 = 0$ because there are no compile-time restrictions on inheritance in the dynamically typed core language.

$$C_p(\text{Python}) = \tfrac{0+0+0+0+1+0}{6} \approx 0.17,$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 0$ since Python does not require a mandatory $\mathrm{override}$ annotation, making accidental overriding possible.

$s_2 = 0$ because visibility control is limited to conventions (e.g., single or double underscore name mangling) rather than strict access modifiers enforced by the language.

$s_3 = 0$ as calls to $\mathrm{super}()$ are not required and method resolution order permits implicit behavior $s_4 = 1$ since the static typing system defines variance rules (e.g., invariance of generics unless specified otherwise) that are enforced by type checkers.

$s_5 = 0$ because all methods are virtual and overridable by default.

$s_6 = 0$ as there are no compile-time guarantees in the language itself, with consistency checks delegated to optional external type checkers. Therefore,

$$F_b(\text{Python}) = \tfrac{0+0+0+1+0+0}{6} \approx 0.17,$$

Final metric value:

$$\begin{aligned} \mathit{ISRI}(\text{Python}) &= 0.2 \cdot H_s(\text{Python}) + 0.25 \cdot L_d(\text{Python}) + 0.3 \cdot C_p(\text{Python}) + 0.25 \cdot F_b \\ &= 0.2 \cdot 0 + 0.25 \cdot 1 + 0.3 \cdot \frac{1}{6} + \end{aligned}$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

In Python, the language specification does not define dedicated delegation keywords or constructs. The delegation is typically implemented manually using

composition and forwarding methods or via dynamic features such as $\_\_\text{getattr}\_\_$ and $\_\_\text{getattribute}\_\_$, which allow an object to redirect attribute access to another object. Since this approach relies on user-defined patterns rather than built-in syntax, delegation is considered emulated via adapters. Python does not provide automatic delegation mechanisms that eliminate boilerplate forwarding code at the language level, even with dynamic attribute hooks, explicit implementation is required.

$$K_{del}(\text{Python}) = 0.5, S_{fwd}(\text{Python}) = 0 \Rightarrow D_{cls}(\text{Python}) = 0.25$$

2) Trait Expressiveness Score $T_{exp}(L)$

In Python, mixins are implemented using multiple inheritance, and name conflict resolution is governed by the formally specified method resolution order (C3 linearization), which resolves conflicts deterministically based on declaration order rather than explicit exclusion or aliasing mechanisms. Python does not provide a dedicated language-level construct for declaring required methods or properties within mixins themselves.

$$R_{conf}(\text{Python}) = 0.5, O_{conf}(\text{Python}) = 0 \Rightarrow T_{exp} = 0.25$$

3) Interface Composition Capacity $I_{comp}(L)$

In Python, there is no dedicated *interface* construct in the core language specification. However, abstract base classes (ABCs) and *Protocols* (PEP 544) serve as functional analogues. A class may inherit from multiple abstract base classes or conform to multiple protocols without restriction. Furthermore, abstract base classes in Python may include concrete method implementations alongside

abstract methods, enabling default behavior reuse, while protocols may also define method bodies.

$$N_{impl}(\text{Python}) = 1, D_{def}(\text{Python}) = 1 \Rightarrow I_{comp} = 1$$

Final metric value:

$$\begin{aligned} CRFI(\text{Python}) &= 0.3 \cdot D_{cls}(\text{Python}) + 0.35 \cdot T_{exp}(\text{Python}) + 0.35 \cdot I_{comp}(\text{Python}) \\ &= 0.3 \cdot 0.25 + 0.35 \cdot 0.25 + 0.35 \cdot 1 \\ &= 0.5125 \end{aligned}$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

In Python, the language specification defines a uniform object model in which all user-defined types (i.e., classes) produce objects with identity, accessible via operations such as $\mathrm{id}()$ and compared using the is operator. There are no alternative user-definable value-type categories with copy semantics distinct from reference behavior, even immutable types preserve identity at the object level. All user-defined types are reference types:

$$S_{id}(\text{Python}) = 1$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

The object creation is primarily class-based via constructor invocation (e.g., calling a class object, which internally triggers $\_\_\mathrm{new}\_\_$ and $\_\_\mathrm{init}\_\_$), and literal-based construction for built-in container types (e.g., lists, dictionaries,

sets), both formally defined in the language. Additionally, Python supports factory-based instantiation patterns through first-class functions and metaclass customization, but these are extensions of the class-based model rather than distinct paradigms such as prototype cloning or actor spawning.

$$P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{Python}) = \frac{2}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

Python provides extensive meta-programming capabilities: it supports full introspection (e.g., via $\mathrm{dir}()$, $\mathrm{type}()$, reflection APIs), runtime structural modification (e.g., dynamic attribute addition, modification of $\_\_\mathrm{dict}\_\_$), and, critically, allows overriding the object creation protocol through metaobjects such as metaclasses and customization hooks like $\_\_\mathrm{new}\_\_$, $\_\_\mathrm{init}\_\_$, and $\_\_\mathrm{call}\_\_$.

$$S_{meta}(\text{Python}) = \frac{3}{3}$$

Final metric value:

$$\begin{aligned} ORCI(\text{Python}) &= 0.35 \cdot S_{id}(\text{Python}) + 0.3 \cdot S_{cr}(\text{Python}) + 0.35 \cdot S_{meta}(\text{Python}) \\ &= 0.35 \cdot 1 + 0.3 \cdot \frac{2}{4} + 0.35 \cdot 1 \\ &= 0.85 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

In Python, single dispatch is natively supported through method invocation on the receiver object, as defined by the object model and method resolution order.

$I_{single}(\text{Python}) = 1$

Multiple dispatch is not natively supported in the core language, although libraries provide limited function overloading based on a single argument, full argument-based multiple dispatch across all parameters is not part of the specification.

$I_{multi}(\text{Python}) = 0$

Predicate-based dispatch is also not a built-in language feature, as conditional branching must be expressed explicitly in code rather than declaratively in dispatch rules.

$I_{pred}(\text{Python}) = 0$

$M_{scope}(\text{Python}) = \frac{1}{5} = 0.2$

2) Dispatch Safety Guarantee $S_{disp}(L)$

First, Python does not provide compile-time guarantees of dispatch correctness: method resolution and function dispatch are determined at runtime via dynamic typing and late binding, while static type checkers (e.g., mypy) remain external and non-mandatory.

$I_{static}(\text{Python}) = 0$

Second, the language does not enforce exhaustiveness of dispatch definitions, even with structural pattern matching (match statement, PEP 634), exhaustiveness checking is not required or guaranteed by the specification.

$I_{exhaust}(\text{Python}) = 0$

$S_{disp}(\text{Python}) = \frac{0+0}{2} = 0,$

3) Virtualization Optimization Potential $O_{virt}(L)$

In Python, the specification does not provide enforced final or non-virtual methods. All methods are dynamically dispatched and overrideable, and annotations such as final are not enforced at runtime.

$K_{final}(\text{Python}) = 0$

Similarly, Python does not support sealed class hierarchies that restrict subclassing.

$K_{sealed}(\text{Python}) = 0$

Although Python includes $\mathrm{@staticmethod}$, this construct does not serve as a static dispatch optimization hint but rather defines methods without implicit receiver binding, and does not eliminate dynamic lookup semantics.

$K_{static}(\text{Python}) = 0$

$O_{virt}(\text{Python}) = 0$

Final metric value:

$$\begin{aligned} PDEI(\text{Python}) &= 0.35 \cdot M_{scope}(\text{Python}) + 0.35 \cdot S_{disp}(\text{Python}) + 0.3 \cdot O_{virt}(\text{Python}) \\ &= 0.35 \cdot 0.2 + 0.35 \cdot 0 + 0.3 \cdot 0 \\ &= 0.07 \end{aligned}$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

In Python, according to its language specification and data model, objects are not uniformly immutable by default: core built-in types such as integers, floats, strings, and tuples are immutable, whereas lists, dictionaries, and sets are

mutable without requiring any explicit annotation. The language does not enforce immutability as a default semantic constraint, nor does it require developers to explicitly declare objects as mutable, mutability is an inherent property of each type and is directly accessible without syntactic modifiers.

$$M_{def}(\text{Python}) = 0$$

2) Static Reference Constraint Density $S_{dens}(L)$

The type system is dynamically enforced and does not impose compile-time constraints on aliasing, mutability, or reference usage. There are no native syntactic constructs analogous to const qualifiers, borrow checking, or ownership types that restrict references statically. Although optional type hints exist, they are not enforced at compile time by the language itself and therefore do not qualify as static constraints under the formal rules.

$$S_{dens}(\text{Python}) = 0$$

3) Concurrency Primitive Safety $P_{conc}(L)$

First, Python does not provide compile-time guarantees of data race freedom in its type system. It relies primarily on programmer discipline when using shared-memory concurrency (e.g., via the $\mathrm{threading}$ module). Although the Global Interpreter Lock (GIL) in CPython restricts true parallel execution of bytecode, it does not eliminate logical data races on shared mutable state, nor is it a formal runtime race detection mechanism as defined by the metric.

$S_{race}(\text{Python}) = 0$

Second, Python provides structured concurrency abstractions such as asyn-

cio with tasks, futures, and event loops, which correspond to the "structured concurrency" category rather than purely low-level primitives. However, it does not natively enforce higher-level models like Actors or CSP as first-class language constructs.

$S_{model}(\text{Python}) = 0.5$

$P_{conc}(\text{Python}) = 0.25$

Final metric value:

$$SCSI(\text{Python}) = 0.35 \cdot M_{def}(\text{Python}) + 0.25 \cdot S_{dens}(\text{Python}) + 0.4 \cdot P_{conc}(\text{Python})$$
$$= 0.35 \cdot 0 + 0.25 \cdot 0 + 0.4 \cdot 0.25 = 0.1$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

First, Python does not include native syntax or semantics for preconditions, while developers may express them using assert statements or external libraries (e.g., icontract, deal), these are not specified as formal precondition constructs in the language definition.

$n_{pre}(\text{Python}) = 0$

Second, Python similarly lacks native postcondition constructs, no built-in mechanism enforces conditions on function outputs beyond general assertions or external tooling.

$n_{post}(\text{Python}) = 0$

Third, Python does not provide first-class invariant constructs at the language level, such behavior must be manually encoded or delegated to third-party

libraries.

$$n_{inv}(\text{Python}) = 0$$

$$C_{nat}(\text{Python}) = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

Python does not provide static verification or compile-time proof mechanisms within its standard language definition, type hints are optional and not enforced by the interpreter, serving only as annotations for external static analyzers. However, Python does support runtime checking through constructs such as assert statements and dynamic type checking during execution, which can enforce certain correctness properties at runtime.

$$V_{enf}(\text{Python}) = 0.5$$

Final metric value:

$$\begin{aligned} FSVI(\text{Python}) &= 0.45 \cdot C_{nat}(\text{Python}) + 0.55 \cdot V_{enf}(\text{Python}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0.5 = 0.275 \end{aligned}$$

4.1.10 Metrics Evaluation for Ruby

Introduction

Ruby is a dynamically typed, purely object-oriented programming language with a highly reflective metaobject protocol and a strong emphasis on programmer flexibility. According to Flanagan and Matsumoto, “Ruby is a pure object-oriented language: all data in Ruby are objects” [59]. However, this flexibility comes at

the cost of formal guarantees in several aspects of object modeling, particularly in type safety, verification, and concurrency. In this subsection, we systematically evaluate Ruby using the previously defined metrics, grounding each numerical assignment in the formal language specification and authoritative literature such as [59].

1) Type System and Subtyping Model

Ruby employs a dynamically typed system without a formal static type specification. Subtyping is implicit and primarily behavioral (duck typing), rather than formally defined.

1) Nominal–Structural Typing Score $S_{nom}(L)$

Ruby is a dynamically typed, object-oriented language that employs *duck typing*, meaning that type compatibility is determined by the presence of methods rather than explicit type declarations. However, this behavior is not formalized as structural subtyping with explicit typing judgments or inference rules in the specification.

$I_{struct}(\text{Ruby}) = 0$

At the same time, Ruby supports nominal typing through its class-based object model, where subtyping relationships are explicitly declared via inheritance (e.g., using $<$ for subclassing), and method lookup follows the class hierarchy, which satisfies the condition for nominal typing.

$I_{nom}(\text{Ruby}) = 1$

$S_{nom}(\text{Ruby}) = \frac{1+0}{2} = 0.5$

2) Variance Formalization Score $S_{var}(L)$

Ruby does not natively support parametric polymorphism (generics) in its core language specification. Although recent extensions such as RBS and type checkers (e.g., Sorbet) introduce generic-like constructs, these are external and not part of the official language semantics.

$V_{exp}(\text{Ruby}) = 0$

The variance (covariance/contravariance) is not formally defined within the Ruby language specification and there are no typing judgments or proofs ensuring type safety for such mechanisms, the soundness component is also absent.

$V_{sound}(\text{Ruby}) = 0$

$S_{var}(\text{Ruby}) = 0$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, Ruby does not provide explicit constructs for subtyping independent of inheritance (e.g., no interfaces or type declarations distinct from class hierarchies), and type compatibility is determined dynamically via method availability rather than formally declared subtype relations.

$I_{dec}(\text{Ruby}) = 0$

Second, subtyping is not defined as an independent formal relation in the specification but is effectively derived from class inheritance and mixin inclusion semantics.

$I_{ind}(\text{Ruby}) = 0$

Third, the Ruby specification does not formally distinguish inheritance and subtyping as separate relations with independent typing judgments.

$I_{form}(\text{Ruby}) = 0.$

$$S_{rel}(\text{Ruby}) = \frac{0+0+0}{3} = 0$$

Final metric value:

$$\begin{aligned} TSSI(\text{Ruby}) &= 0.3 \cdot S_{nom}(\text{Ruby}) + 0.3 \cdot S_{var}(\text{Ruby}) + 0.4 \cdot S_{rel}(\text{Ruby}) \\ &= 0.3 \cdot 0.5 + 0.3 \cdot 0 + 0.4 \cdot 0 = 0.15 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

Ruby provides three semantically distinct visibility levels for methods: public, protected, and private [59].

$$n_{levels}(\text{C\#}) = 7 \Rightarrow V(\text{Ruby}) = \frac{3}{7}$$

2) Module Isolation Strength $M(L)$

Ruby provides modularization through class and module constructs, which define namespaces and method lookup scopes.

$b_{total}(\text{Ruby}) = 2$

However, Ruby does not support explicit export control lists or enforced visibility at the module or file boundary level, and all constants within a module or class are accessible via namespace qualification.

$b_{strict}(\text{Ruby}) = 0$

$$M(\text{Ruby}) = 0$$

3) Interface Abstraction Capacity $I(L)$

Ruby does not provide dedicated interface or protocol constructs. It relies

on implicit behavioral abstraction via duck typing and mixins (modules), where module can define method signatures but also includes implementations, thus not serving as a purely abstract specification mechanism. Therefore, only one abstraction-related construct (module) can be partially attributed to behavioral abstraction.

$n_{abstraction\_constructs}(\text{Ruby}) = 1$

The language defines two primary object-defining constructs, class and module, both of which can produce objects or object-like namespaces.

$n_{object}(\text{Ruby}) = 2$

$$I(\text{Ruby}) = \frac{1}{2} = 0.5$$

4) Encapsulation Enforcement Coefficient $E(L)$

Ruby provides reflection and metaprogramming facilities (e.g., send, instance_eval, class_eval) that allow invocation of private methods and modification of object internals regardless of declared visibility, effectively bypassing encapsulation constraints. Additionally, instance variables can be accessed and modified dynamically without compile-time checks, and method visibility is not strictly enforced at the runtime level when reflective mechanisms are used. Since these capabilities are part of the core language semantics and are not restricted by a sound type system, Ruby permits encapsulation violations.

$$E(\text{Ruby}) = 0$$

Final metric value:

$$\begin{aligned} AE(\text{Ruby}) &= 0.2 \cdot V(\text{Ruby}) + 0.25 \cdot M(\text{Ruby}) + 0.25 \cdot I(\text{Ruby}) + 0.3 \cdot E(\text{Ruby}) \\ &= 0.2 \cdot \frac{3}{7} + 0.25 \cdot 0 + 0.25 \cdot 0.5 + 0.3 \cdot 0 \\ &\approx 0.2107 \end{aligned}$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

Ruby enforces single inheritance for classes, meaning each class can have exactly one direct superclass (specified using the $<$ syntax), while additional code reuse is achieved via mixins (module inclusion) rather than multiple inheritance. The formal inheritance hierarchy remains strictly single-parent.

$$H_s(\text{Ruby}) = 1$$

2) Linearization Determinism Score $L_d(L)$

Ruby specifies a well-defined and deterministic method lookup algorithm based on a linearized ancestor chain, where the search order follows the class, included modules (in reverse order of inclusion), superclass, and so on up to $\mathrm{BasicObject}$. This linearization is explicitly defined in the language semantics and consistently applied across implementations, ensuring predictable method dispatch even in the presence of mixins [59].

$$L_d(\text{Ruby}) = 1$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 0$: Ruby does not support final classes or methods.

$p_2 = 0$: It lacks sealed or closed hierarchies.

$p_3 = 0$: There is no requirement for explicit override annotations.

$p_4 = 0$: Methods are virtual by default.

$p_5 = 0$: The language does not enforce abstract/interface separation constraints due to the absence of formal interfaces.

$p_6 = 0$: The language does not impose compile-time inheritance restrictions, as classes can be freely extended and reopened at runtime.

$$C_p(\text{Ruby}) = \frac{0+0+0+0+0+0}{6} = 0$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 0$: Ruby does not require explicit override annotations.

$s_2 = 1$: However, it provides method visibility modifiers (public, protected, private), which partially restrict unintended access and override scope.

$s_3 = 0$: The language does not enforce mandatory superclass invocation.

$s_4 = 0$: The language lacks formal variance constraints due to the absence of a static type system.

$s_5 = 0$: Methods are overridable by default.

$s_6 = 0$: There are no compile-time checks for override consistency.

$$F_b(\text{Ruby}) = \frac{0+1+0+0+0+0}{6} = \frac{1}{6} \approx 0.17$$

Final metric value:

$$\begin{aligned} \mathit{ISRI}(\text{Ruby}) &= 0.2 \cdot H_s(\text{Ruby}) + 0.25 \cdot L_d(\text{Ruby}) + 0.3 \cdot C_p(\text{Ruby}) + 0.25 \cdot F_b(\text{Ruby}) \\ &= 0.2 \cdot 1 + 0.25 \cdot 1 + 0.3 \cdot 0 + 0.25 \cdot \frac{1}{6} \\ &\approx 0.4917 \end{aligned}$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

Ruby does not provide built-in delegation keywords at the core language syntax level. However, delegation can be idiomatically achieved via standard library facilities such as $\mathrm{Forwardable}$ and metaprogramming constructs, which emulate delegation without requiring explicit adapter boilerplate for each method. Furthermore, Ruby supports automatic method forwarding through dynamic features such as $\mathrm{method\_missing}$ and utilities like $\mathrm{SimpleDelegator}$, which allow forwarding of method calls to an internal object with minimal or no boilerplate, satisfying the condition for automatic delegation.

$$K_{del}(\text{Ruby}) = 0.5, S_{fwd}(\text{Ruby}) = 0 \Rightarrow D_{cls}(\text{Ruby}) = 0.25$$

2) Trait Expressiveness Score $T_{exp}(L)$

The method name conflicts during mixin composition are resolved deterministically by linearization order (i.e., the last included module takes precedence in the method lookup chain), without requiring explicit exclusion or aliasing constructs at the language level. Additionally, modules do not provide formal mechanisms to declare required methods or constraints on the including class within the language specification (no native "requires" clauses), meaning mixins must rely on implicit assumptions or runtime errors

$$R_{conf}(\text{Ruby}) = 0.5, O_{mod}(\text{Ruby}) = 0 \Rightarrow T_{exp}(\text{Ruby}) = 0.25$$

3) Interface Composition Capacity $I_{comp}(L)$

Ruby does not define interfaces as separate constructs. However, modules

serve as compositional units that can be included in classes without restriction on their number, effectively acting as interface-like mechanisms. Furthermore, unlike traditional interfaces, Ruby modules can contain full method implementations, allowing them to act as reusable behavioral units rather than pure type specifications

$$N_{iml}(\text{Ruby}) = 1, D_{def}(\text{Ruby}) = 1 \Rightarrow I_{comp}(\text{Ruby}) = 1$$

Final metric value:

$$\begin{aligned} CRFI(\text{Ruby}) &= 0.3 \cdot D_{cls}(\text{Ruby}) + 0.35 \cdot T_{exp}(\text{Ruby}) + 0.35 \cdot I_{comp}(\text{Ruby}) \\ &= 0.3 \cdot 0.25 + 0.35 \cdot 0.25 + 0.35 \cdot 1 \\ &= 0.5125 \end{aligned}$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

Ruby follows a uniform object model in which all user-defined entities are objects created via class or module, and all variables store references to these objects rather than values

$$S_{id}(\text{Ruby}) = 1$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

Ruby primarily supports class-based instantiation via the new method (which internally invokes $\mathrm{allocate}$ and $\mathrm{initialize}$), and also provides literal-based construction for built-in object types such as arrays, hashes, strings, and numeric values,

which constitute a second paradigm of object creation. The language supports advanced metaprogramming facilities such as Class.new, define_method, and dynamic evaluation via eval, which enable runtime generation of classes and objects. Ruby does not natively support prototype-based cloning as a primary paradigm (despite clone and dup, which operate on existing objects rather than serving as independent instantiation models), nor actor-based spawning in its core specification.

$$P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{Ruby}) = \tfrac{3}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

Ruby provides full reflective capabilities, including introspection (e.g., methods, instance_variables), structural modification (e.g., define_method, class_eval, module_eval), and, critically, the ability to override the object creation protocol via hooks such as allocate, initialize, and dynamic class construction with Class.new. These features allow programmers to intercept and redefine how objects are instantiated and structured at runtime, satisfying the highest level of intercession.

$$S_{meta}(\text{Ruby}) = \tfrac{3}{3}$$

Final metric value:

$$\begin{aligned} ORCI(\text{Ruby}) &= 0.35 \cdot S_{id}(\text{Ruby}) + 0.3 \cdot S_{cr}(\text{Ruby}) + 0.35 \cdot S_{meta}(\text{Ruby}) \\ &= 0.35 \cdot 1 + 0.3 \cdot \frac{3}{4} + 0.35 \cdot 1 \\ &= 0.925 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

Ruby natively supports single dispatch, where method selection is based on the receiver object and resolved through a deterministic method lookup chain.

$I_{single}(\text{Ruby}) = 1$

However, Ruby does not provide native multiple dispatch based on the dynamic types of multiple arguments.

$I_{multi}(\text{Ruby}) = 0$

Additionally, predicate-based dispatch is not part of the formal method dispatch mechanism in the specification. Although conditional logic can be implemented manually within methods, it is not integrated into the dispatch system itself.

$I_{pred}(\text{Ruby}) = 0$

$M_{scope}(\text{Ruby}) = \frac{1}{5} = 0.2$

2) Dispatch Safety Guarantee $S_{disp}(L)$

First, Ruby is dynamically typed and does not perform compile-time checking of method existence or dispatch correctness, method resolution occurs at runtime and may raise exceptions such as NoMethodError.

$I_{static}(\text{Ruby}) = 0$

Second, Ruby does not enforce exhaustive dispatch definitions (e.g., no requirement to cover all possible cases in polymorphic behavior or pattern matching at the specification level).

$I_{exhaust}(\text{Ruby}) = 0$

$$S_{disp}(\text{Ruby}) = \frac{0+0}{2} = 0$$

3) Virtualization Optimization Potential $O_{virt}(L)$

Ruby does not support final or non-virtual methods, as all methods are dynamically dispatched and can be overridden at runtime.

$$K_{final}(\text{Ruby}) = 0$$

Similarly, Ruby does not provide sealed class hierarchies or any mechanism to restrict subclassing.

$$K_{sealed}(\text{Ruby}) = 0$$

Furthermore, the language lacks static dispatch constructs or annotations (such as static methods or compiler hints for devirtualization), since all method calls are resolved dynamically.

$$K_{static}(\text{Ruby}) = 0$$

$$O_{virt}(\text{Ruby}) = 0$$

Final metric value:

$$\begin{aligned} PDEI(\text{Ruby}) &= 0.35 \cdot M_{scope}(\text{Ruby}) + 0.35 \cdot S_{disp}(\text{Ruby}) + 0.3 \cdot O_{virt}(\text{Ruby}) \\ &= 0.35 \cdot 0.2 + 0.35 \cdot 0 + 0.3 \cdot 0 \\ &= 0.07 \end{aligned}$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

In Ruby, all values are objects, and most objects are *mutable by default*, instances of classes such as String, Array, and Hash can be modified in place

using methods like «, push, or direct element assignment. Although Ruby provides mechanisms to enforce immutability, such immutability must be explicitly invoked and is not the default behavior. Furthermore, there is no requirement for explicit annotation to enable mutability, as mutation is inherently permitted unless restricted.

$$M_{def}(\text{Ruby}) = 0$$

2) Static Reference Constraint Density $S_{dens}(L)$

In Ruby, all variables are references to objects, and syntactic constructs include local variables, instance variables (@x), class variables (@@x), global variables ($x), constants, and method parameters.

$N_{ref}(\text{Ruby}) = 6$ as distinct reference-related constructs explicitly represented in the language syntax.

Ruby is a dynamically typed language with no static type system or compile-time enforcement of aliasing, mutability, or reference restrictions (all checks occur at runtime, and features like freeze are dynamic).

$N_{constr}(\text{Ruby}) = 0$

$$S_{dens}(\text{Ruby}) = 0$$

3) Concurrency Primitive Safety $P_{conc}(L)$

First, Ruby does not provide compile-time guarantees of data race freedom, as it lacks a static type system capable of enforcing aliasing or ownership constraints. Moreover, while implementations such as MRI include a Global Interpreter Lock (GIL), this is an implementation detail rather than a language-level guarantee

and does not eliminate race conditions in all cases (e.g., with native extensions or alternative runtimes). There are also no systematic runtime mechanisms that universally detect or prevent data races. Therefore, concurrency safety relies primarily on programmer discipline.

$S_{race}(\text{Ruby}) = 0$

Second, Ruby's core concurrency primitives consist of low-level threads (Thread) and synchronization mechanisms such as Mutex, without built-in high-level models like Actors or CSP channels in the core language (libraries may provide them, but they are not part of the language specification).

$S_{model}(\text{Ruby}) = 0$

$P_{conc}(\text{Ruby}) = 0$

Final metric value:

$$SCSI(\text{Ruby}) = 0.35 \cdot M_{def}(\text{Ruby}) + 0.25 \cdot S_{dens}(\text{Ruby}) + 0.4 \cdot P_{conc}(\text{Ruby})$$
$$= 0.35 \cdot 0 + 0.25 \cdot 0 + 0.4 \cdot 0 = 0$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

Ruby does not include native support for design-by-contract features such as preconditions, postconditions, or invariants. There are no built-in keywords or type-system constructs to declare and enforce such properties at the language level. While developers may simulate preconditions and postconditions using runtime checks (e.g., raise statements) and invariants via method definitions or metaprogramming, these approaches rely on libraries or idiomatic patterns rather

than standardized syntax or semantics defined by the language.

$$C_{nat}(\text{Ruby}) = 0$$

2) Verification Enforcement Level $V_{enf}(L)$

Ruby does not support static verification: it lacks a static type system and does not include compile-time proof mechanisms for correctness properties such as contracts, invariants, or type safety. However, Ruby does support runtime checking through dynamic features, including exception handling (e.g., raise), assertions implemented via standard or third-party libraries, and reflective capabilities that allow validation during execution. These mechanisms enable enforcement of certain correctness properties, but only at runtime rather than compile time.

$$V_{enf}(\text{Ruby}) = 0.5$$

Final metric value:

$$\begin{aligned} FSVI(\text{Ruby}) &= 0.45 \cdot C_{nat}(\text{Ruby}) + 0.55 \cdot V_{enf}(\text{Ruby}) \\ &= 0.45 \cdot 0 + 0.55 \cdot 0.5 \\ &= 0.275 \end{aligned}$$

4.1.11 Metrics Evaluation for Zonnon

Zonnon is a statically-typed, object-oriented language in the Pascal/Modula-2/Oberon lineage, designed with an emphasis on abstraction unification, concurrency, and modular composition [25]. Its object model is centered around four primary constructs—*module*, *object*, *definition*, and *implementation*—which collectively define both structural and behavioral aspects of programs. The language explicitly separates interface specification (*definitions*) from implementation and supports concurrency via active objects and protocol-controlled interactions. In this section, we compute the proposed object-model metrics based strictly on the formal language specification.

Type System and Subtyping Model

1) Nominal–Structural Typing Score $S_{nom}(L)$

Zonnon employs a **nominal type system**. Subtyping is explicitly defined through the implements relation between objects and definitions: "an object implementing a definition is required to implement all of its fields and methods". There is no notion of structural subtyping in the specification.

According to the language specification, Zonnon employs an object model where types (objects) explicitly declare conformance to interfaces ("definitions") via the implements clause, and relationships such as implements and refines are explicitly stated rather than inferred structurally. Hence, it satisfies nominal typing requirements with explicit subtype declarations [25].

Conversely, there is no evidence of structural subtyping based solely on shape compatibility or formal structural typing rules (i.e., no typing judgments allowing implicit compatibility based on member structure).

$$I_{nom}(\text{Zonnon}) = 1, I_{struct}(\text{Zonnon}) = 0$$
$$S_{nom}(\text{Zonnon}) = \frac{1+0}{2} = 0.5$$

2) Variance Formalization Score $S_{var}(L)$

Zonnon does not support parametric polymorphism (generics) or type constructors with variance annotations. It relies on object types, definitions (interfaces), and explicit implementation relations without any mechanism for parameterized type abstraction. Consequently, variance cannot be expressed at the language level. Furthermore, since variance is not part of the type system, there are no formal rules, constraints, or proofs of soundness related to variance in the specification.

$$V_{exp}(\text{Zonnon}) = 0, V_{sound}(\text{Zonnon}) = 0$$
$$S_{var}(\text{Zonnon}) = 0$$

3) Inheritance–Subtyping Separation Score $S_{rel}(L)$

First, Zonnon explicitly supports subtyping via *definitions* (interfaces) that are independent constructs from objects, allowing an object to implement multiple definitions without relying on inheritance hierarchies. This constitutes explicit subtyping without inheritance constructs.

$$I_{dec}(\text{Zonnon}) = 1$$

Second, subtyping semantics are defined through the "implements" relation between objects and definitions, which is orthogonal to object construction and does not derive from inheritance semantics, demonstrating full independence.

$$I_{ind}(\text{Zonnon}) = 1$$

Third, the specification formally distinguishes constructs such as *object*, *definition*, and their relations (e.g., "implements", "refines"), thereby providing an explicit and formal separation between interface-based subtyping and structural composition.

$$I_{form}(\text{Zonnon}) = 1$$

Therefore

$$S_{rel}(\text{Zonnon}) = \frac{1+1+1}{3} = 1$$

Final metric value:

$$\begin{aligned} TSSI(\text{Zonnon}) &= 0.3 \cdot S_{nom}(\text{Zonnon}) + 0.3 \cdot S_{var}(\text{Zonnon}) + 0.4 \cdot S_{rel}(\text{Zonnon}) \\ &= 0.3 \cdot 0.5 + 0.3 \cdot 0 + 0.4 \cdot 1 = 0.55 \end{aligned}$$

Abstraction and Encapsulation Mechanisms

1) Visibility Granularity Index $V(L)$

Zonnon provides explicit visibility modifiers $\mathrm{private}$ and public, where $\mathrm{private}$ restricts access to the local scope of the declaration and public allows visibility across imported program units. Additionally, there exists an implicit module-level encapsulation boundary, since declarations are only accessible if explicitly exported and imported [25]. Thus, the language supports three semantically distinct access scopes: local (private), module/exported (public), and non-exported encapsulated scope.

$$n_{levels}(\text{Zonnon}) = 3, n_{max} = n_{levels}(\text{C\#}) = 7$$
$$V(\text{Zonnon}) = \frac{3}{7}$$

2) Module Isolation Strength $M(L)$

The language defines four primary program units: module, object, definition, and implementation, which together form the basis of program composition and encapsulation. Among these, module, definition, and object enforce strict export control via explicit use of visibility modifiers (e.g., public) and require explicit import declarations for external access, thereby acting as boundaries with enforced interface exposure. In contrast, implementation serves as a reusable code container aggregated into other units and does not independently enforce a strict export boundary.

$$M(\text{Zonnon}) = \tfrac{3}{4} = 0.75$$

3) Interface Abstraction Capacity $I(L)$

The language defines definition as a dedicated abstraction construct representing an interface (a set of method signatures and fields), clearly decoupled from implementation, while implementation provides reusable method bodies, and object serves as the concrete type that implements one or more definitions via the implements clause [25]. Additionally, protocol types describe interaction behavior but are primarily oriented toward concurrency rather than general interface abstraction, and thus are not counted as core abstraction constructs.

$$n_{abstraction\_constructs}(\text{Zonnon}) = 1, n_{object}(\text{Zonnon}) = 2$$
$$I(\text{Zonnon}) = \frac{1}{1} = 1$$

4) Encapsulation Enforcement Coefficient $E(L)$

The specification enforces encapsulation through visibility modifiers (private, public) and requires all access to object state to occur via declared fields or methods, with no indication of unchecked field access or unsafe casts that bypass the type system. Although Zonnon includes a reflection facility, it is defined in a controlled manner based on structured metadata (e.g., XML schema) and does not provide arbitrary runtime mutation or violation of access restrictions. Furthermore, type conversions and interface checks (e.g., is, implements) are explicitly constrained and type-safe.

$$E(\text{Zonnon}) = 1$$

Final metric value:

$$AE(\text{Zonnon}) = 0.2 \cdot V(\text{Zonnon}) + 0.25 \cdot M(\text{Zonnon}) + 0.25 \cdot I(\text{Zonnon}) + 0.3 \cdot E(\text{Z}$$
$$= 0.2 \cdot \frac{3}{7} + 0.25 \cdot 0.75 + 0.25 \cdot 1 + 0.3 \cdot 1 \approx$$

Inheritance Semantics and Hierarchy Management

1) Hierarchy Simplicity Factor $H_s(L)$

Zonnon does not support classical multiple inheritance. Reuse is achieved via implementations and definitions. According to the specification, Zonnon does not support class-based inheritance between object types, objects cannot extend other objects and thus there is no notion of multiple superclass inheritance. Instead, reuse is achieved via implementation aggregation and definition conformance via implements, which are not counted as superclass inheritance.

$$M(\text{Zonnon}) = 0 \Rightarrow H_s(\text{Zonnon}) = 1$$

2) Linearization Determinism Score $L_d(L)$

The language does not support class inheritance hierarchies and therefore does not require a method resolution order (MRO) for multiple inheritance. Instead, behavior composition is achieved through explicit implements relations with definitions and optional aggregation from implementation units, where method bindings are statically defined and unambiguous at compile time [25]. Since there is no inheritance-based method overriding chain requiring linearization, and all method resolutions are explicitly specified and deterministic by construction, the language effectively satisfies the strongest determinism condition.

$$L_d(\text{Zonnon}) = 1$$

3) Constraint Protection Coefficient $C_p(L)$

$p_1 = 0$: Zonnon does not provide explicit final or non-inheritable class/method modifiers.

$p_2 = 0$: Zonnon does not support sealed or closed class hierarchies with explicitly enumerated subclasses.

$p_3 = 0$: There is no mandatory override annotation enforced by the compiler for method redefinition

$p_4 = 1$: Zonnon follows a model where methods are not implicitly virtual unless defined within extensible class contexts, effectively aligning with non-virtual-by-default semantics

$p_5 = 1$: Zonnon enforces a clear distinction between abstract definitions

(via definition modules and abstract classes) and implementations, supporting separation constraints

$p_6 = 1$: Zonnon includes compile-time inheritance restrictions such as controlled visibility and module-based encapsulation that limit illegal extensions.

$$C_p(\text{Zonnon}) = \frac{3}{6} = 0.5$$

4) Fragile Base Mitigation Factor $F_b(L)$

$s_1 = 0$: Zonnon does not require mandatory $\mathrm{override}$ annotations.

$s_2 = 1$: Zonnon enforces strict visibility control through module-based encapsulation and access modifiers, limiting unintended exposure of overridable members.

$s_3 = 0$: Zonnon does not mandate explicit superclass invocation (e.g., enforced super calls).

$s_4 = 1$: Zonnon relies on a statically typed system with type-safe method redefinitions, implying basic variance safety.

$s_5 = 1$: The methods are effectively non-overridable by default unless defined within extensible class contexts, corresponding to opt-in polymorphism.

$s_6 = 1$: The compiler enforces consistency of method signatures during overriding, though without formal contract checking.

$$F_b(\text{Zonnon}) = \frac{4}{6} \approx 0.67$$

Final metric value:

$$ISRI(\text{Zonnon}) = 0.2 \cdot H_s(\text{Zonnon}) + 0.25 \cdot L_d(\text{Zonnon}) + 0.3 \cdot C_p(\text{Zonnon}) + 0.25 \cdot$$
$$= 0.2 \cdot 1 + 0.25 \cdot 1 + 0.3 \cdot 0.5 + 0.25 \cdot$$

Composition and Alternatives to Inheritance

1) Class-Based Delegation Score $D_{cls}(L)$

The language does not provide dedicated syntactic delegation keywords (such as delegate in some languages). However, it enables composition and reuse through implementation aggregation and object composition, which can be used to emulate delegation patterns manually. Zonnon does not include mechanisms for automatic forwarding of method calls (i.e., no language-level support for implicit delegation without boilerplate code).

- Belegation is supported only indirectly $\Rightarrow K_{del} = 0.5$
- All forwarding must be explicitly implemented $\Rightarrow S_{fwd} = 0$

$$D_{cls}(\text{Zonnon}) = 0.25$$

2) Trait Expressiveness Score $T_{exp}(L)$

Zonnon does not provide traits or mixins as first-class language constructs. It supports composition via implementation aggregation and interface conformance via definition. In cases where multiple implementations introduce conflicting method names, the specification requires explicit resolution at the object level, and there is no support for aliasing or exclusion mechanisms, conflicts result in a compilation requirement for disambiguation. Additionally, implementation units do not declare behavioral requirements on consuming objects (unlike traits with

required methods). They are purely containers of reusable code without constraint specification.

$$T_{exp}(\text{Zonnon}) = 0$$

3) Interface Composition Capacity $I_{comp}(L)$

In Zonnon, object types can implement multiple definitions (interfaces) via the implements clause without any specified upper bound, implying unrestricted interface composition [25]. However, definition constructs serve purely as abstract interfaces containing field declarations and method signatures. They do not allow method implementations, as behavior is provided separately via implementation or directly within objects.

$$N_{impl}(\text{Zonnon}) = 1, D_{def}(\text{Zonnon}) = 0$$
$$I_{comp}(\text{Zonnon}) = 0.5$$

Final metric value:

$$CRFI(\text{Zonnon}) = 0.3 \cdot D_{cls}(\text{Zonnon}) + 0.35 \cdot T_{exp}(\text{Zonnon}) + 0.35 \cdot I_{comp}(\text{Zonnon})$$
$$= 0.3 \cdot 0.25 + 0.35 \cdot 0 + 0.35 \cdot 0.5 = 0.25$$

Object Representation and Creation Model

1) Identity Semantics Score $S_{id}(L)$

According to the specification, the primary user-definable types are object and record. Objects in Zonnon can be declared with modifiers ref (reference semantics) or value (value semantics), with value being the default, while record is explicitly defined as a value object type [25]. Since reference semantics are

not enforced uniformly across all user-defined types—only optionally for object types—there is no type category that strictly enforces reference semantics by definition.

Zonnon distinguishes value and reference objects:

$$T_{user} = 2(\text{value, reference})T_{ref} = 1$$

$$S_{id}(\text{Zonnon}) = 0.5$$

2) Instantiation Paradigm Diversity $S_{cr}(L)$

The specification defines instantiation primarily via the new operator, which is used to create instances of object types and dynamically allocated arrays. Additionally, Zonnon supports the instantiation of concurrent entities through activity creation (i.e., spawning activity instances with their own execution thread), representing a distinct actor-like paradigm [25]. However, the language does not support prototype-based cloning or object literal construction for user-defined objects.

$$P_{supported}(\text{C\#}) = 4 \Rightarrow S_{cr}(\text{Zonnon}) = \frac{2}{4}$$

3) Meta-level Extensibility Score $S_{meta}(L)$

The specification defines a reflection mechanism that enables querying program structure and metadata (e.g., via XML schema representations), allowing inspection of types, declarations, and program composition at runtime [25]. However, there is no support for modifying object structures dynamically, nor for altering the instantiation process or introducing metaobject protocols. Thus, Zonnon provides introspection without structural or behavioral intercession.

$I_{level}(\text{JS/Python/Ruby}) = 3 \Rightarrow S_{meta}(\text{Zonnon}) = \frac{1}{3}$

Final metric value:

$$\begin{aligned} ORCI(\text{Zonnon}) &= 0.35 \cdot S_{id}(\text{Zonnon}) + 0.3 \cdot S_{cr}(\text{Zonnon}) + 0.35 \cdot S_{meta}(\text{Zonnon}) \\ &= 0.35 \cdot 0.5 + 0.3 \cdot \frac{2}{4} + 0.35 \cdot \frac{1}{3} \\ &\approx 0.4417 \end{aligned}$$

Polymorphism and Dispatch Mechanisms

1) Dispatch Scope Mechanism $M_{scope}(L)$

Zonnon supports single dispatch, where method invocation is determined by the dynamic type of the receiver object (i.e., receiver-based method binding) [25].

$I_{single} = 1$

However, the language does not support multiple dispatch, as method selection is not based on the runtime types of multiple arguments.

$I_{multi} = 0$

There is no support for predicate-based or condition-based dispatch mechanisms (such as guard-based method selection). The control flow must be expressed explicitly within method bodies.

$I_{pred} = 0$

$M_{scope}(\text{Zonnon}) = \frac{1}{5} = 0.2$

2) Dispatch Safety Guarantee $S_{disp}(L)$

According to the specification, method calls are statically type-checked against definitions (interfaces), and objects must fully implement all declared

methods before instantiation. The method availability is guaranteed at compile time [25]. Furthermore, there is no support for unsafe downcasting or dynamic method lookup that could lead to "method not found" errors during dispatch. Type tests (e.g., is) are explicitly checked and must be handled by the programmer, preventing invalid dispatch paths.

First, Zonnon is a statically typed language with compile-time type checking of method calls and dynamic dispatch resolution constrained by class hierarchies. Incorrect method calls are rejected at compile time.

$I_{static}(\text{Zonnon}) = 1$

Second, the language does not enforce exhaustive dispatch definitions: there is no requirement that all possible variants of a type hierarchy must be covered (e.g., no pattern matching exhaustiveness or sealed hierarchy guarantees), and method overriding is optional rather than complete by construction.

$I_{exhaust}(\text{Zonnon}) = 0$

$S_{disp}(\text{Zonnon}) = \frac{1+0}{2} = 0.5$

3) Virtualization Optimization Potential $O_{virt}(L)$

Zonnon does not support classical method overriding or virtual dispatch. All methods are effectively non-virtual due to the absence of class inheritance and overriding semantics, which corresponds to implicit support for final methods [25].

$K_{final} = 1$

The language also does not define class hierarchies and therefore does not include constructs such as sealed for restricting subclassing.

$K_{sealed} = 0$

Zonnon supports procedures and functions that are statically bound (i.e., not subject to dynamic dispatch), which serves as an inherent static dispatch mechanism.

$K_{static} = 1$

$O_{virt}(\text{Zonnon}) = \frac{2}{3} \approx 0.67$

Final metric value:

$$PDEI(\text{Zonnon}) = 0.35 \cdot M_{scope}(\text{Zonnon}) + 0.35 \cdot S_{disp}(\text{Zonnon}) + 0.3 \cdot O_{virt}(\text{Zonnon})$$
$$= 0.35 \cdot 0.2 + 0.35 \cdot 0.5 + 0.3 \cdot \frac{2}{3}$$
$$= 0.445$$

State Model, Mutability, and Concurrency Guarantees

1) Default Mutability Score $M_{def}(L)$

The object types are mutable by default, allowing modification of their fields unless explicitly restricted by qualifiers such as immutable or by using value semantics (e.g., value objects or records) [25].

$M_{def}(\text{Zonnon}) = 0$

2) Static Reference Constraint Density $S_{dens}(L)$

The language provides reference semantics primarily through ref objects and implicitly via object references, which constitute reference-related constructs alongside qualifiers such as immutable that affect reference behavior [25].

$N_{ref}(\text{Zonnon}) = 2$ (reference objects via ref, and reference qualifiers such as

immutable).

Among these, only immutable imposes a static constraint by restricting mutation across scope boundaries at compile time, while ref itself introduces reference semantics without additional safety guarantees.

First, Zonnon includes syntactic constructs that introduce or manipulate references: object variables declared with var, which denote references to objects, class types, since all class instances are accessed via references, and procedure parameters, which may implicitly pass references depending on their type semantics.

$N_{ref}(\text{Zonnon}) = 3.$

Second, Zonnon includes only constructs that impose static constraints on reference usage: the immutable modifier, which enforces compile-time restrictions on mutation through references, and access modifiers (e.g., protected, public) that constrain reference accessibility at compile time.

$N_{constr}(\text{Zonnon}) = 2$

$S_{dens}(\text{Zonnon}) = \frac{2}{3} \approx 0.67$

3) Concurrency Primitive Safety $P_{conc}(L)$

The language introduces concurrency through activity constructs, which encapsulate independent threads of execution and communicate via well-defined interfaces and synchronization mechanisms, resembling an actor-like model [25]. However, the specification does not provide formal guarantees of data race freedom enforced by the type system. The shared state may still be accessed, and correctness relies on disciplined use of synchronization constructs.

$S_{race}(\text{Zonnon}) = 0$

On the other hand, the active object model represents a higher-level abstraction compared to raw threads and locks, as it structures concurrency around object interaction and implicit synchronization, aligning partially with actor-like paradigms but without strict isolation guarantees. It corresponds to structured/high-level primitives.

$S_{model}(\text{Zonnon}) = 0.5$

$P_{conc}(\text{Zonnon}) = \frac{0+0.5}{2} = 0.5$

Final metric value:

$$\begin{aligned} SCSI(\text{Zonnon}) = 0.35 \cdot M_{def}(\text{Zonnon}) + 0.25 \cdot S_{dens}(\text{Zonnon}) + 0.4 \cdot P_{conc}(\text{Zonnon}) \\ = 0.35 \cdot 0 + 0.25 \cdot \frac{2}{3} + 0.4 \cdot 0.5 \\ \approx 0.3667 \end{aligned}$$

Support for Formal Specification and Verifiability

1) Native Contract Constructs Score $C_{nat}(L)$

$n_{pre}(\text{Zonnon}) = 1$: Zonnon includes explicit require clauses in procedure declarations, which define preconditions and are both syntactically integrated and semantically enforced.

$n_{post}(\text{Zonnon}) = 1$: Zonnonalso provides ensure clauses for postconditions, similarly defined at the language level.

$n_{inv}(\text{Zonnon}) = 0$: Zonnon does not define a dedicated invariant construct (e.g., class invariants) as a first-class syntactic feature. Invariants must be expressed indirectly through assertions or programmer discipline.

$$C_{nat}(\text{Zonnon}) = \tfrac{2}{3} \approx 0.67$$

2) Verification Enforcement Level $V_{enf}(L)$

Zonnon does not provide a static verification system capable of proving correctness properties at compile time. There is no integrated formal verification or proof-based type system. However, the language defines preconditions and postconditions with a well-specified runtime semantics, meaning these conditions can be checked during program execution (e.g., triggering exceptions if violated). Therefore, enforcement exists but is limited to runtime checking rather than compile-time guarantees.

$$V_{enf}(\text{Zonnon}) = 0.5$$

Final metric value:

$$\begin{aligned} FSVI(\text{Zonnon}) &= 0.45 \cdot C_{nat}(\text{Zonnon}) + 0.55 \cdot V_{enf}(\text{Zonnon}) \\ &= 0.45 \cdot \frac{2}{3} + 0.55 \cdot 0.5 = 0.575 \end{aligned}$$

4.2 Final Calculation Results

TABLE 4.1
Comparison of Programming Languages by Metrics

Language	TSSI	AE	ISRI	CRFI	ORCI	PDEI	SCSI	FSVI
C#	0.7833	0.3875	0.9083	0.5	0.7433	0.685	0.25	0.275
Java	0.5	0.3643	0.731	0.425	0.6917	0.545	0.25	0.275
Smalltalk	0	0.0286	0.45	0.225	0.5417	0.07	0	0
C++	0.3	0.3774	0.3907	0.425	0.3417	0.37	0.1667	0.275
C	0	0.0571	0	0.075	0.15	0.1	0.1667	0
Golang	0.6334	0.4321	0.7167	0.25	0.5917	0.245	0.3	0.275
Scala	0.8	0.3381	0.8167	0.6875	0.6917	0.62	0.4	0.275
JavaScript	0.2	0.3071	0.4917	0.075	0.925	0.07	0.1	0.275
Python	0.8583	0.5207	0.3417	0.5125	0.85	0.07	0.1	0.275
Ruby	0.15	0.2107	0.4917	0.5125	0.925	0.07	0	0.275
Zonnon	0.55	0.8232	0.7667	0.25	0.4417	0.445	0.3667	0.575

4.2.1 Comparison summary: C#

The obtained results demonstrate that C# provides one of the most structurally disciplined object-oriented models among mainstream industrial languages, particularly in the areas of inheritance management and type-system safety. The high values of $ISRI(\text{C\#}) \approx 0.9083$ and $TSSI(\text{C\#}) \approx 0.7833$ indicate that the language strongly separates inheritance management concerns from subtype relations while simultaneously maintaining a formally constrained nominal type system with explicit variance annotations. This result is consistent with the design goals of the .NET type system, where generic variance and interface-based abstraction were introduced specifically to improve type safety and behavioral substitutability [29]. In particular, the use of explicit variance modifiers (in/out) and deterministic inheritance semantics reduces ambiguity in polymorphic hierarchies and mitigates

the fragile base class problem through mandatory override rules and sealed constructs. Such characteristics correspond to the observations of Cardelli and Cook that inheritance and subtyping should remain conceptually distinct relations in robust object-oriented systems [1], [8]. The relatively high $ORCI(\text{C\#}) \approx 0.7433$ additionally reflects the flexibility of the runtime object model and the availability of controlled reflective mechanisms through the Common Language Runtime.

At the same time, the lower scores in $AE(\text{C\#}) = 0.3875$, $SCSI(\text{C\#}) = 0.25$, and $FSVI(\text{C\#}) = 0.275$ reveal several structural limitations of the language from the perspective of strict object-oriented correctness and formal verifiability. Despite providing multiple visibility modifiers and interface abstractions, C# does not enforce strong module isolation and allows extensive runtime reflection, which weakens encapsulation guarantees. This observation corresponds to the critique of unrestricted reflective systems discussed by Bracha and Ungar, who note that reflection often compromises abstraction barriers when not separated through dedicated metaobject protocols [31]. Furthermore, mutability-by-default semantics and the absence of ownership-based aliasing restrictions significantly reduce the language's static guarantees regarding concurrency safety, despite the availability of higher-level asynchronous constructs. Research on ownership types and alias control emphasizes that unrestricted mutable sharing substantially complicates reasoning about correctness in concurrent systems [36]. Finally, the low *FSVI* value reflects the absence of native Design by Contract mechanisms in the language specification, since preconditions, postconditions, and invariants rely primarily on external libraries or runtime assertions rather than first-class language constructs. As argued by Meyer, the absence of specification-level contracts limits the possibility of integrating correctness reasoning directly into the object model itself [37]. Overall, the results indicate that C# prioritizes

practical industrial flexibility and interoperability over strict encapsulation, formal verification, and state safety guarantees.

4.2.2 Comparison summary: Java

The obtained metric values indicate that Java provides a relatively mature and formally constrained object model, but one that remains strongly oriented toward classical class-based object-oriented programming. The value of $TSSI(\text{Java}) = 0.5$ reflects the hybrid nature of Java's type system: although the language separates implementation inheritance from subtype polymorphism through interfaces, its subtype model remains fundamentally nominal. This corresponds to the observations of Cardelli, who distinguishes nominal inheritance systems from structurally defined subtyping systems and notes that nominal models constrain substitutability to explicitly declared hierarchies [1]. Java's generic variance model is also intentionally conservative due to type-erasure compatibility and wildcard restrictions, which partially limits formal expressiveness despite preserving type safety [29]. At the same time, the relatively high value of $ISRI(\text{Java}) \approx 0.731$ demonstrates that the language specification places strong emphasis on deterministic inheritance semantics and hierarchy control. Java avoids multiple implementation inheritance, defines deterministic method resolution, and includes mechanisms such as final, explicit override-style semantics, and sealed hierarchies, which collectively reduce ambiguity and mitigate fragile base class effects. Such restrictions are consistent with the broader tendency in object-oriented language design to prioritize predictability and maintainability over unrestricted inheritance flexibility [8], [32].

The remaining metrics reveal several limitations of Java from the perspec-

tive of modern object-oriented expressiveness. The relatively low values of $AE(\text{Java}) = 0.3643$ and $CRFI(\text{Java}) = 0.425$ indicate that Java provides only moderate support for abstraction-oriented composition mechanisms beyond conventional interfaces. Although interfaces with default methods partially increase behavioral composability, the language lacks native trait systems, structured delegation, and explicit conflict-resolution mechanisms characteristic of more composition-oriented models [32]. The value of $ORCI(\text{Java}) \approx 0.6917$ confirms that Java preserves a strong and consistent reference-based object identity model, while also supporting several object instantiation paradigms. However, metaprogramming capabilities remain intentionally restricted due to the controlled reflection model of the JVM. This design corresponds to the argument of Bracha and Ungar that unrestricted reflection often conflicts with encapsulation guarantees and system safety [31]. The medium value of $PDEI(\text{Java}) = 0.545$ further demonstrates that Java supports only classical single dispatch, without native multiple dispatch or predicate dispatch mechanisms, thereby limiting extensibility of polymorphic behavior compared to more expressive object systems [35]. Finally, the lowest values were observed for $SCSI(\text{Java}) = 0.25$ and $FSVI(\text{Java}) = 0.275$. These results show that Java relies predominantly on programmer discipline for mutability and concurrency safety and does not provide native Design-by-Contract constructs. While runtime assertion libraries and static analyzers partially compensate for these limitations, formal specification mechanisms are not embedded into the core language semantics, unlike approaches proposed in Eiffel and contract-oriented systems [37]. Overall, the results characterize Java as a conservative and industrially stable object-oriented language whose design prioritizes compatibility, predictability, and controlled extensibility over maximal flexibility of object composition and formal behavioral specification.

4.2.3 Comparison summary: Smalltalk

The obtained metric values demonstrate that Smalltalk embodies one of the earliest and most conceptually pure object-oriented models, but at the same time lacks many of the formal safety and modularity mechanisms that later became central in statically analyzed object-oriented languages. The zero value of the TSSI metric indicates the absence of a formally specified type and subtyping system. This result is consistent with the classical description of Smalltalk as a dynamically typed pure object-oriented language in which polymorphism is based on message passing rather than static subtype relations [47]. Since the language specification does not distinguish nominal and structural subtyping and provides no formal variance model, type compatibility is resolved entirely at runtime. Such an approach significantly increases flexibility but removes compile-time guarantees discussed in type-theoretic studies by Cardelli and Pierce [1], [29]. Similarly, the extremely low AE and PDEI values reflect the historically minimalistic encapsulation model of Smalltalk. The language traditionally exposes most object structure through reflection and image-level introspection, while method dispatch relies exclusively on dynamic single dispatch without static exhaustiveness guarantees or devirtualization mechanisms.

At the same time, several metrics reveal important architectural strengths of the language. The ORCI value is comparatively high due to the consistency of reference semantics and the highly reflective metaobject environment. Smalltalk was historically one of the first languages to treat the runtime image, classes, and metaclasses as uniformly accessible objects, which corresponds to the reflective object model described in [34]. The ISRI score also demonstrates that despite the lack of formal restrictions, the inheritance model itself remains relatively

simple and deterministic because Smalltalk uses only single inheritance with a well-defined message lookup order. However, the low CRFI value shows that the language predates modern composition-oriented reuse mechanisms such as traits, mixins, and interface default implementations. This limitation became one of the motivations for later research on traits in dynamically typed object-oriented systems [32]. Finally, the zero values for SCSI and FSVI indicate that the language specification contains neither static concurrency guarantees nor native support for contracts and formal verification. Consequently, the obtained results characterize Smalltalk as a historically foundational but weakly formalized object-oriented system whose design prioritizes dynamism, reflection, and conceptual uniformity over static correctness, modular isolation, and verifiability.

4.2.4 Comparison summary: C++

The obtained metric values demonstrate that the object model of C++ remains strongly oriented toward classical implementation inheritance and low-level memory control rather than toward formally safe and semantically isolated object-oriented abstractions. The low value of the Type System and Subtyping Soundness Index ($TSSI = 0.3$) indicates that subtyping in C++ is almost entirely derived from inheritance relations, while the language lacks formal separation between subtype compatibility and implementation reuse. This observation is consistent with the analysis of Cardelli, who emphasized that inheritance and subtyping represent conceptually different mechanisms and should not be treated as equivalent relations [1]. Although C++ provides nominal typing and generic mechanisms through templates, variance rules are not formally integrated into the type system specification, which reduces predictability of generic substitutability.

Furthermore, the relatively low values of the Abstraction and Encapsulation score ($AE \approx 0.3774$) and the Inheritance Semantics Robustness Index ($ISRI \approx 0.3907$) reflect the fact that the language prioritizes flexibility over encapsulation guarantees. In particular, unrestricted reflection is absent, yet direct memory manipulation and unsafe casts allow bypassing abstraction boundaries, which weakens encapsulation integrity. Multiple inheritance additionally increases hierarchy complexity and introduces ambiguity in method resolution, a problem extensively discussed in studies on inheritance semantics and trait-based composition [8], [32].

The metrics related to composition, state safety, and formal verification reveal even more substantial limitations of the C++ object model from the perspective of robust object-oriented design. The Composition and Reuse Flexibility Index ($CRFI = 0.425$) demonstrates that although interfaces can be composed flexibly through multiple inheritance and abstract classes, the language lacks native trait systems and structured delegation mechanisms. As noted by Schärli et al., traits were specifically introduced to avoid the semantic fragility and ambiguity caused by inheritance-based reuse [32]. The low values of the Object Representation and Creation Index ($ORCI \approx 0.3417$) and the Polymorphism and Dispatch Efficiency Index ($PDEI = 0.37$) further indicate that C++ retains a predominantly class-centric and manually controlled runtime model with limited metaobject extensibility and only single-dispatch polymorphism. Most critically, the State and Concurrency Safety Index reaches the lowest result among all evaluated dimensions ($SCSI \approx 0.1667$), demonstrating the absence of immutable-by-default semantics, ownership enforcement, and compile-time guarantees of data-race freedom. This result directly corresponds to the observations of Clarke et al. regarding the importance of ownership and aliasing restrictions for safe concurrent

object manipulation [36]. Finally, the Formal Specification and Verifiability Index ($FSVI = 0.275$) confirms that Design by Contract principles are not represented as first-class language constructs in C++, correctness constraints rely mainly on external libraries and runtime assertions rather than on formally verifiable specifications, contrary to the approach proposed by Meyer [37]. Overall, the resulting metric profile characterizes C++ as a language with extensive expressive power and implementation flexibility, but with comparatively weak formal guarantees for safe, verifiable, and semantically isolated object-oriented programming.

4.2.5 Comparison summary: C

The obtained metric values demonstrate that the C programming language provides almost no native support for object-oriented abstractions and therefore serves primarily as a procedural systems language rather than an object-oriented environment. The value $TSSI(C) = 0$ reflects the absence of a formal subtype system, inheritance semantics, variance rules, and separation between inheritance and substitutability. This result is consistent with the classical distinction between procedural and object-oriented type systems described by Cardelli and Wegner, who emphasize that subtype polymorphism and inheritance relations are fundamental components of object-oriented languages [1]. Similarly, the values $ISRI(C) = 0$ and $FSVI(C) = 0$ indicate that the language specification contains no formal inheritance model, no hierarchy management mechanisms, and no native support for contracts or formal behavioral verification. Such characteristics correspond to the historical design goals of C as a portable low-level systems language prioritizing direct memory access and minimal runtime overhead rather than semantic safety or formal abstraction mechanisms.

The low values of $AE(C) \approx 0.0571$, $CRFI(C) = 0.075$, and $PDEI(C) = 0.1$ further demonstrate that encapsulation, compositional reuse, and polymorphic dispatch are not represented as first-class language concepts in C. Encapsulation is limited to translation-unit visibility through the static keyword, while module isolation and interface abstraction are absent at the language level. As noted by Parnas, effective modular decomposition requires explicit information hiding boundaries [4], which are not formally enforced in C. The value $ORCI(C) = 0.15$ indicates that although the language supports several allocation paradigms (automatic and dynamic allocation), it lacks a unified object model and reflective meta-level capabilities. The result $SCSI(C) \approx 0.1667$ also highlights that mutability is unrestricted by default and concurrency correctness depends entirely on programmer discipline, which corresponds to observations in ownership-type research emphasizing that unrestricted aliasing significantly complicates safe concurrent programming [36]. Overall, the obtained metrics confirm that C provides only primitive mechanisms from which object-oriented abstractions may be manually emulated, but the language itself does not formally support the semantic guarantees expected from modern object-oriented systems.

4.2.6 Comparison summary: Golang

The obtained metric values demonstrate that Go implements a deliberately simplified object model focused on composition, interface-based abstraction, and implementation minimalism rather than on classical object-oriented extensibility. The relatively high value of $TSSI(\text{Golang}) \approx 0.6334$ indicates that the language partially separates subtyping from inheritance through its interface system. Go does not support implementation inheritance, while interfaces are satisfied im-

plicitly, which creates a form of structural subtyping independent from class hierarchies. Such separation corresponds to the distinction between inheritance and subtype polymorphism discussed by Cardelli and Wegner [1]. At the same time, the absence of variance formalization and the intentionally restricted generic system reduce the overall expressiveness of the type model. The relatively high value of $\mathit{ISRI}(\text{Golang}) \approx 0.7167$ is primarily explained by the absence of multiple inheritance and by deterministic method resolution semantics. Since Go eliminates inheritance hierarchies entirely, it avoids a substantial portion of ambiguity traditionally associated with object-oriented inheritance systems and the fragile base class problem. However, this simplification is achieved by reducing inheritance expressiveness rather than by introducing advanced hierarchy management mechanisms.

The metrics related to composition, polymorphism, and formal correctness demonstrate the limitations of Go as a fully object-oriented language. The low values of $\mathit{CRFI}(\text{Golang}) = 0.25$ and $\mathit{PDEI}(\text{Golang}) = 0.245$ show that the language provides only minimal support for reusable behavioral composition and advanced dispatch mechanisms. Go lacks traits, mixins, multiple dispatch, and native delegation constructs, relying almost exclusively on interfaces and explicit composition patterns. This design reflects the Go philosophy of simplicity and explicitness introduced by Pike et al. in the original language design discussions [22]. Similarly, the low values of $\mathit{FSVI}(\text{Golang}) = 0.275$ and $\mathit{SCSI}(\text{Golang}) = 0.3$ indicate that Go prioritizes pragmatic concurrency support over formal verification guarantees. Although the language includes high-level concurrency primitives, it does not provide static race-freedom guarantees, ownership-based aliasing control, or native contract mechanisms. The moderate value of $\mathit{ORCI}(\text{Golang}) \approx 0.5917$ additionally reflects that Go supports several object construction styles and runtime

introspection through reflection, but intentionally restricts metaobject extensibility and runtime intercession. Overall, the collected metrics confirm that Go represents a minimalistic and composition-oriented reinterpretation of object-oriented programming rather than a feature-rich object-oriented model.

4.2.7 Comparison summary: Scala

The obtained results demonstrate that Scala provides one of the most formally expressive object models among contemporary mainstream object-oriented languages. The high value of $TSSI = 0.8$ indicates that the language combines nominal and structural typing while also supporting formally specified variance annotations for parametric polymorphism. Such a result reflects Scala's orientation toward mathematically rigorous type safety and flexible subtype relations. The language specification explicitly separates several subtype mechanisms from direct implementation inheritance through traits, abstract type members, and structural types, although this separation is not entirely independent semantically, which explains the partial reduction of the S_{rel} component. This observation is consistent with the description of Scala's type system provided by Odersky et al., where the language is characterized as a synthesis of object-oriented and functional abstraction mechanisms with a highly expressive static type system [38]. In addition, the relatively high $ISRI = 0.8167$ confirms that Scala introduces strong hierarchy-management mechanisms, including deterministic linearization, explicit override semantics, and restrictions aimed at reducing fragile base class effects. The trait linearization model follows deterministic composition semantics similar to those discussed in trait-oriented research by Schärli et al. [32].

The results also show that Scala strongly emphasizes compositional reuse

and abstraction flexibility. The value $CRFI = 0.6875$ reflects extensive support for trait-based composition and interface-oriented behavioral reuse, which substantially reduces dependence on classical inheritance hierarchies. Traits in Scala act not only as contracts but also as reusable behavioral units, corresponding to the compositional approach proposed in trait research [32]. Similarly, the relatively high $ORCI = 0.6917$ demonstrates support for several object creation paradigms and a fully reference-oriented object model. However, the moderate $PDEI = 0.62$ indicates that Scala remains largely based on single dispatch despite advanced pattern matching facilities. More significant limitations become visible in the areas of encapsulation enforcement and formal verification. The low values of $AE = 0.3381$ and $FSVI = 0.275$ are primarily caused by the absence of strict runtime encapsulation guarantees and the lack of native Design by Contract constructs. While Scala provides expressive abstraction mechanisms, reflection and JVM interoperability weaken strict encapsulation boundaries. Furthermore, formal correctness specifications rely mostly on external libraries rather than language-level constructs. Finally, the moderate $SCSI = 0.4$ demonstrates that Scala only partially addresses immutability and concurrency safety at the language level: although immutable collections and actor-based concurrency models are available, the language does not enforce race freedom or immutability by default. This partially confirms the broader observation that advanced abstraction capabilities in hybrid object-functional languages do not automatically imply strong guarantees of state safety or formal verifiability [36], [37].

4.2.8 Comparison summary: JavaScript

The obtained metric values demonstrate that JavaScript provides only limited support for classical object-oriented semantics despite offering object-based programming constructs. The low value of $TSSI = 0.2$ reflects the absence of a formally specified static type system and the lack of explicit subtype relations in the language specification. As noted by Cardelli, subtype reasoning requires formally defined substitutability relations independent from implementation reuse [1], whereas JavaScript historically relies on dynamic prototype delegation instead of explicit subtyping. Similarly, the relatively weak values of $AE = 0.3071$ and $FSVI = 0.275$ indicate insufficient language-level support for strict encapsulation and formal specification. Although ECMAScript modules provide explicit export boundaries, the language still exposes reflective runtime mechanisms and lacks native Design by Contract constructs. Meyer emphasized that reliable object-oriented abstraction requires explicit contractual semantics and controlled visibility mechanisms [30], [37], which JavaScript only partially achieves through conventions and runtime checks. The extremely low values of $CRFI = 0.075$, $PDEI = 0.07$, and $SCSI = 0.1$ further demonstrate that JavaScript prioritizes flexibility and dynamic adaptation over formal correctness guarantees. In particular, the language lacks native trait systems, multiple dispatch, static exhaustiveness guarantees, and compile-time race prevention mechanisms.

At the same time, JavaScript achieves one of the highest values for object representation flexibility, with $ORCI = 0.925$. This result reflects the dynamic and highly extensible nature of the ECMAScript object model. The language supports multiple object creation paradigms, including constructor-based instantiation, object literals, and prototype cloning, while also exposing powerful meta-

programming facilities through reflective APIs and prototype manipulation. Such characteristics correspond to the metaobject-oriented ideas described by Kiczales et al. [34]. The moderate value of $ISRI = 0.4917$ additionally indicates that JavaScript avoids some classical inheritance complexity problems due to its prototype-based model and deterministic property lookup semantics, but simultaneously lacks formal safeguards against fragile base hierarchy effects. Overall, the metric profile confirms conclusions commonly discussed in research on dynamic languages: JavaScript provides high runtime flexibility and extensibility, but achieves this at the cost of weak formal guarantees, limited encapsulation safety, and reduced suitability for constructing large-scale verifiable object-oriented systems.

4.2.9 Comparison summary: Python

The obtained metric values demonstrate that Python provides a comparatively flexible and expressive object model, but this flexibility is achieved at the expense of strict semantic guarantees and formal safety mechanisms. The high value of the Type System and Subtyping Soundness Index ($TSSI = 0.8583$) reflects Python's hybrid typing approach, which combines nominal inheritance with structural typing through protocols introduced in PEP 544. This confirms the observation of Cardelli that structural and nominal subtyping solve different compatibility problems and may coexist within a single type system [1]. At the same time, the relatively high Object Representation and Creation Index ($ORCI = 0.85$) indicates the expressive meta-object capabilities of Python, including runtime reflection and dynamic object manipulation, which correspond to the principles of reflective systems described by Kiczales et al. [34]. The moderate values of

the Abstraction and Encapsulation score ($AE = 0.5207$) and Composition and Reuse Flexibility Index ($CRFI = 0.5125$) show that Python supports interfaces, mixin-oriented reuse, and delegation patterns, but lacks strong encapsulation barriers and formalized trait composition semantics. This observation is consistent with research on traits and compositional reuse, where explicit conflict resolution and formal requirements are considered necessary for scalable composition mechanisms [32].

However, the remaining metrics reveal substantial limitations of Python from the perspective of robust object-oriented system design. The low value of the Inheritance Semantics Robustness Index ($ISRI = 0.3417$) reflects the complexity introduced by unrestricted multiple inheritance and the absence of strong compile-time hierarchy constraints. Although Python formally specifies deterministic method resolution order through C3 linearization, it provides only limited protection against fragile base class problems and accidental overriding. The extremely low value of the Polymorphism and Dispatch Efficiency Index ($PDEI = 0.07$) demonstrates that Python relies almost entirely on conventional single dispatch without compile-time dispatch verification or optimization-oriented hierarchy restrictions. Similarly, the low State and Concurrency Safety Index ($SCSI = 0.1$) confirms the absence of language-level guarantees for immutability, aliasing control, or data-race freedom, forcing correctness to depend primarily on programmer discipline rather than static enforcement mechanisms. Finally, the Formal Specification and Verifiability Index ($FSVI = 0.275$) indicates that Python does not provide native Design by Contract facilities and relies mostly on external libraries or runtime assertions. This contrasts with the Design by Contract approach proposed by Meyer, where contracts are treated as first-class semantic elements of the language specification [37]. Overall, the results characterize Python as a lan-

guage optimized for flexibility, runtime adaptability, and rapid software evolution rather than for formally verifiable and semantically constrained object-oriented architecture.

4.2.10 Comparison summary: Ruby

The obtained metric values demonstrate that Ruby prioritizes runtime flexibility and metaprogramming expressiveness over static correctness guarantees and formally constrained object-oriented semantics. The low value of the TSSI metric (0.15) indicates that Ruby lacks a formally expressive subtype system: the language follows a dynamically typed object model without explicit variance semantics or formal separation between inheritance and subtyping. This observation corresponds to the classical distinction between inheritance and subtype polymorphism discussed by Cardelli and Wegner, who emphasize that dynamic object systems often conflate implementation reuse with substitutability [1]. Similarly, the low AE value (0.2107) reflects Ruby's intentionally permissive encapsulation model. Although the language provides several visibility modifiers ($\mathrm{private}$, $\mathrm{protected}$, public), these mechanisms remain advisory rather than strictly enforced, since reflection and metaprogramming allow unrestricted runtime access to object internals. Such behavior is consistent with the reflective philosophy described in metaobject protocol research [34]. The absence of strict modular isolation and native contract mechanisms further reduces the FSVI score (0.275), as Ruby relies primarily on runtime assertions and external libraries instead of language-level specification constructs.

At the same time, Ruby demonstrates comparatively strong results in flexibility-oriented metrics. The high ORCI value (0.925) reflects the language's highly

dynamic object representation model, extensive metaprogramming facilities, and support for multiple object creation paradigms. Ruby's object system allows runtime structural modification, singleton methods, and dynamic class extension, closely aligning with reflective object model approaches described in [34]. The CRFI metric (0.5125) also indicates moderate support for composition-oriented reuse through modules and mixins, which partially mitigate limitations of single inheritance. However, inheritance-related robustness remains limited despite deterministic method lookup semantics. The ISRI value (0.4917) shows that Ruby benefits from single inheritance simplicity and deterministic linearization, but lacks stronger hierarchy restriction mechanisms such as sealed hierarchies, mandatory override annotations, or non-virtual-by-default semantics. Particularly significant are the extremely low PDEI (0.07) and SCSI (0) values, which reveal the absence of static dispatch safety guarantees, exhaustive polymorphism checking, ownership constraints, immutability guarantees, and compile-time concurrency safety mechanisms. This confirms that Ruby's object model is optimized primarily for developer productivity and runtime adaptability rather than for formal correctness, static verifiability, or safe large-scale composition.

4.2.11 Comparison summary: Zonnon

The obtained metric values for the Zonnon language demonstrate a heterogeneous level of maturity of the object model. The strongest side of the language is the abstraction and encapsulation subsystem ($AE \approx 0.8232$), which indicates well-developed support for modularity, clear separation of interface and implementation, as well as strict access control. This result is consistent with the classical principles of modular design [4], as well as with the concept of encapsu-

lation as a key mechanism for managing complexity in OOP [30]. Additionally, the high ISRI score (≈ 0.7667) indicates a well-formalized inheritance semantics, including a deterministic method resolution order and partial hierarchy protection mechanisms. This confirms the thesis about the need for a strict distinction between reuse and subtyping mechanisms, formulated in the work [8], which notes that inheritance and subtyping have different semantics, and mixing these concepts leads to architectural limitations.

At the same time, a number of key aspects of the object model are implemented significantly less well. Low values of CRFI (0.25) and SCSI (≈ 0.3667) indicate limited support for composition and insufficient guarantees of state security and competitiveness. The lack of expressive traits/mixins mechanisms and automatic delegation reduces the flexibility of reuse, which contradicts modern approaches that consider composition as a preferred alternative to inheritance [32]. Similarly, weak guarantees of immutability and link control confirm the findings of ownership and aliasing studies, which emphasize that the absence of strict restrictions leads to competitive access errors [36]. The moderate values of TSSI (0.55), ORCI (≈ 0.4417), and PDEI (0.445) additionally show that the language only partially implements the formal aspects of the model system, polymorphism, and object model. In particular, the lack of formalization of variation and limited dispatch mechanisms contradict the results of fundamental research of standard systems, where strict formalization is a necessary condition for security [29]. Taken together, the results show that Zonnon demonstrates a strong architectural discipline at the level of modularity and inheritance, but is inferior to modern languages in composition flexibility, formal rigor of the standard system and guarantees of execution safety, which confirms the need to develop a more balanced object model within the framework of the proposed language.

Chapter 5

Conclusion

The conducted comparative analysis demonstrates that modern programming languages implement object-oriented principles through fundamentally different architectural approaches, each prioritizing a distinct balance between flexibility, formal safety, extensibility, runtime dynamism, and implementation simplicity. The calculated metrics revealed that no analyzed language provides a universally balanced object model across all considered dimensions. Languages oriented toward industrial stability and static correctness demonstrate stronger inheritance control and subtype formalization, while dynamically oriented systems provide higher flexibility of object representation and metaprogramming at the cost of weaker encapsulation and verification guarantees. At the same time, composition-oriented models reduce the structural complexity of inheritance hierarchies, but often sacrifice expressive polymorphism and formal abstraction mechanisms. The obtained results confirm that many classical problems of object-oriented programming are not caused by OOP itself, but rather by inconsistencies between inheritance semantics, subtype relations, mutability models, and runtime extensibility mechanisms.

The analysis also demonstrates several common architectural tendencies in the evolution of modern object systems. First, there is a gradual transition from inheritance-centered reuse toward composition-based models using interfaces, traits, mixins, and delegation. Second, contemporary languages increasingly attempt to separate subtype polymorphism from implementation inheritance, confirming the theoretical distinction established in type-system research. Third, there is a growing emphasis on deterministic hierarchy semantics, explicit override control, and restrictions intended to mitigate fragile base class effects. At the same time, the study revealed that mechanisms related to formal verification, immutability guarantees, concurrency safety, and contract-based specification remain insufficiently integrated into the majority of modern object-oriented languages. In most cases, such guarantees rely on external libraries, conventions, or runtime discipline rather than on the core semantics of the language itself.

Based on the identified limitations and recurring design trade-offs, this thesis proposes the conceptual foundation for a unified object model intended to combine strict semantic separation of inheritance and subtyping, composition-oriented reuse, deterministic polymorphic behavior, controlled metaprogramming, and stronger guarantees of state safety and formal verifiability. The proposed direction aims to preserve the expressive flexibility of object-oriented programming while reducing architectural ambiguity and improving correctness guarantees at the language level. Consequently, the results of this work may serve both as a systematic framework for further comparative studies of object-oriented languages and as a theoretical basis for the development of future programming languages with more balanced and formally robust object models.

Bibliography cited

[1] L. Cardelli and P. Wegner, "On understanding types, data abstraction, and polymorphism," *ACM Computing Surveys (CSUR)*, vol. 17, no. 4, pp. 471–523, 1985.

[2] G. Booch, R. A. Maksimchuk, M. W. Engle, B. J. Young, J. Connallen, and K. A. Houston, "Object-oriented analysis and design with applications," *ACM SIGSOFT software engineering notes*, vol. 33, no. 5, pp. 29–29, 2008.

[3] B. Liskov, "Keynote address-data abstraction and hierarchy," in *Addendum to the proceedings on Object-oriented programming systems, languages and applications (Addendum)*, 1987, pp. 17–34.

[4] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

[5] B. H. Liskov and J. M. Wing, "Behavioral subtyping using invariants and constraints," Tech. Rep., 1999.

[6] E. Gamma, *Design patterns: elements of reusable object-oriented software*. Addison-Wesley, 1995, vol. 431.

[7] M. Atkinson and R. Morrison, "Orthogonally persistent object systems," *The VLDB Journal*, vol. 4, no. 3, pp. 319–401, 1995.

[8] W. R. Cook, W. Hill, and P. S. Canning, "Inheritance is not subtyping," in *Proceedings of the 17th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1989, pp. 125–135.

[9] B. C. Pierce and D. N. Turner, "Simple type-theoretic foundations for object-oriented programming," *Journal of functional programming*, vol. 4, no. 2, pp. 207–247, 1994.

[10] L. Mikhajlov and E. Sekerinski, "A study of the fragile base class problem," in *European Conference on Object-Oriented Programming*, Springer, 1998, pp. 355–382.

[11] M. Hitz and B. Montazeri, *Measuring coupling and cohesion in object-oriented systems*. na, 1995.

[12] P. Graham, *The Hundred-Year Language*, https://www.paulgraham.com/hundred.html, [Online; accessed 27-11-2025], Apr. 2003.

[13] J. C. Reynolds, "Three approaches to type structure," in *Colloquium on Trees in Algebra and Programming*, Springer, 1985, pp. 97–138.

[14] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, "Design patterns: Abstraction and reuse of object-oriented design," in *European conference on object-oriented programming*, Springer, 1993, pp. 406–431.

[15] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Transactions on software engineering*, vol. 20, no. 6, pp. 476–493, 1994.

[16] P. Wegner, "Dimensions of object-based language design," *ACM Sigplan Notices*, vol. 22, no. 12, pp. 168–182, 1987.

[17] M. Vandeloise, "Towards a unified framework for programming paradigms: A systematic review of classification formalisms and methodological foundations," *arXiv preprint arXiv:2508.00534*, 2025.

[18] W. Crichton, G. Gray, and S. Krishnamurthi, "A grounded conceptual model for ownership types in rust," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA2, pp. 1224–1252, 2023.

[19] D. J. Armstrong, "The quarks of object-oriented development," *Communications of the ACM*, vol. 49, no. 2, pp. 123–128, 2006.

[20] L. Prechelt, "An empirical comparison of seven programming languages," *Computer*, vol. 33, no. 10, pp. 23–29, 2002.

[21] M. S. Farooq et al., "Comparative analysis of widely use object-oriented languages," *arXiv preprint arXiv:2306.01819*, 2023.

[22] R. Pike, "Go at google: Language design in the service of software engineering. online," *URL: https://talks.golang.org/2012/splash.article*, 2012.

[23] I. Balbaert, *The way to Go: A thorough introduction to the Go programming language*. IUniverse, 2012.

[24] M. Odersky and T. Rompf, "Unifying functional and object-oriented programming with scala," *Communications of the ACM*, vol. 57, no. 4, pp. 76–86, 2014.

[25] J. Gutknecht and E. Zueff, "Zonnon language report," *Zurich, Institute of Computer Systems ETH Zentrum*, 2004.

[26] R. W. Sebesta, *Concepts of programming languages*. Pearson Education India, 2016.

[27] *Iso/iec 25010: Systems and software engineering – system and software product quality models*, 2023.

[28] T. R. Fayzrakhmanov, "Introducing programming language metrics," *Труды Института системного программирования РАН*, vol. 34, no. 6, pp. 67–84, 2022.

[29] B. C. Pierce, *Types and programming languages*. MIT press, 2002.

[30] B. Meyer, *Object-oriented software construction*. Prentice hall Englewood Cliffs, 1997, vol. 2.

[31] G. Bracha and D. Ungar, "Mirrors: Design principles for meta-level facilities of object-oriented programming languages," *ACM SIGPLAN Notices*, vol. 39, no. 10, pp. 331–344, 2004.

[32] N. Schärli, S. Ducasse, O. Nierstrasz, and A. P. Black, "Traits: Composable units of behaviour," in *European Conference on Object-Oriented Programming*, Springer, 2003, pp. 248–274.

[33] G. Bracha, *Generics in the java programming language*, 2004.

[34] G. Kiczales, J. Des Rivieres, and D. G. Bobrow, *The art of the metaobject protocol*. MIT press, 1991.

[35] G. Castagna, "Covariance and contravariance: Conflict without a cause," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 17, no. 3, pp. 431–447, 1995.

[36] D. G. Clarke, J. M. Potter, and J. Noble, "Ownership types for flexible alias protection," in *Proceedings of the 13th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, 1998, pp. 48–64.

[37] B. Meyer, "Applying'design by contract'," *Computer*, vol. 25, no. 10, pp. 40–51, 2002.

[38] M. Odersky et al., "An overview of the scala programming language," 2004.

[39] M. Abadi and L. Cardelli, *A theory of objects*. Springer Science & Business Media, 2012.

[40] P. Wadler et al., "The expression problem," *Posted on the Java Genericity mailing list*, vol. 7, p. 36, 1998.

[41] H.-J. Boehm and S. V. Adve, "Foundations of the c++ concurrency memory model," *ACM SIGPLAN Notices*, vol. 43, no. 6, pp. 68–78, 2008.

[42] J. Bloch, *Effective java*. Addison-Wesley Professional, 2017.

[43] Ecma International, "Ecma-334: C# language specification," Ecma International, Standard ECMA-334, 7th edition, 2023, December 2023. [Online]. Available: https://ecma-international.org/publications-and-standards/standards/ecma-334/

[44] M. Barnett, K. R. M. Leino, and W. Schulte, "The spec# programming system: An overview," in *International Workshop on Construction and Analysis of Safe, Secure, and Interoperable Smart Devices*, Springer, 2004, pp. 49–69.

[45] J. Gosling, "The java language specification: Java se 8 edition," *(No Title)*, 2015.

[46] M. Buchholz, J. C. Czar, D. Rajwan, T. Neward, and K. Pepperdine, "Java concurrency in practice,"

[47] A. Goldberg and D. Robson, *Smalltalk-80: the language and its implementation*. 1983.

[48] A. Black, S. Ducasse, O. Nierstrasz, D. Pollet, D. Cassou, and M. Denker, "Pharo by example. square bracket associates, 2009," *URL http://pharobyexample.org*,

[49] B. Stroustrup, *The design and evolution of C++*. Pearson Education India, 1994.

[50] C. S. Committee et al., "Iso international standard iso/iec 14882: 2017, programming language c++," *Geneva, Switzerland: International Organization for Standardization (ISO)., Tech. Rep*, 2017.

[51] D. Abrahams and A. Gurtovoy, *C++ template metaprogramming: concepts, tools, and techniques from Boost and beyond*. Pearson Education, 2004.

[52] D. M. Ritchie, S. C. Johnson, M. Lesk, B. Kernighan, et al., "The c programming language," *Bell Sys. Tech. J*, vol. 57, no. 6, pp. 1991–2019, 1978.

[53] International Organization for Standardization and International Electrotechnical Commission, "ISO/IEC 9899:2024 programming languages — c," ISO/IEC, International Standard ISO/IEC 9899:2024, 2024.

[54] Google Inc., *The go programming language specification*, https://go.dev/ref/spec, Accessed: 2026-05-04, 2024.

[55] B. Emir, M. Odersky, and J. Williams, "Matching objects with patterns," in *European Conference on Object-Oriented Programming*, Springer, 2007, pp. 273–298.

[56] Ecma International and TC39. "Ecmascript® language specification." Living standard, accessed: 2026-03-29. [Online]. Available: https://tc39.es/ecma262/

[57] Python Software Foundation, *Pep 544 – protocols: Structural subtyping (static duck typing)*, 2017. [Online]. Available: https://peps.python.org/pep-0544/

[58] G. van Rossum, J. Lehtosalo, and L. Langa, *Pep 484 – type hints*, https://peps.python.org/pep-0484/, Python Enhancement Proposal, Status: Final, accessed: 2026-03-29, 2015.

[59] D. Flanagan and Y. Matsumoto, *The Ruby Programming Language: Everything You Need to Know*. O'Reilly Media, Inc., 2008.